

Software Engineering

Unit 3 – Lecture 1

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Course Coordinator and Anchor (RR Campus): Prof Vadiraja Acharya
Course Anchor (EC Campus) : Prof M Sheela Devi

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

AI For Software Engineering

Capabilities of AI Now



https://www.youtube.com/watch?v=_dZoscOdDkg



https://openai.com/sora/



https://www.youtube.com/watch?v=_dZoscOdDkg

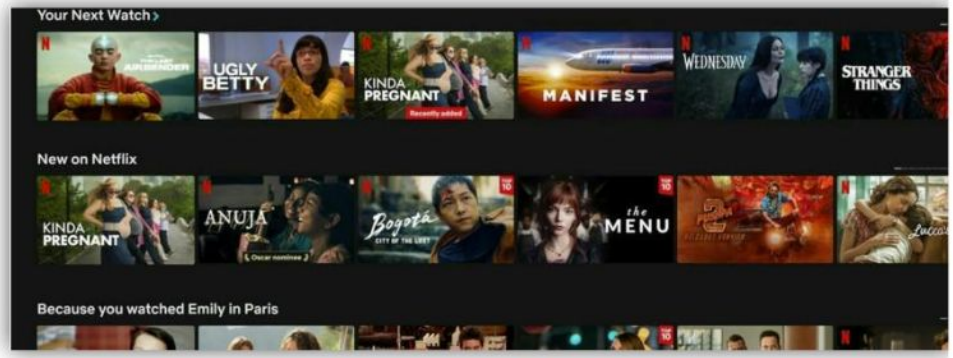
Capabilities of AI Now



Virtual Assistants



Fraud Detection



Recommendation Systems

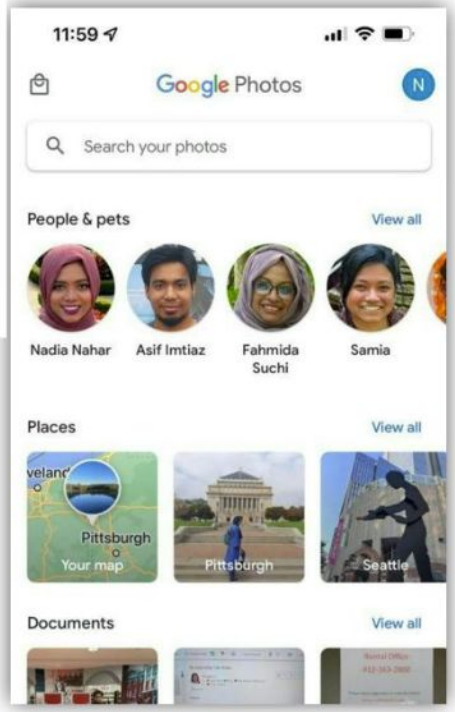


Image Recognition

Capabilities of AI Now

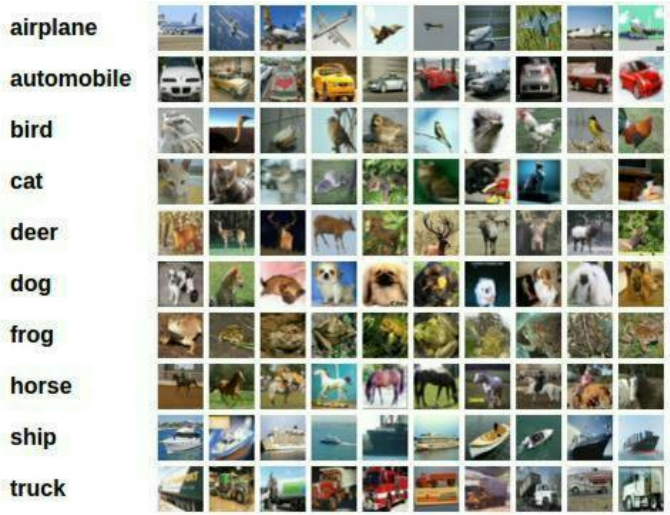


AI Assists in Healthcare Diagnostics



Autonomous Vehicles

Machine Learning in One Slide - (Supervised)



Lots of labelled data
(Inputs, outputs)

Training

Model



Input

bird”

Output



Input

dog”

Output

Tradeoffs

- Accuracy
- Capabilities (e.g. classification, recommendation, clustering…)
- Amount of training data needed
- Inference latency
- Learning latency
- Model size
- Explainable

Black-box View of ML

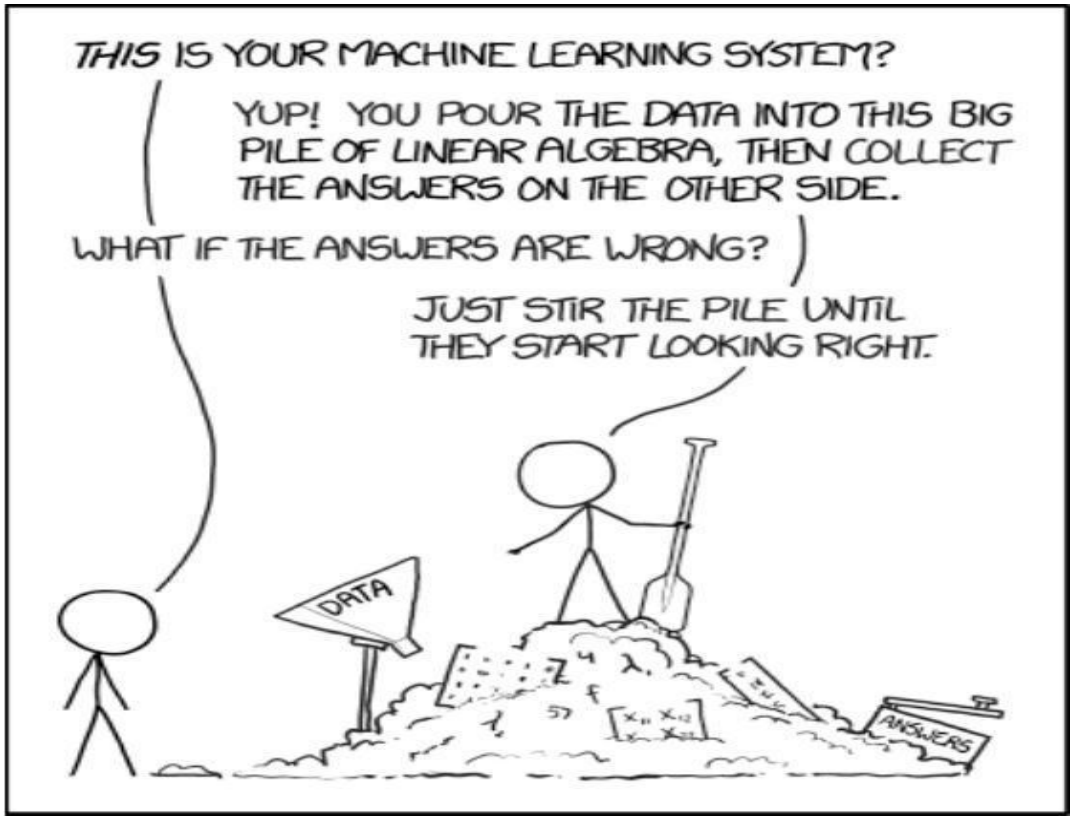


Image: https://xkcd.com/1838/

SE vs ML

- ML is more data-focused
  - Relies heavily on data to train models; data preprocessing is crucial
- ML is more experimental
  - Experiment-driven with model training, testing, and refinement based on empirical data.
- SE is more structured or process-oriented
  - Structured methodologies (e.g., Agile, Waterfall) guiding the development lifecycle from design to deployment
- ML is more algorithmic focused
  - Priority on development of algorithms (e.g., supervised, unsupervised learning) for pattern recognition.
- The concept of evaluation is very different
  - Functional correctness vs accuracy

SE & ML: Connected in Two Ways

- Using ML for Software Engineering
  - How to use AI for SE
  - AI For SE
- Software Engineering for ML Systems
  - How to integrate AI components in SW Systems
  - SE For AI

AI in Software Development

A high level look at AI, ML, DL and GenAI
https://www.youtube.com/watch?v=qYNweeDHiyU

Gen AI v/s AI Agents v/s Agentic AIs
https://www.youtube.com/watch?v=O2gerCxEXvc

Prompt engineering guide
https://www.youtube.com/watch?v=riO3e-FBieA

Learning Software Engineering During the Era of AI | Raymond Fu | TEDxCSTU
https://www.youtube.com/watch?v=w4rG5GY9IlA

AI in Software Engineering

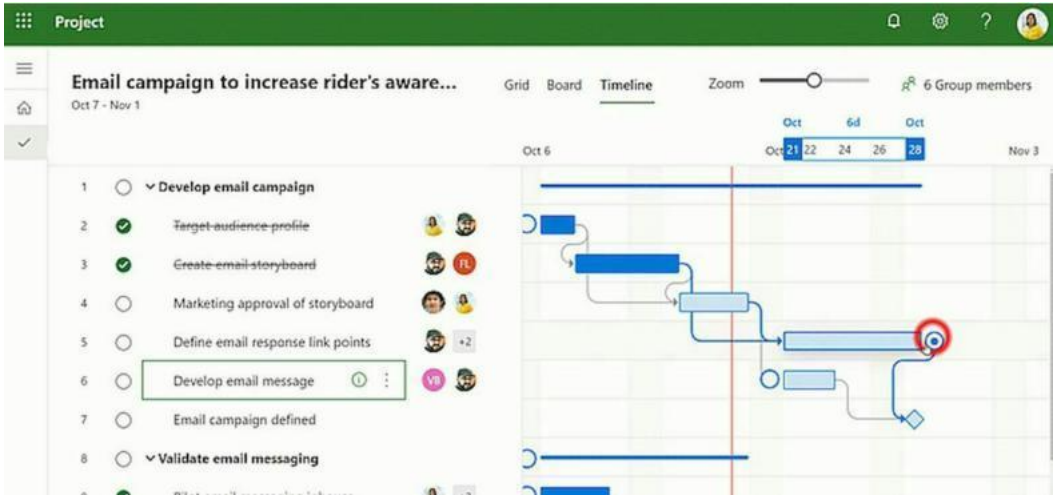
AI offers tools and techniques that enhance efficiency, creativity and the overall development process. Generative AI is driving key advancements by automating tasks and boosting productivity.

AI is a force-multiplier—accelerating routine tasks and augmenting human judgment—rather than a wholesale replacement of the SDLC.

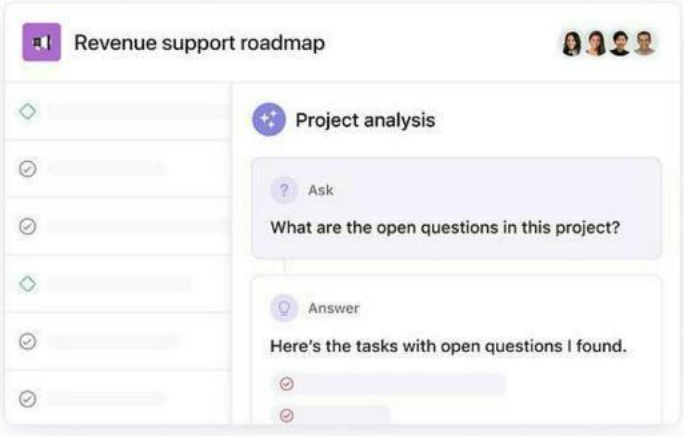
- Code generation
- Bug detection and fixing
- Testing automation
- Project management
- Documentation
- Refactoring and optimization
- Security enhancement
- DevOps and CI/CD pipelines
- UX design
- Architecture design

Project Management

- ML models analyze historical project data to forecast timelines, determine resource allocation, and predict budgetary needs, aiding in proactive decision-making.



Microsoft Project



Asana Intelligence

Requirements Engineering

- **Old-fashioned**: Stakeholder interviews, manual document drafts, lengthy review cycles.
- **With AI**:
  - **Automated elicitation aids** (e.g., chatbots that ask probing questions).
  - **Language models** that generate first-draft requirement specifications from user stories or transcripts.
  - **Gap analysis** by comparing new requirements against a repository of past projects.

LLM generated Example – College Event Mgmt App

- “AI-generated” requirements draft (Based on Summary prompt)
  - Users can browse upcoming events.
  - Students and faculty can register for events.
  - Organizers can create and edit events.
  - Participants can cancel registrations.
  - Notifications will be sent for upcoming events.
- Analyze and identify 2 missing requirements and 1 incomplete requirement
- Reflection
  - “Why might the AI have missed these?”
  - “What if these gaps are not caught before development?”
  - Write one concise sentence about the necessity of human review for requirements, even with AI tools.

You still need to validate requirements with real stakeholders, resolve conflicts, and manage scope—AI can suggest, but human judgment must accept or reject.

Architecture & Design

- **Old-fashioned**: White-boarding, UML diagrams sketched by hand, pattern catalogues consulted manually.
- **With AI**:
  - **Design-pattern recommendation engines** that suggest candidate architectures given functional and non-functional constraints.
  - **Automated diagram generation** from textual descriptions.
  - **Architecture risk analysis** tools that flag single points of failure or security exposures.

LLM generated Example – College Event Mgmt App

- AI generated Architecture choice:
  - “Implement the app with a client-server architecture where the client is a web or mobile app interacting directly with a single monolithic server that handles all event management, user registration, notification dispatching, and data persistence.”
- Identify at least three potential issues or missing considerations in the AI’s architecture
- Suggest **improvements or extensions** that would address these limitations

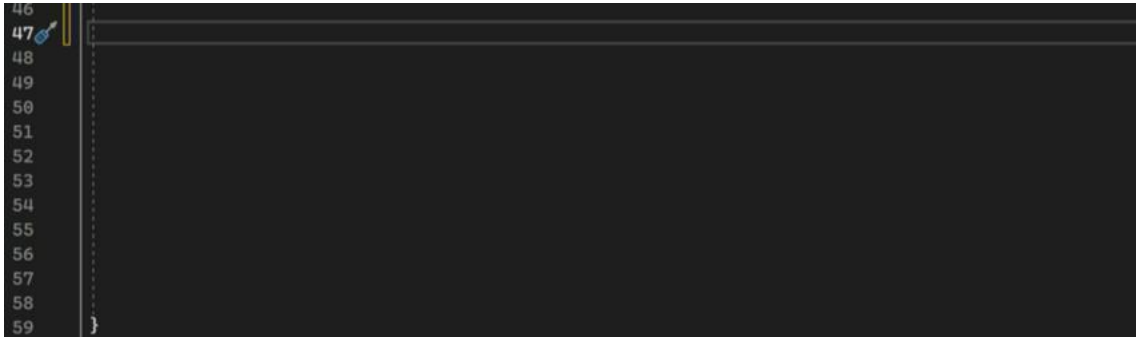
You still need to weigh trade-offs (performance vs. maintainability, monolith vs. microservices) and perform peer reviews to ensure the chosen design fits your context.

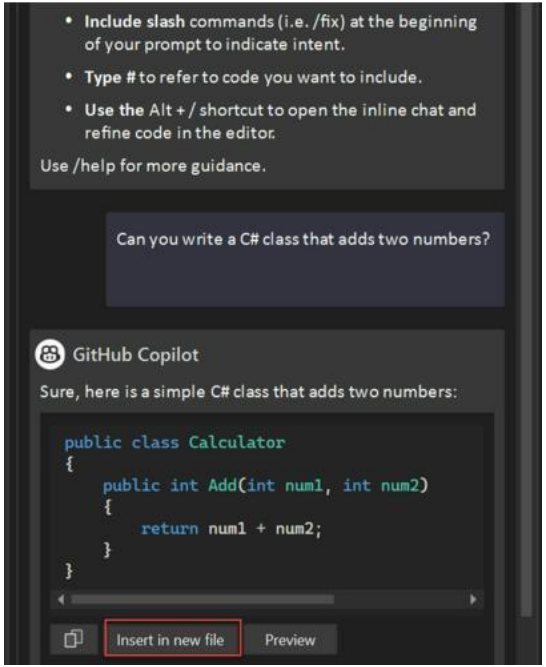
Implementation & Coding

- **Old-fashioned**: Developers write every line from scratch, relying on IDE support for syntax and basic refactoring.
- **With AI**:
  - **AI pair-programmers** (e.g., GitHub Copilot) that autocomplete code, suggest entire functions, and translate pseudo-code into working implementations.
  - **Automated code-style enforcers** that refactor for consistency.

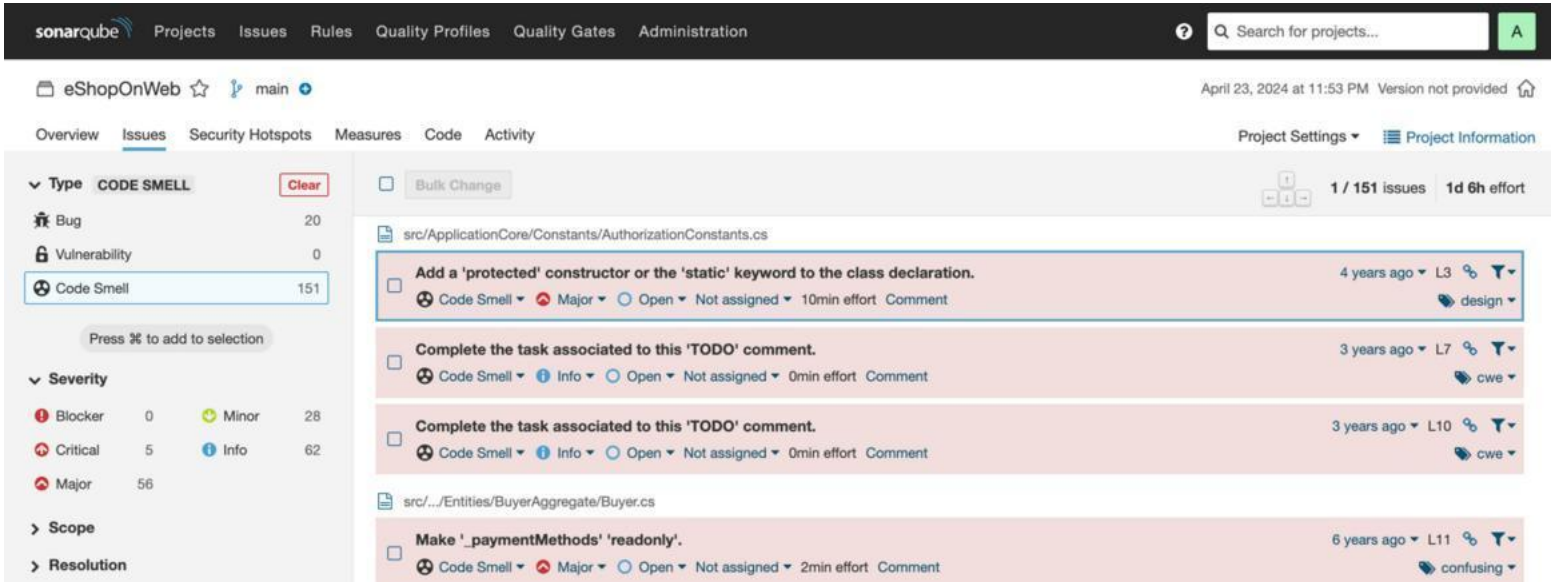
Code Generation and Assistance







Automates Code Reviews



LLM generated Example – College Event Mgmt App

- AI generated sample code:

```
void process_event_name(char *input) {
    char event_name[50];
    // Copy input into event_name buffer
    strcpy(event_name, input);
    printf("Processing event: %s\n", event_name);
}
```

- Identify security vulnerabilities
- Suggest mitigations

You still need to understand the code you accept, ensure it meets business logic, enforce security best practices, and avoid “black-box” dependencies that you cannot maintain.

Testing & Quality Assurance

- **Old-fashioned**: Manually writing unit/integration tests, hand-crafted test cases, and ad-hoc bug hunts.
- **With AI**:
  - **Test-case generators** that infer edge cases from your code and produce unit tests automatically.
  - **AI-driven fuzzers** that dynamically explore unusual inputs.
  - **Automated code-coverage analyses** with remediation suggestions.

LLM generated Example – College Event Mgmt App

- AI-Generated Test Cases

```
def test_valid_registration():
    assert register_event("Spring Fest", "JohnDoe") == True
def test_empty_event_name():
    assert register_event("", "JohnDoe") == False
def test_empty_user():
    assert register_event("Spring Fest", "") == False
def test_event_name_too_long():
    long_name = "x" * 500
    assert register_event(long_name, "JohnDoe") == False
```

- Identify possible gaps or risks in the AI-generated tests
- Suggest additional test cases or improvements to strengthen coverage.

You still need to define acceptance criteria, interpret test results, handle false positives/negatives, and ensure that business-critical scenarios are explicitly tested.

Automated Testing

*Very active research area

- ML-based test generation
  - Generate test cases intelligently by analyzing code changes, defect history, and user behaviors, improving test coverage and efficiency
- Designing effective metrics
  - Develop metrics to evaluate test effectiveness and prioritize testing efforts.
- Intelligent orchestration
  - Use ML to prioritize and orchestrate test execution efficiently.
- Enhancing CI pipeline
  - Integrate AI/ML to streamline and enhance the continuous integration process.









Deployment & DevOps

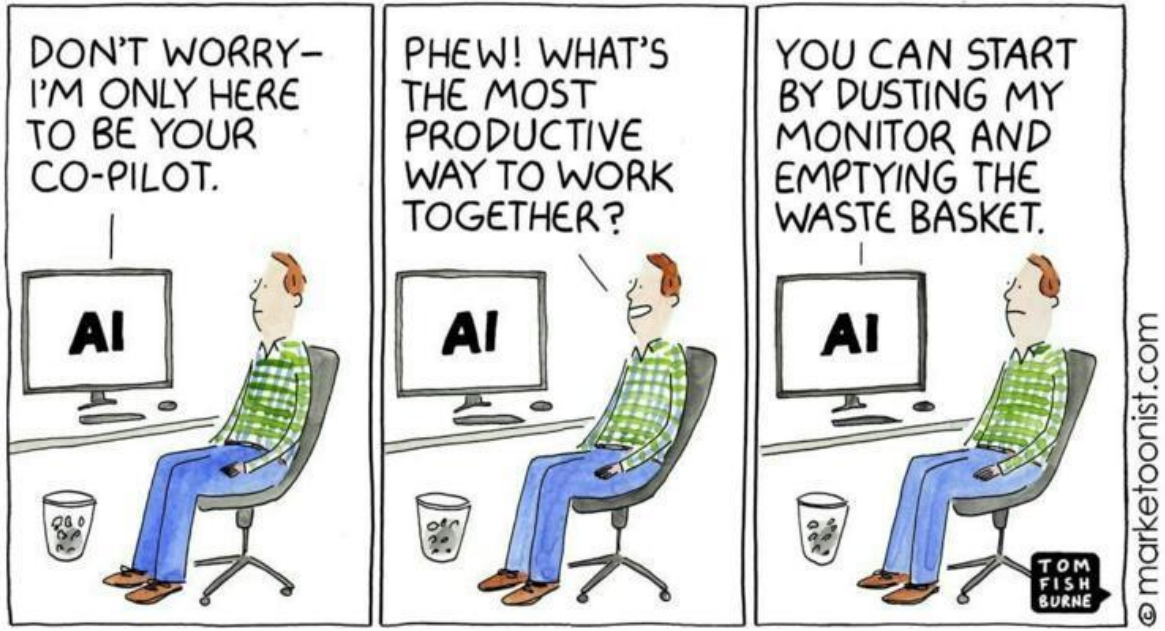
- **Old-fashioned**: Hand-written deployment scripts, manual approvals, and reactive monitoring.
- **With AI**:
  - **Pipeline optimizers** that suggest improvements to CI/CD flows based on historical run data.
  - **Predictive anomaly detection** in monitoring dashboards to catch performance regressions before they impact users.
  - **Automated rollback decision support** when a deployment metric crosses a safety threshold.
- **Fundamental**: You still need to set your service-level objectives (SLOs), define rollback policies, and cultivate a culture of “you broke it, you fix it”.

Maintenance & Evolution

- **Old-fashioned**: Bug-tracking backlogs managed manually, refactoring planned on a calendar, and decision-making based on ad-hoc metrics.
- **With AI**:
  - **Bug-triage assistants** that classify and route issues to the right owners.
  - **Code-health dashboards** that predict technical-debt hotspots.
  - **Automated refactoring suggestions** (e.g., extracting methods or modules).
- **Fundamental**: You still need strategic road-mapping, prioritization against new feature work, and human oversight to balance short term fixes with long-term code health.

Benefit

- Increased productivity
- Assists new programmers



Limitation: Incorrect/ Non-optimal code



Yetistiren, Burak, Isik Ozsoy, and Eray Tuzun. "Assessing the quality of GitHub copilot's code generation." *Proceedings of the 18th international conference on predictive models and data analytics in software engineering*. 2022.

Limitation: Security

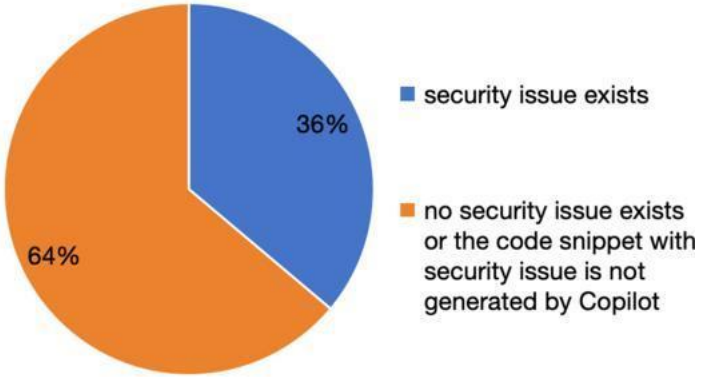
Copilot amplifies insecure codebases by replicating vulnerabilities in your projects

Written by: RD Randall Degges

February 22, 2024 11 mins read

Did you know that GitHub Copilot may suggest insecure code if your existing codebas

the other hand, if your codebase is already highly secure, Copilot is less likely to gene



Fu, Yujia, et al. "Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study." ACM Transactions on Software Engineering and Methodology (2025).

Security Issues - Additional Studies

- Around 40-45% of AI-generated code snippets contain security flaws, indicating almost half of code suggested by popular tools like GitHub Copilot, ChatGPT, and Claude introduces risks.
- Common vulnerability types include:
  - Cross-Site Scripting (XSS) — insecure 86% of time due to lack of context for proper input sanitization
  - Log Injection — insecure 88% of time due to insufficient data sanitization and validation
  - SQL Injection — 20% risk remains even though AI models somewhat handle this better
  - Cryptographic weaknesses — insecure 14% cases with improper key or algorithm usage
- One study analyzing 1689 GitHub Copilot-generated programs found 40% had exploitable security bugs or design flaws.

Reasons for these Faults

- The security challenges stem from:
  - Training on publicly available code repositories which include vulnerable examples, teaching the model insecure patterns
  - AI's limited semantic and contextual understanding, leading to inadequate handling of authentication, session management, and input validation
  - Lack of application-specific knowledge, such as user roles, threat models, or environment, which humans would use to strengthen security
- Additional risks include potential leakage of private keys or sensitive data embedded in generated code

Risk: Overreliance

Why Copilot is Making Programmers Worse at Programming

posted Wed, Sep 11, 2024 by darren horrocks

opinion software-development copilot llm ai

Will an overreliance on Copilot and ChatGPT make you dumb? A new Microsoft study says AI 'atrophies' critical thinking: "I already feel like I have lost some brain cells."

News By Kevin Okemwa published February 11, 2025

https://www.darrenhorrocks.co.uk/why-copilot-making-programmers-worse-at-programming/

Lee, Hao-Ping Hank, et al. "The Impact of Generative AI on Critical Thinking: Self-Reported Reductions in Cognitive Effort and Confidence Effects From a Survey of Knowledge Workers." (2025).

Which of these problems should be solved by LLMs? Why or Why not?

- Type checking Java code
- Grading mathematical proofs
- Answering emergency medical questions
- Unit test generation

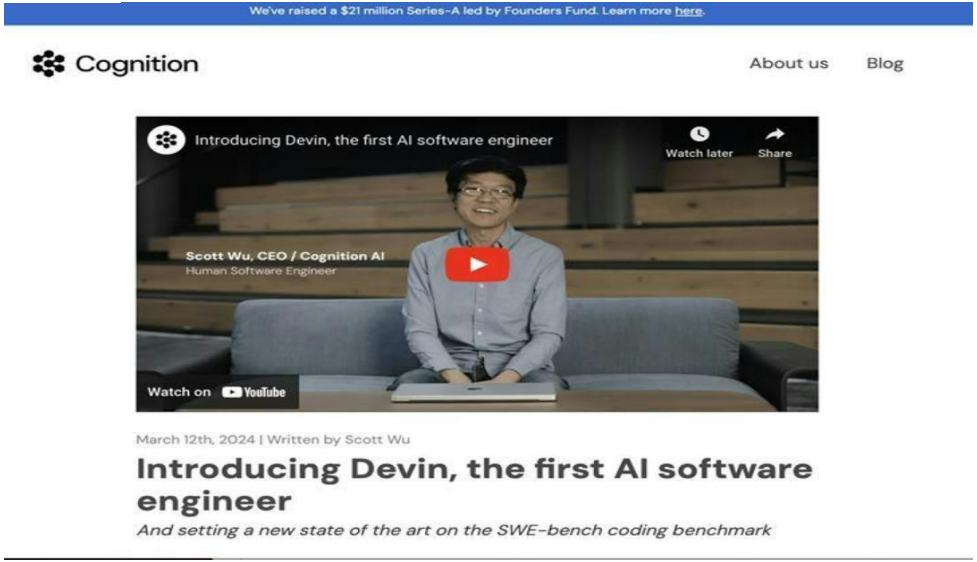
Consider the following

- **Alternative Solutions**: Are there alternative solutions to your task that deterministically yield better results? Eg: Type checking Java code.
- **Error Probability**: How often do we expect the LLM to correctly solve an instance of your problem? This will change over time. Eg: Grading mathematical proofs
- **Risk tolerance**: What's the cost associated with making a mistake? Eg: Answering emergency medical questions
- **Risk mitigation strategies**: Are there ways to verify outputs and/or minimize the cost of errors? Eg: Unit test generation

BLAME DEVIN | JAN 24, 11:19 AM EST *by* VICTOR TANGERMANN

The "First AI Software Engineer" Is Bungling the Vast Majority of Tasks It's Asked to Do

It took longer than a human, and failed at the vast majority of tasks.



Key Takeaways

- **SDLC Still Governs the Process**
  - AI accelerates and automates subtasks, but the overarching lifecycle remains unchanged.
- **Human Oversight Is Non-Negotiable**
  - Every AI suggestion must be validated by engineers who understand domain context, security/regulatory constraints, and business impact.
- **New Skills, Not New Fundamentals**
  - You'll need to learn how to prompt and guide AI tools effectively, interpret their outputs critically, and integrate them into your existing toolchains—but your grasp of core engineering principles is still your greatest leverage.
- In short, AI and its associated tools are revolutionizing *how* we do software engineering—but they do **not** obviate the age-old need for rigorous processes, clear thinking, and human accountability.
- Think of AI as your most powerful new team-member: invaluable, but still one member among many in your engineering ensemble.



Software Engineering

Thank You

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Attribution and Licensing

- Parts of this slideset include content adapted from the CMU 17-313: Foundations of Software Engineering course offered by Carnegie Mellon University.
- This material is used under the terms of the MIT License.
  © 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.

- Parts of the slideset include content derived from various Internet reports and articles.

- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Unit 3 – Lecture 2

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Course Coordinator and Anchor (RR Campus): Prof Vadiraja Acharya
Course Anchor (EC Campus) : Prof M Sheela Devi

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

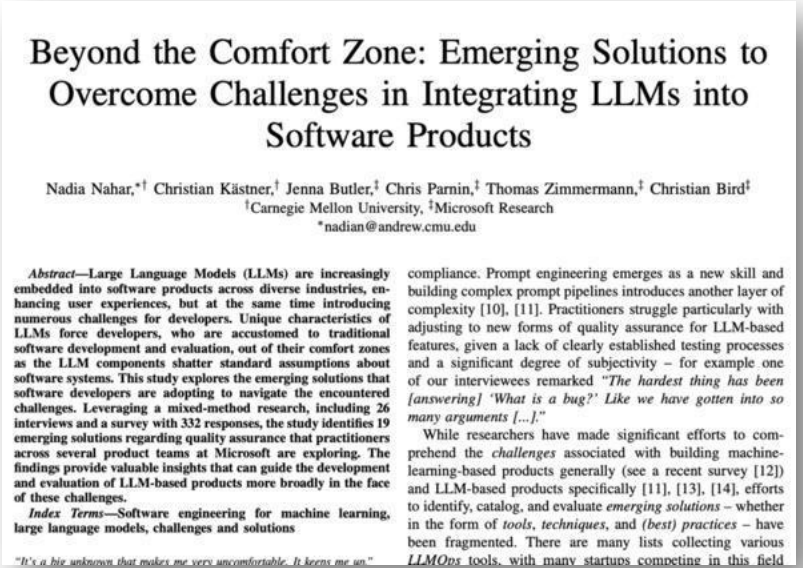
AI and Software Engineering

SE & ML: Connected in Two Ways

- Using ML for Software Engineering
  - How to use AI for SE
  - AI4SE
- Software Engineering for ML Systems
  - How to integrate AI components in SW Systems
  - SE4AI

Change of process/ metrics / mindsets needed...

> We often run into engineers thinking about these as unit tests. […] It is OK that there is 63 failures. Engineers tend to think about it as ohh [...] I need […]. **100% pass rate**



Beyond the Comfort Zone: Emerging Solutions to Overcome Challenges in Integrating LLMs into Software Products

Nadia Nahar,*† Christian Kästner,† Jenna Butler,‡ Chris Parnin,‡ Thomas Zimmermann,‡ Christian Bird‡
†Carnegie Mellon University, ‡Microsoft Research
*nadian@andrew.cmu.edu

***Abstract*—Large Language Models (LLMs) are increasingly embedded into software products across diverse industries, enhancing user experiences, but at the same time introducing numerous challenges for developers. Unique characteristics of LLMs force developers, who are accustomed to traditional software development and evaluation, out of their comfort zones as the LLM components shatter standard assumptions about software systems. This study explores the emerging solutions that software developers are adopting to navigate the encountered challenges. Leveraging a mixed-method research, including 26 interviews and a survey with 332 responses, the study identifies 19 emerging solutions regarding quality assurance that practitioners across several product teams at Microsoft are exploring. The findings provide valuable insights that can guide the development and evaluation of LLM-based products more broadly in the face of these challenges.**

***Index Terms*—Software engineering for machine learning, large language models, challenges and solutions**

"It's a big unknown that makes me very uncomfortable. It keeps me up."

compliance. Prompt engineering emerges as a new skill and building complex prompt pipelines introduces another layer of complexity [10], [11]. Practitioners struggle particularly with adjusting to new forms of quality assurance for LLM-based features, given a lack of clearly established testing processes and a significant degree of subjectivity – for example one of our interviewees remarked *"The hardest thing has been [answering] 'What is a bug?' Like we have gotten into so many arguments […]."*

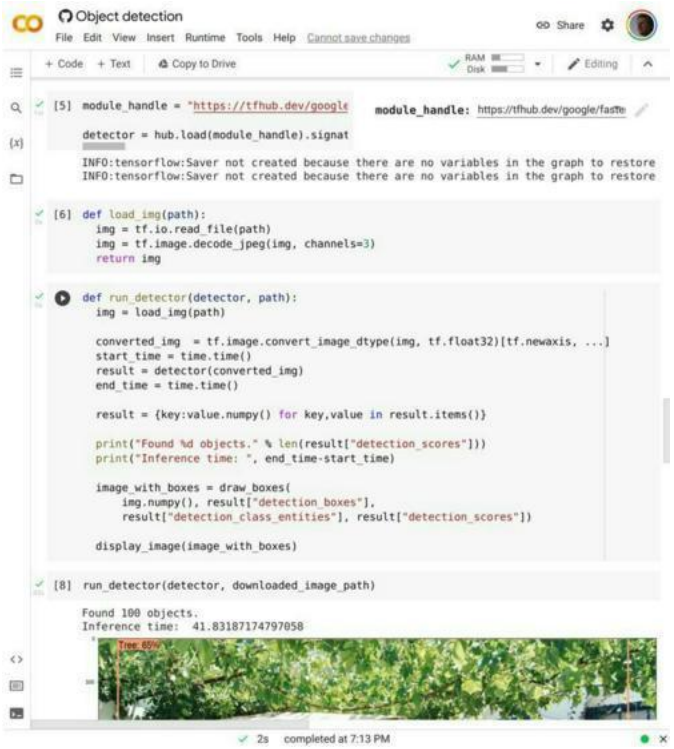
While researchers have made significant efforts to comprehend the *challenges* associated with building machine-learning-based products generally (see a recent survey [12]) and LLM-based products specifically [11], [13], [14], efforts to identify, catalog, and evaluate *emerging solutions* – whether in the form of *tools*, *techniques*, and *(best) practices* – have been fragmented. There are many lists collecting various *LLMOps* tools, with many startups competing in this field

Nahar, Nadia, et al. "Beyond the Comfort Zone: Emerging Solutions to Overcome Challenges in Integrating LLMs into Software Products." ICSE SEIP 2024.

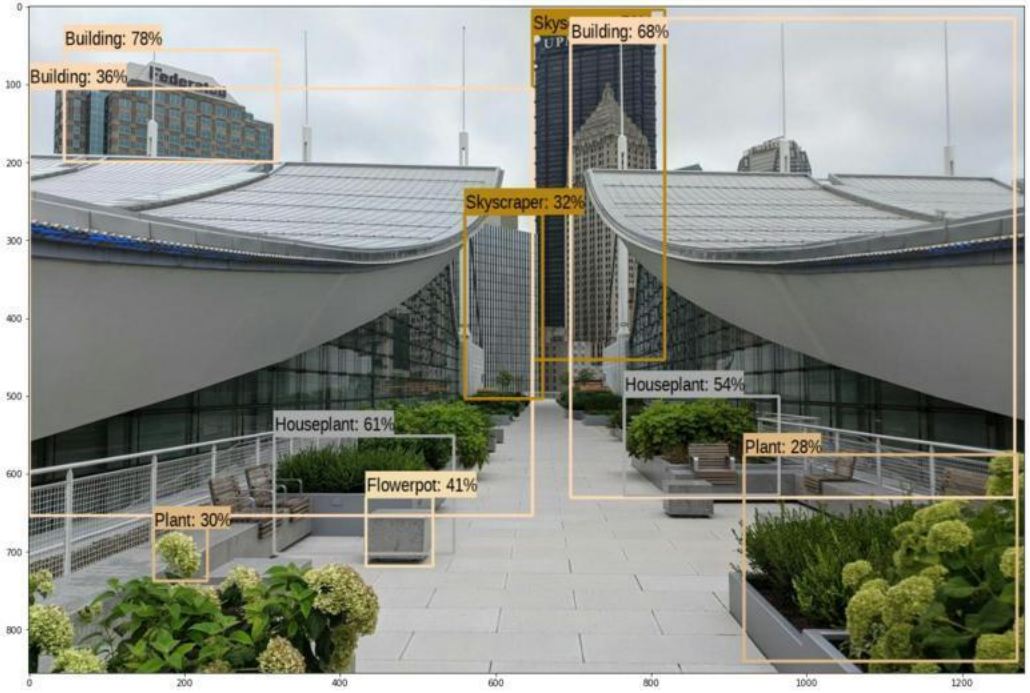
From Models to Systems

ML Model vs ML System

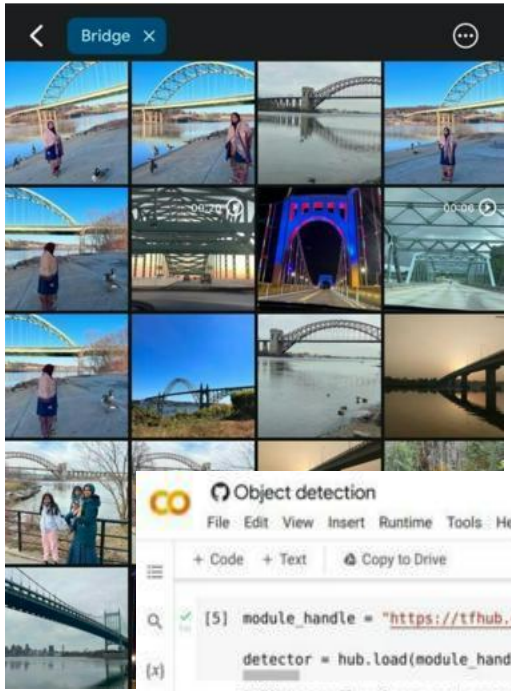
ML Model

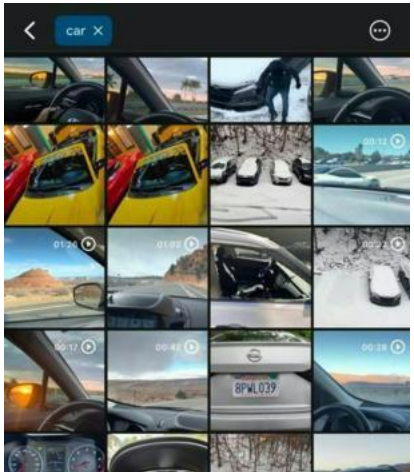


ML System



ML Model vs ML System

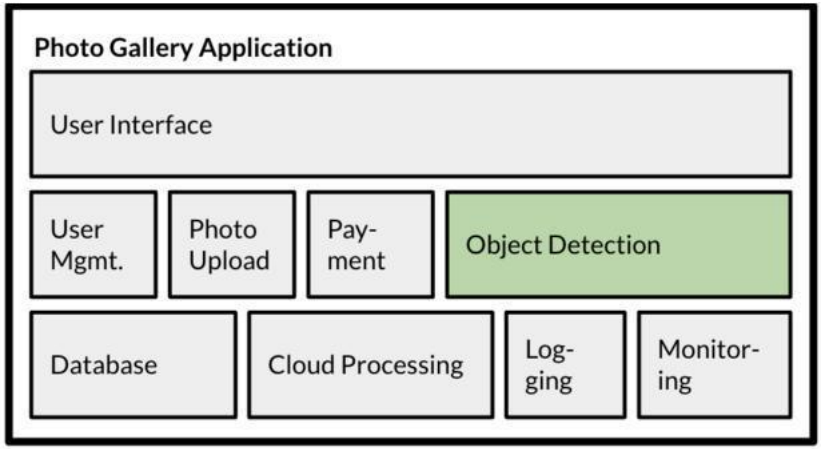




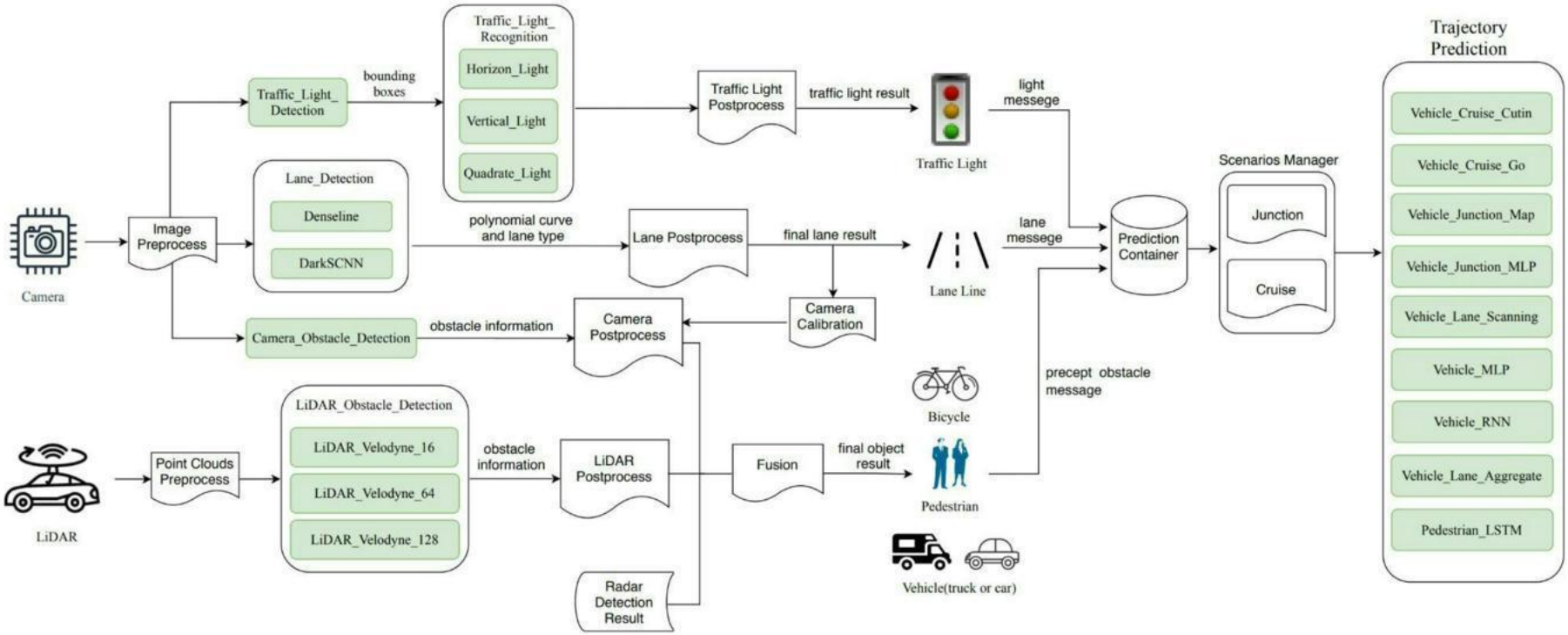


ML Model

ML System



Apollo ML Models



Source: Zi Peng, Jinqiu Yang, Tse-Hsun (Peter) Chen, and Lei Ma. 2020. A First Look at the Integration of Machine Learning Models in Complex Autonomous Driving Systems: A Case Study on Apollo. In Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '20)

Apollo ML Models



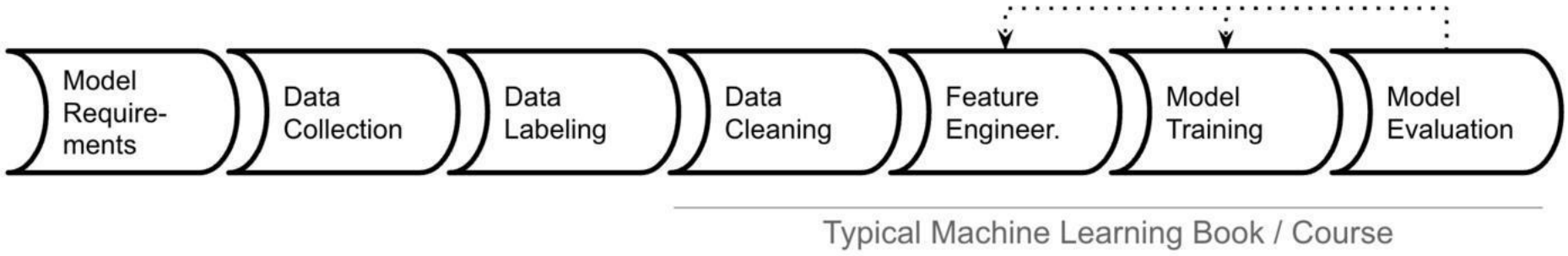
Source: Zi Peng, Jinqiu Yang, Tse-Hsun (Peter) Chen, and Lei Ma. 2020. A First Look at the Integration of Machine Learning Models in Complex Autonomous Driving Systems: A Case Study on Apollo. In Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '20)

Augmented Reality Smart Glasses





Let's Take a Closer Look



Focus: building models from given data, evaluating accuracy

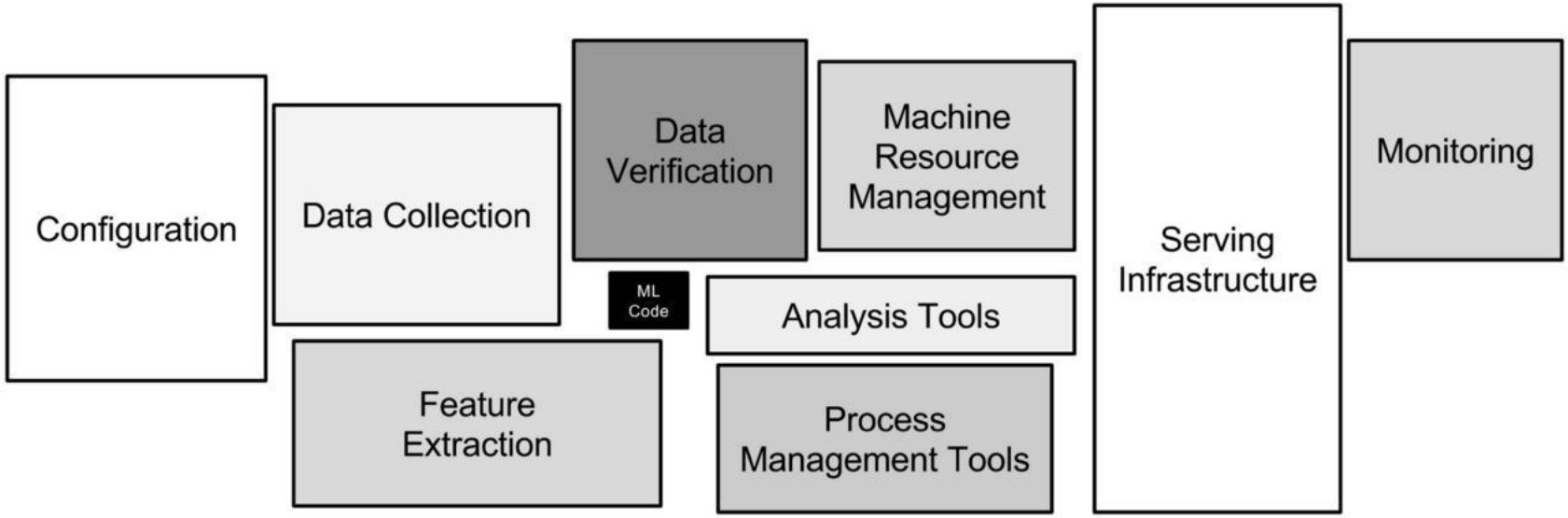


Typical ML Pipeline

- **Static**
  - Get labeled data (data collection, cleaning and, labeling)
  - Identify and extract features (feature engineering)
  - Split data into training and evaluation set
  - Learn model from training data (model training)
  - Evaluate model on evaluation data (model evaluation)
  - Repeat, revising features

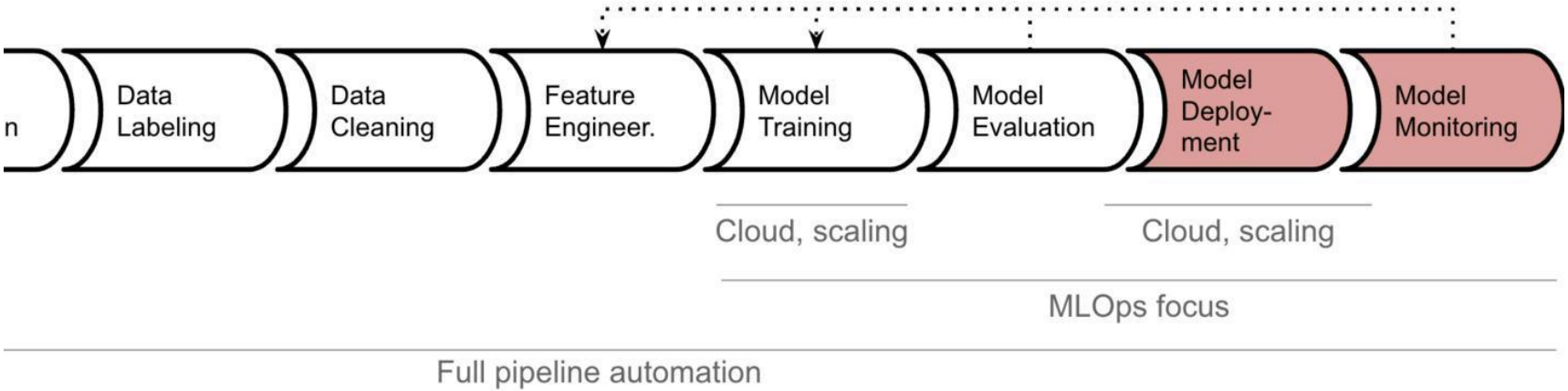
- **In production**
  - Evaluate model on production data; monitor (model monitoring)
  - Select production data for retraining (model training + evaluation)
  - Update model regularly (model deployment)

Only a fraction of real-world ML systems is ML code…



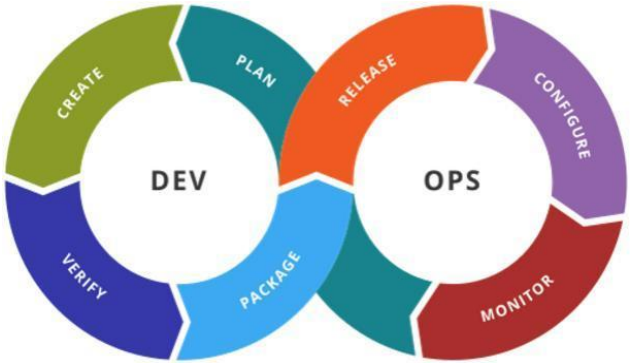
Sculley, et al. *"Hidden technical debt in machine learning systems."* NeurIPS 28 (2015): 2503-2511.

Pipeline Automation and MLOps

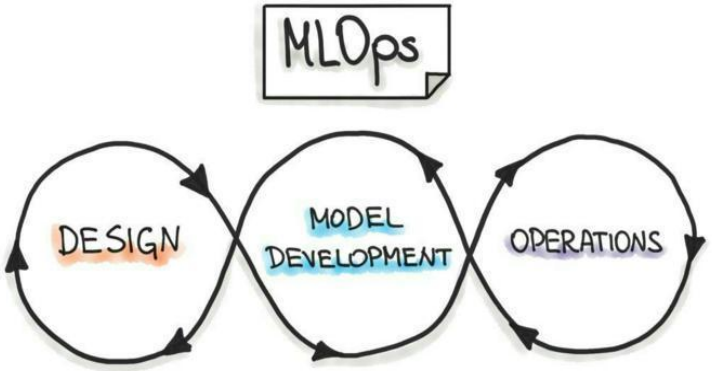


Focus: experimenting, deploying, scaling training and serving, model monitoring and updating

DevOps and MLOps



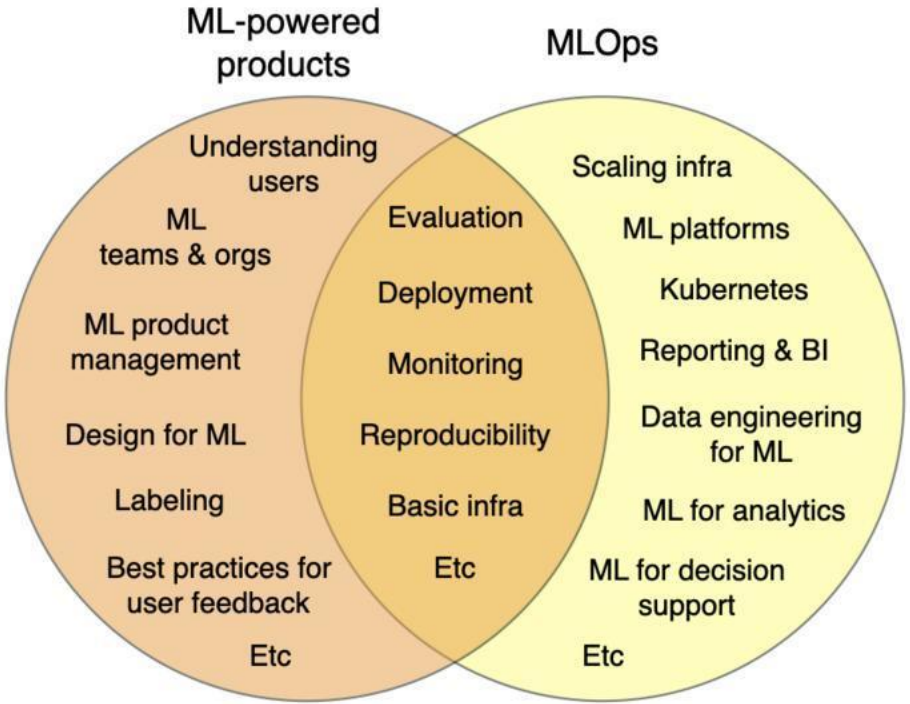
Set of practices for continuous delivery; relies on heavy automation, e.g., continuous delivery, monitoring



Automation around Machine Learning pipeline, including training, evaluation, versioning, and deployment

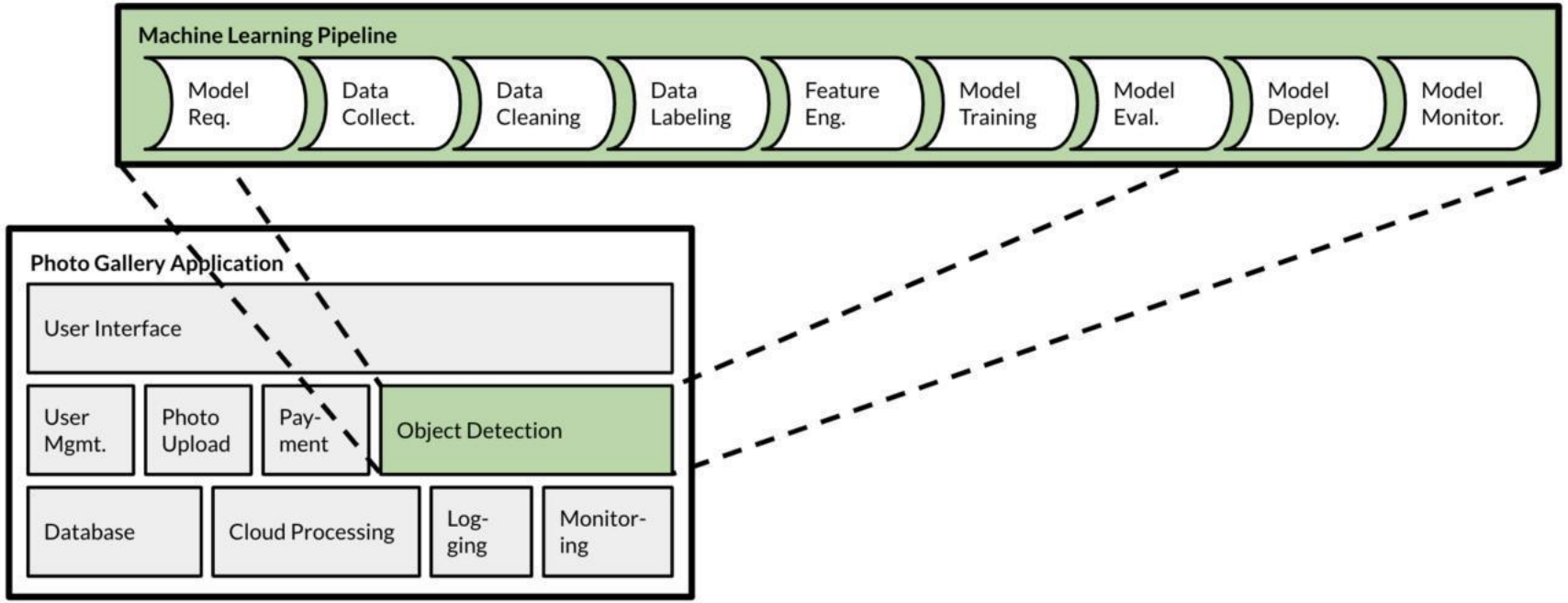
Think about MLOps as a specialized subset of DevOps for machine learning applications

There is more to ML systems than MLOps

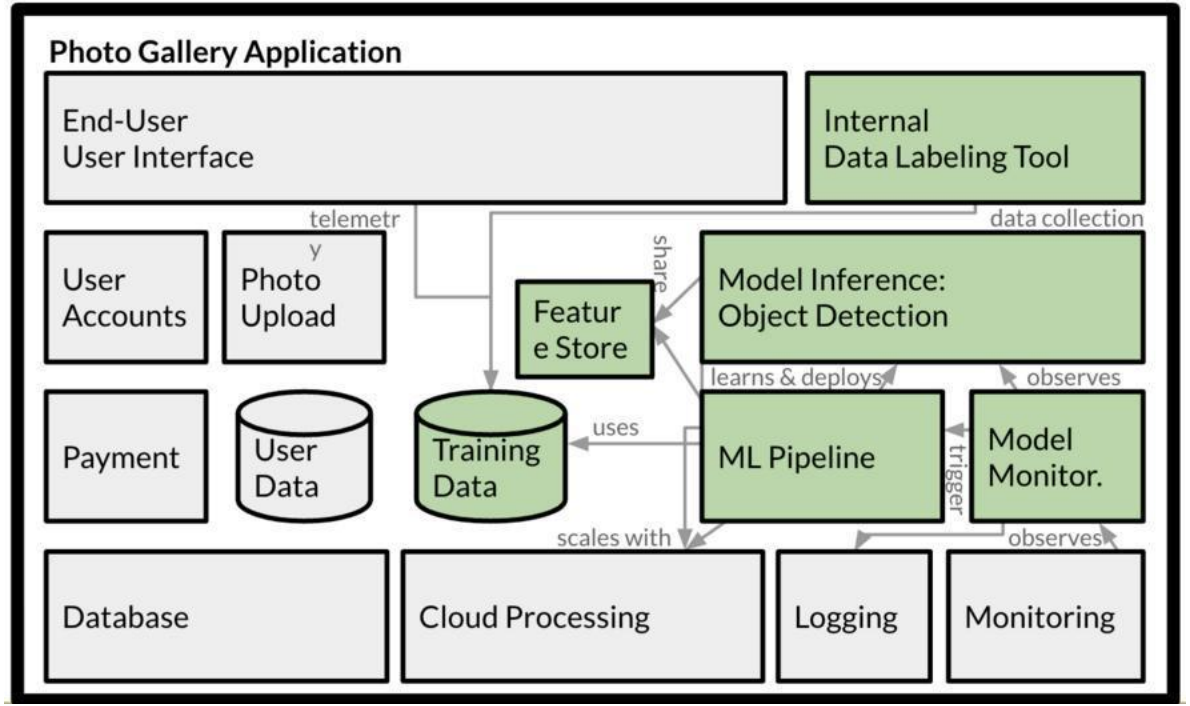


https://fullstackdeeplearning.com/course/2022/lecture-1-course-vision-and-when-to-use-ml/

ML is a Component in a System



Or Many ML Components Actually

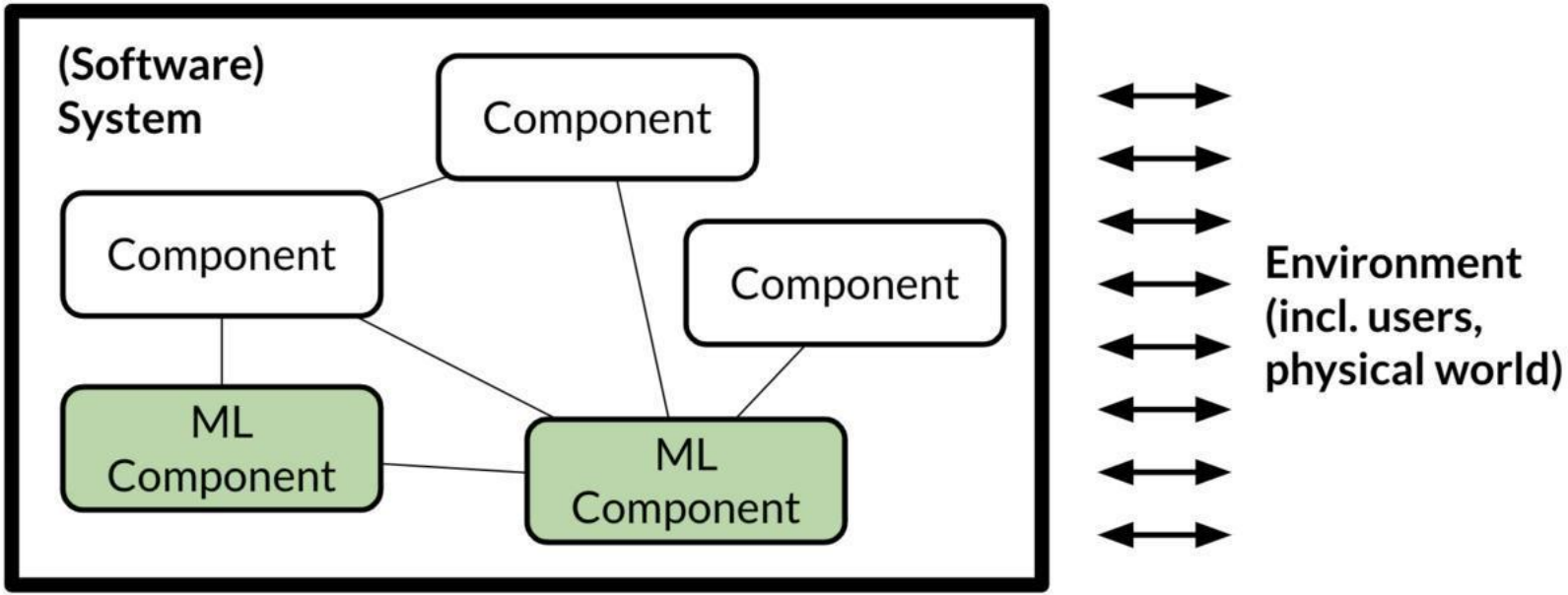


Case Study: Augmented Reality Smart Glasses for Navigation

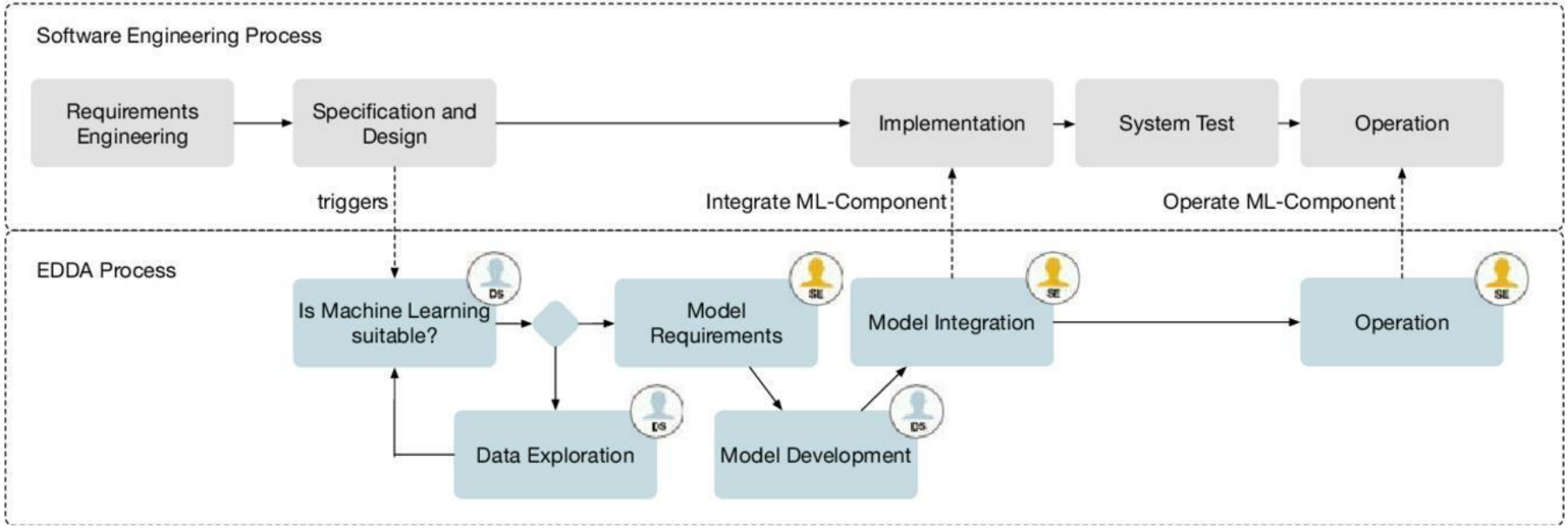




System Thinking



ML Introduces Additional Complexities in Software Systems



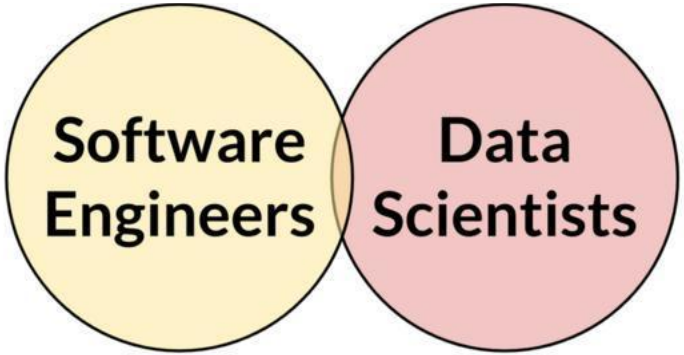
Hesenius, Marc, et al. "Towards a software engineering process for developing data-driven applications." 2019 IEEE/ACM 7th International Workshop on Realizing Artificial Intelligence Synergies in Software Engineering (RAISE). IEEE, 2019.

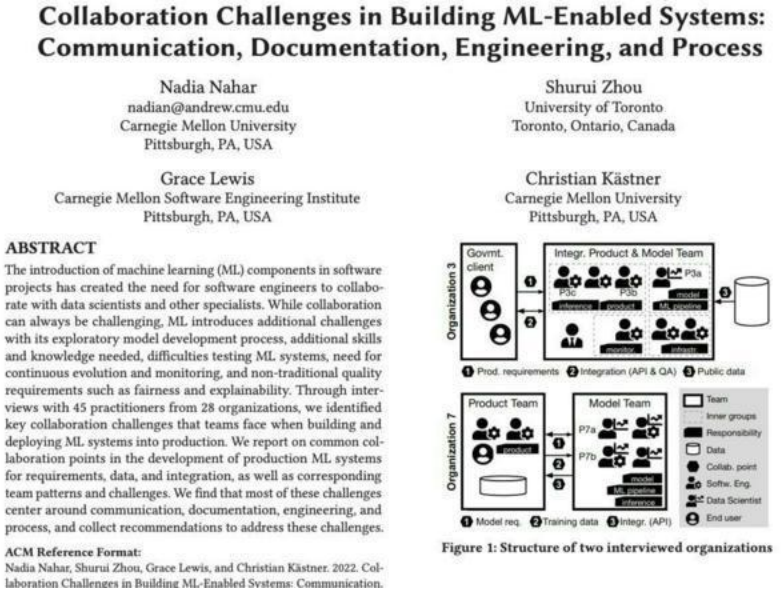
What changes with ML?

Contrast with SE

- **Experimental:** Experiment-driven with model training, testing, refinement based on empirical data.
- **Data-Driven:** Relies heavily on data to train models; data preprocessing is crucial.
- **Algorithmic Focus:** Development of algorithms (e.g., supervised, unsupervised learning) for pattern recognition.
- **Model Evaluation:** Continuous refinement through metrics like accuracy, precision, and recall.

Change of process/ metrics/ mindsets needed…





Collaboration Challenges in Building ML-Enabled Systems: Communication, Documentation, Engineering, and Process

Nadia Nahar
nadian@andrew.cmu.edu
Carnegie Mellon University
Pittsburgh, PA, USA

Shurui Zhou
University of Toronto
Toronto, Ontario, Canada

Grace Lewis
Carnegie Mellon Software Engineering Institute
Pittsburgh, PA, USA

Christian Kästner
Carnegie Mellon University
Pittsburgh, PA, USA

ABSTRACT

The introduction of machine learning (ML) components in software projects has created the need for software engineers to collaborate with data scientists and other specialists. While collaboration can always be challenging, ML introduces additional challenges with its exploratory model development process, additional skills and knowledge needed, difficulties testing ML systems, need for continuous evolution and monitoring, and non-traditional quality requirements such as fairness and explainability. Through interviews with 45 practitioners from 28 organizations, we identified key collaboration challenges that teams face when building and deploying ML systems into production. We report on common collaboration points in the development of production ML systems for requirements, data, and integration, as well as corresponding team patterns and challenges. We find that most of these challenges center around communication, documentation, engineering, and process, and collect recommendations to address these challenges.

ACM Reference Format:
Nadia Nahar, Shurui Zhou, Grace Lewis, and Christian Kästner. 2022. Collaboration Challenges in Building ML-Enabled Systems: Communication.

Figure 1: Structure of two interviewed organizations

Nahar, Nadia, et al. "Collaboration challenges in building ml-enabled systems: Communication, documentation, engineering, and process." *Proceedings of the 44th international conference on software engineering*. 2022.

Specifications and Testing in SE

```
/**
 * Return the sum of all values
 * @ensures \result = \sum int i; 0 <= i < …
 */
int sum(int[] values);
```

```
@Test
void testSentence1() {
  assertEquals(9, sum({2, 3, 4}));
}
```

Lack of Specification in ML

```
/**
 * Detect objects visible in image
 * ????
 */
ObjectId[] detectObjects(File image);
```



Lack of Specification in ML

```
@Test
void testHomePhoto() {
  assertEquals({HOUSE, PLANT},
               detectObjects("img1.jpg"));
}
```



```
@Test
void testStreetPhoto() {
  assertEquals({PERSON, DOG, BICYCLE},
               detectObjects("img2.jpg"));
}
```

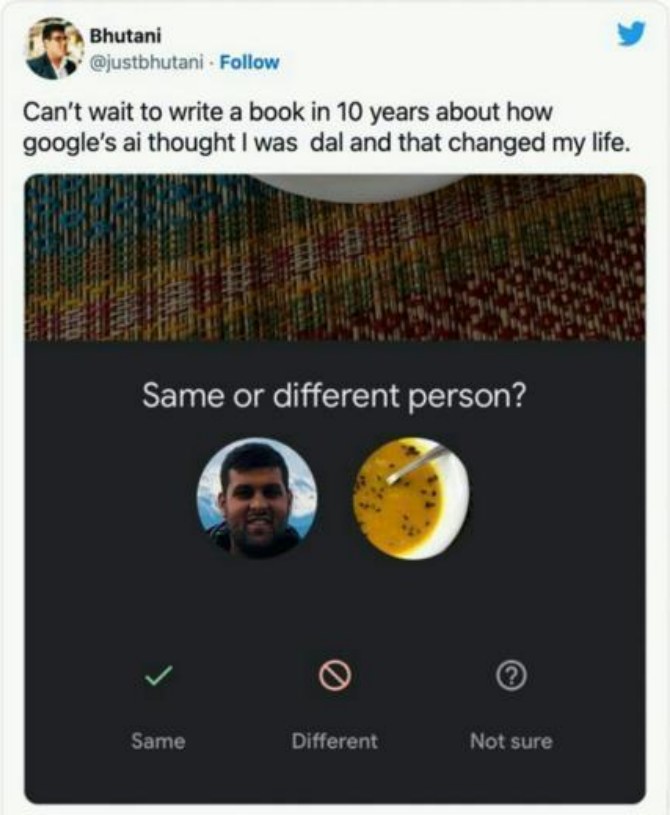


Lack of Specification in ML

- breaks modular reasoning
- challenges quality assurance
- inhibits safety and fairness reasoning
- hinders coordination across teams

(though, we didn't need ML to build low quality, harmful, and unethical software)

Model makes mistake





Mistakes cause harms





Mistakes cause harms

Self-driving Uber car that hit and killed woman did not recognize that pedestrians jaywalk

The automated car lacked "the capability to classify an object as a pedestrian unless that object was near a crosswalk," an NTSB report said.



ML Model = Unreliable Function



Object Detection Model



Building 99%
Path 97%
Plants 98%
Flowerpot 41%
Tree 4%

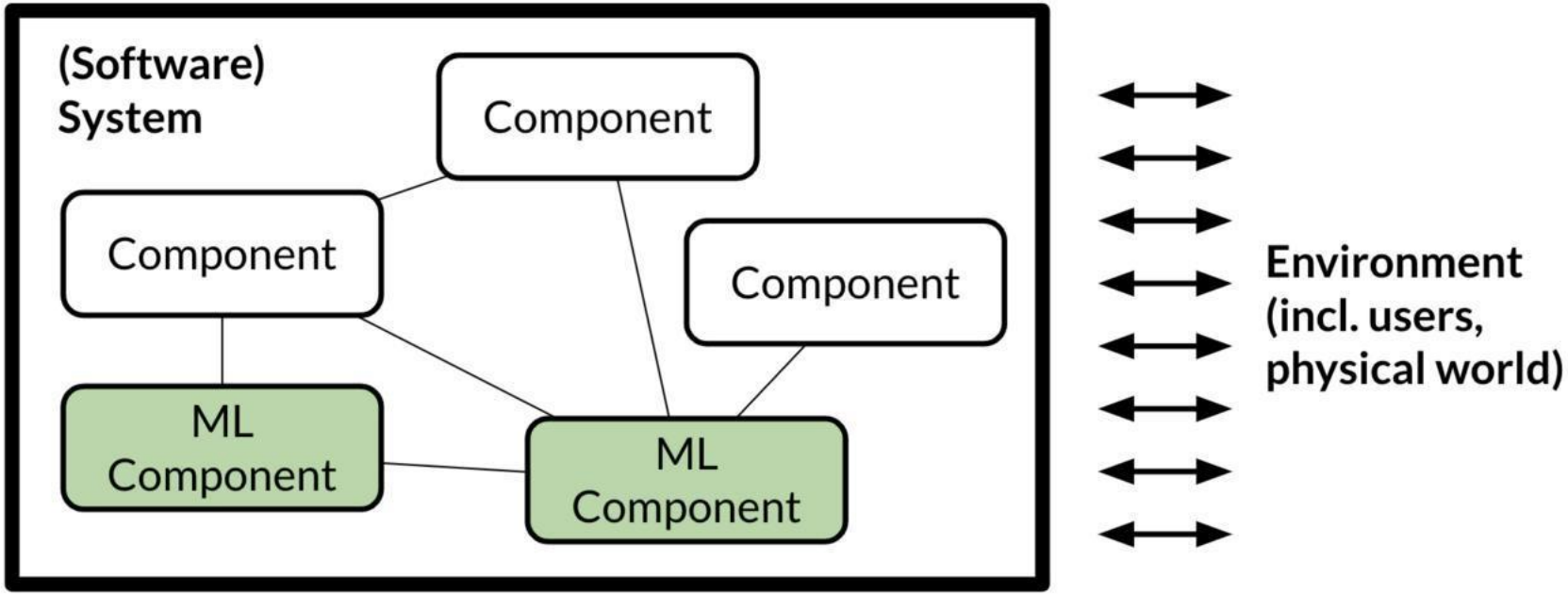
- Guarantees, may make mistakes, confidence unreliable
- Model often inscrutable, opaque
- Evaluated in terms of accuracy, not correctness

Building ML Systems

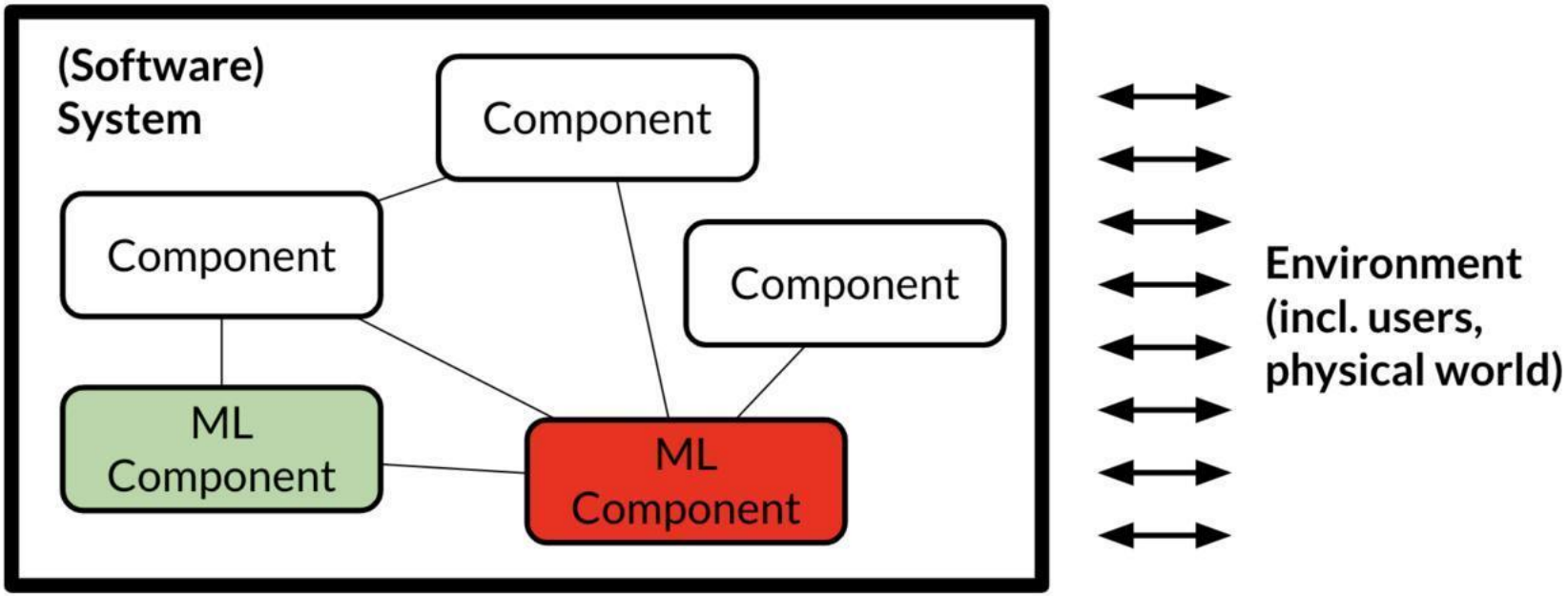
Systems Thinking

- Understand system needs and goals and interactions with environment
- Designing components and integrating ML and non-ML parts into a system
- Many roles and stakeholders, interdisciplinary endeavour

System Thinking



What to do when the ML component makes mistake?



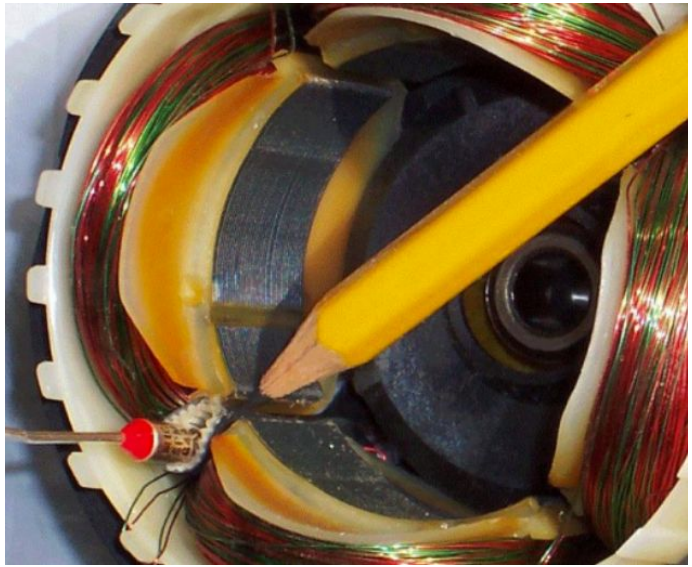
Planning for Mistakes

Example: Smart Toaster



Safety Assurance in/outside the Model

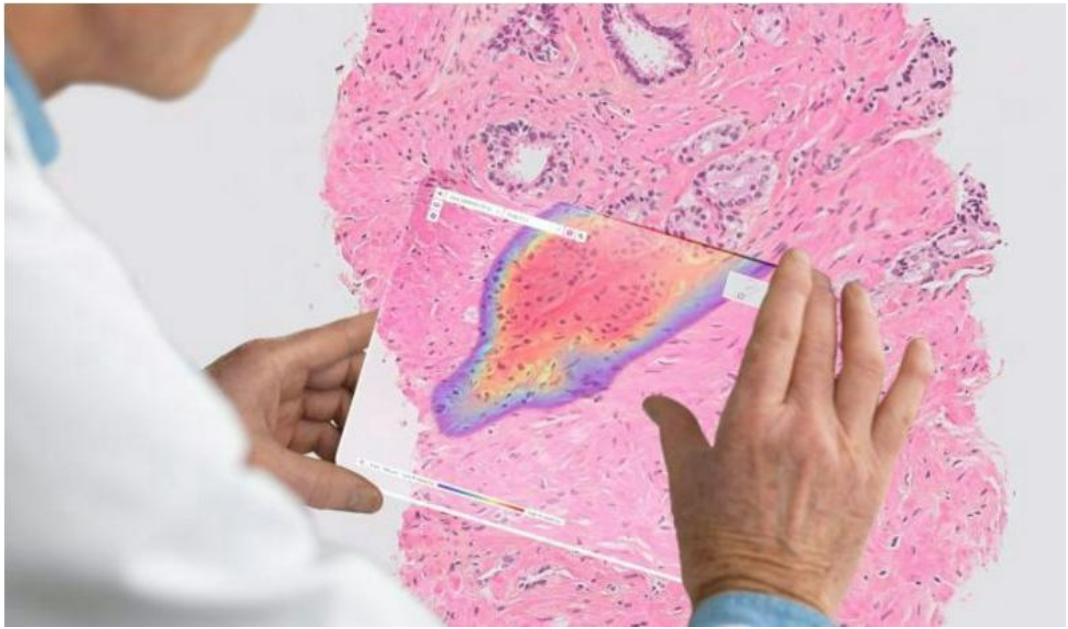
- In the model
  - Ensure maximum toasting time
  - Use heat sensor and past outputs for prediction
  - Hard to make guarantees
- Outside the model
  - Simple code check for max toasting time
  - Non-ML rule to shut down if too hot
  - Hardware solution: thermal fuse



Many different strategies

- Based on fault-tolerant design, assuming that there will be software/ML mistakes or environment changes violating assumptions
  - Human in the loop
  - Undoable actions
  - Mistake detection and recovery (monitoring, doer-checker, fail-over, redundancy)
  - Guardrails
  - Containment and isolation

Humans in the Loop



AI powered diagnostic systems for cancer does not replace pathologists

Humans in the Loop

Food delivery robot pauses operations after Monday incident

Emily Ackerman relies on a wheelchair for mobility and was trapped on Forbes Avenue when robot wouldn't move





Undoable Actions



Get Your Account Back

from blocked listings or suspension



Appeal a suspension

get your appeal done the right way

Blocked Listings Reinstatement

with a managed appeal

Escalate a Denied Appeal

with a custom Bezos escalation letter

Contact Chris

Guardrails





- Code check for max toasting time
- Non-ML rule to shut down if too hot
- Thermal fuse

Architecting ML Systems

Hazard Analysis

- Anticipate mistakes and their consequences.
  - Worst thing that can happen?
- Backup strategy? Undoable? Nontechnical compensation?

Architecture Decisions

- What are the major components in the system? What does each component do?
- Where do the components live? Monolithic vs microservices?
- How do components communicate to each other? Synchronous vs asynchronous calls?
- What API does each component publish? Who can access this API?
- Where does the ML inference happen? Client-side or server-side?
- Where is the telemetry data collected from the users stored?
- How large should the user database be? Centralized vs decentralized?

Quality Requirements Drive Architecture Design

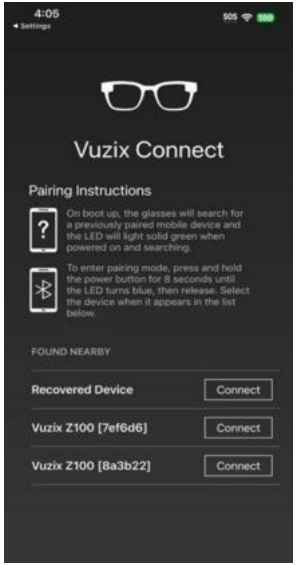
- Development cost, operational cost, time to release
- Scalability, availability, response time, throughput
- Security, safety, usability, fairness
- Ease of modifications and updates
- ML: Accuracy, ability to collect data, training latency

Quality Tradeoffs

- Accuracy
- Capabilities (e.g. classification, recommendation, clustering...)
- Amount of training data needed
- Inference latency
- Learning latency
- Model size
- Explainable

What Qualities are Important?

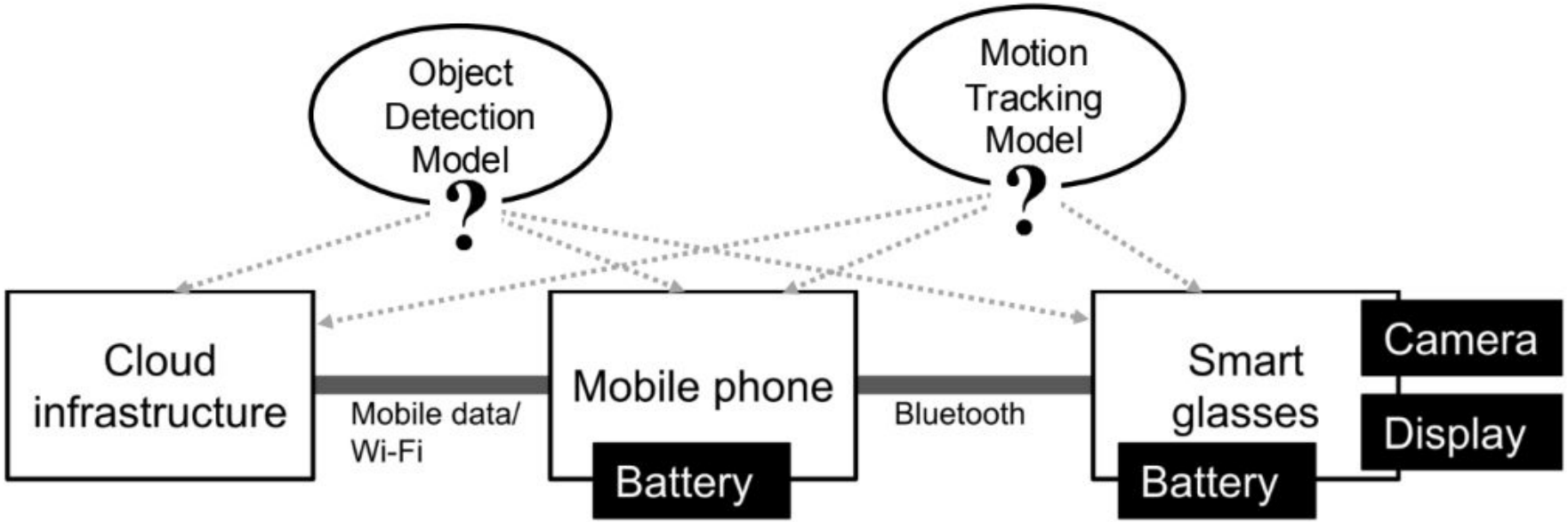






Accuracy? Latency? Model Size?

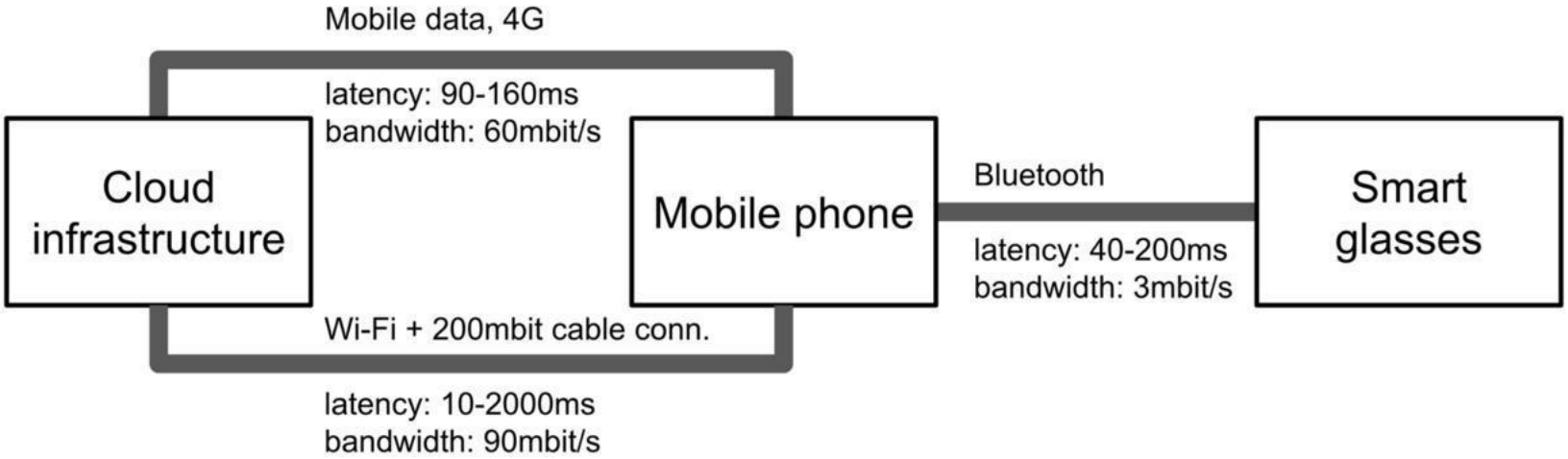
Architecture Decision: Where Should the Model Live?



Considerations

- How much data is needed as input for the model?
- How much output data is produced by the model?
- How fast/energy consuming is model execution?
- What latency is needed for the application?
- How big is the model? How often does it need to be updated?
- Cost of operating the model? (distribution + execution)
- What happens if users are offline?

Latency and Bandwidth Analysis



Activity: Where Should the Model Live?

- Discuss and decide
  - Where should the Object Detection component live?
    - Cloud? Phone? Glasses?
  - Where should the Motion Tracking component live?
    - Cloud? Phone? Glasses?

- Justify your choice
  - What qualities are relevant for the decision? (e.g. latency, bandwidth, battery, offline capability)

Summary

- ML systems ≠ ML models - The model is just one component
- Failures are inevitable - Design for graceful degradation
- Quality tradeoffs are complex - Accuracy isn't everything
- Architecture matters - Where components live affects system behavior
- Safety requires multiple layers - Never rely solely on the model
- The companies that master SE For AI will build the most reliable, safe, and successful AI products.



Software Engineering

Thank You

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Attribution and Licensing

- Parts of this slideset include content adapted from the CMU 17-313: Foundations of Software Engineering course offered by Carnegie Mellon University.
- This material is used under the terms of the MIT License.
  © 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.

- Parts of the slideset include content derived from various Internet reports and articles.

- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Unit-3 Lecture – 2-3

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Course Coordinator and Anchor (RR Campus): Prof. Rajesh Banginwar
Course Anchor (EC Campus) : Prof M Sheela Devi

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Implementation

Programming Personalities - Types

- "The Cryptic Ninja"
- "The Verbose Novelist"
- "The Chaos Demon"
- "The Professional"

Programming Personalities - "The Cryptic Ninja"

```
int f(int n){return n<2?n:f(n-1)+f(n-2);}
```

Programming Personalities - "The Verbose Novelist"

```
// This function calculates the fibonacci sequence recursively
// It takes an integer parameter representing the position in sequence
// It returns the fibonacci number at that position
int calculateFibonacciNumberRecursively(int fibonacciSequencePosition) {
    // First we check if the position is less than 2
    if (fibonacciSequencePosition < 2) {
        // If it is, we return the position itself
        return fibonacciSequencePosition;
    } else {
        // Otherwise we recursively call this function
        return calculateFibonacciNumberRecursively(fibonacciSequencePosition - 1) +
               calculateFibonacciNumberRecursively(fibonacciSequencePosition - 2);
    }
}
```

Programming Personalities - "The Chaos Demon"

```
int fibonacci(int x) {
int result;
if(x<2)
result=x;
else{
result= fibonacci(x-1)
+fibonacci(x-2);}
return result;
}
```

Programming Personalities - "The Professional"

```
int Fibonacci (int position) {
    if (position < 2) {
        return position;
    }
    return Fibonacci (position - 1) + Fibonacci (position - 2);
}
```

Program personalities

- Programs have personalities: messy, verbose, cryptic, neat
- Reflective of the programmers' personality
  - Under documenter
  - CYA Specialist
  - CIO types (pass responsibility to other modules)
  - Dynamic types (create variables on the fly)
  - Fakers (have repetitive code)
  - Multitasker (uses wrappers and glue instead of rewriting)
  - True Believer (Extensive Documentation)

Design to Code

- In most software projects, implementation isn't the hardest part - it's everything that comes with it. Look at these numbers..."
  - 40-50% of project effort goes to implementation
  - 1000s-10000s of configuration items managed
  - 80% of total development time spent reading existing code

Software Implementation (Development)

- Design activity results in a design document.
- This design document is the input to the development (programming) activity.
- The output is executable software which is the input to the next phase – Software Testing.

Software Implementation (Development)

- Before getting into the implementation (development) stage, the following should be looked into:
  - Choice between compiled/interpreted language
  - Decision on development environment
  - A Configuration Management Plan

- During the development (implementation) phase:
  - Use Coding Standards and Coding Guidelines
  - Follow language syntax
  - Address Quality, Security and Testability
  - Provide Unit Tested, Peer-Reviewed, functioning code

Software Implementation (Development)

What are the main characteristics of this phase?

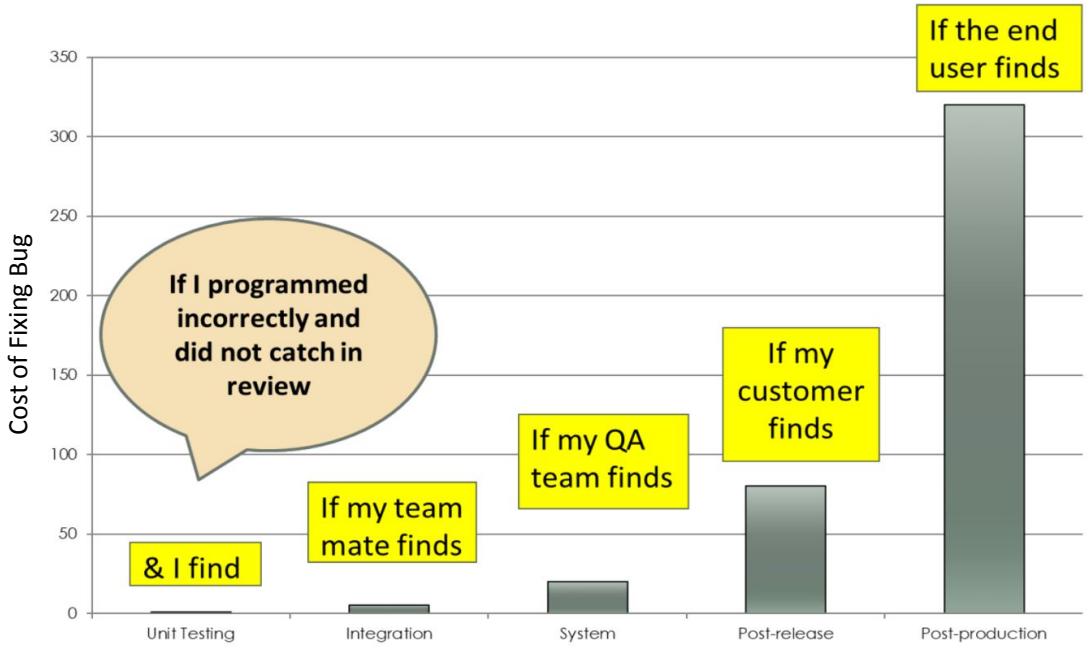
- Produces High Volume of Configuration items
  Eg: Source Files, Test Cases.

- Extensive usage of Computer Science Knowledge
  Eg: Algorithms, Coding Practices

- Related to Software Quality
  Eg: Clean and Maintainable Code

- Tool Intensive
  Eg: Compilers, Debuggers, GUI Builders

Cost of fixing a bug



This is why we obsess over code quality NOW, not later!"

Choice of programming language

Questions to Ask When Choosing a Programming Language

1. Does the language have proper ecosystem support? Is it going to work for the long haul? Is vendor support available for the language?
2. What is the environment for the project (web, mobile, etc)?
3. Do we need to consider some infrastructure such as new hardware? What kind of deployment do we need?
4. What's the preference of the client?
5. Any specific requirements for libraries, features, and tools for the programming language?
6. Is the developer available to code in this language or do we need to hire new developers? Are they experienced and comfortable in working with this language, or do they need to learn the language quickly?
7. What are some important constraints of this project? Time, budget, resources?
8. What's the performance consideration and is the languages suitable to accommodate this performance?
9. What's the security consideration and do we need to use any third-party tool?

Choice of programming language

- **Based on level of Abstraction**
  - **Assembly Languages**: map directly onto CPU architecture
  - **Procedural Languages**: modest level of abstraction from underlying layer
  - **Aspect Orientation Support**: allow separation of aspects during development (An aspect is a subprogram that is associated with a specific property of a program)
  - **Object-Oriented Languages**: allows the developer to code in terms of objects

Choice of programming language

- Performance requirements - "Java vs Python for high-frequency trading?"
- Development speed - "Startup with 3-month runway?"
- Team expertise - "5 C++ experts or 1 Python beginner?"
- Platform constraints - "iOS app in JavaScript? Good luck!"
- Maintenance needs - "Who maintains this in 5 years?"
- Integration requirements - "Must talk to Oracle database?"
- Scalability demands - "100 users or 100 million?"
- Security requirements - "Banking app in assembly language?"
- Cost considerations - "Oracle licenses vs open source?"

Example Scenario

- **Scenario 1: Real-time Notification Service**
  - Must handle 5000+ concurrent users during peak events
  - Push notifications to mobile apps and browsers
  - Integrates with existing Java-based student portal
  - Budget: Limited, must use existing infrastructure
- **Scenario 2: Data Analytics Dashboard**
  - Process event attendance, user preferences, feedback score
  - Generate reports for college administration
  - Must handle statistical calculations and visualizations
  - Used by non-technical staff
  - Timeline: 6 weeks to prototype

Characteristics of code

- Programs should be simple and clear
- Programs should follow previously agreed upon naming conventions.
- Structuring
- Easy to read and understand

Characteristics of code – Simple and Clear

Programs should be well thought out, designed and structured although programmers have a tendency to jumpstart.

- **Programmers should use a reasonable amount of**
  - Lines per function
  - Lines/Functions per file
  - Arguments per function
  - Levels of Nesting
  - Conditions

Characteristics of code – Naming and naming conventions

Naming is Important!!

- **Naming Conventions**
  - Names have to be meaningful and descriptive
  - Avoid using names similar to keywords
  - No variables name must start with _
  - Use a minimum of 2-3 characters and a maximum of 30 characters in a name
  - Do not use Numeric values that are used elsewhere in a name

- **Types of Naming:**
  - **Pascal Casing** (first letter Upper Case)
    Eg: BackColor
    Usage: Class/Variable Names, Method Parameters
  - **Camel Casing** (first letter Upper Case except first word)
    Eg: backColor
    Usage: Method Names
  - **Underscore Prefix** (after underscore, camel case)
    Eg: _backColor
    Usage: for internal use in Class/Methods

Characteristics of code – Naming In-class exercise

Identify the Naming issue with the below block of code

```
int uglyduck(int timer_count)
{
    if(timer_count<=1)
        return 1;
    else
        return
timer_count*uglyduck(timer_count-1);
}
```

Rewrite the following block of code with better Naming

```
char* process_str(char* INPUT_STR, int FLAG) {
    char* temp = malloc(strlen(INPUT_STR) + 1);
    if(FLAG) {
        // some processing
        return temp;
    }
    return INPUT_STR;
}
```

Characteristics of code – Naming In-class exercise - Solution

Identify the Naming issue with the below block of code

```
int uglyduck(int timer_count)
{
    if(timer_count<=1)
        return 1;
    else
        return timer_count*uglyduck(timer_count-1);
}
```

Rewrite the following block of code with better Naming

```
char* process_str(char* INPUT_STR, int FLAG) {
    char* temp = malloc(strlen(INPUT_STR) + 1);
    if(FLAG) {
        // some processing
        return temp;
    }
    return INPUT_STR;
}
```

Issues:

>> Misleading Function Name (`uglyduck`)
>> Misleading Parameter Name (`timer_count`)

```
int factorial(int n)
{
    if (n <= 1)
        return 1;
    else
        return n * factorial(n - 1);
}
```

Rewritten Code with better Naming

```
char* transform_string(char* input_string, int should_transform)
{
    char* transformed_string = malloc(strlen(input_string) + 1);
    if(should_transform) {
        // some processing
        return transformed_string;
    }
    return input_string;
}
```

Characteristics of code – Structuring

- **Structure**:
  - Dependencies between software components (subprograms, parameters, code blocks, etc)
  - Dependencies can be highlighted via proper naming and layout
    - For example: dependent subprogram interface/implementation declarations should be closer

- **File Structure:**
  - **Logically grouped**: #includes, #defines, structures, functions
  - **Well partitioned**: Decide what goes into header files and C files. Which functions/variables are to be exposed
  - **Code and Data Structure**: Well encapsulated and logically grouped. Properly initialized

Characteristics of code – Ease of reading and understanding

```
// Original Horror
for(int i=0;i<40;i++)
if(c>='0'&&c<='9'||c>='a'&&c<='f'||c>='A'&&c<='F')
{count++;if(count>max)max=count;
}else{count=0;}
```

```
// Improved Version 1
for (int i = 0; i < MAX_HEX_CHARS; i++) {
    if (is_hex_digit(c)) {
        count++;
        if (count > max) {
            max = count;
        }
    } else {
        count = 0;
    }
}
```

- Whitespace is free - use it liberally
- Vertical space helps mental parsing - don't cram everything together
- Meaningful names reduce need for comments

Characteristics of code – Ease of reading and understanding

- Comments should concisely and clearly explain the logic of the program.
- They should not be cryptic and misleading.
- Must be easy to identify using proper indentation, spacing and highlighting.

- Insert relevant comments in the block of Code shown below:

```
function sourceCodeComment () {
  var comment = document.getElementbyID("Code Comment").value;
  if (comment != null && comment != '') {
        return console.log("Thank you for your comment.")
}
```

Food for thought: Is it better to comment each line of code individually or add one comment for the overall function? Think of a case where in-line comments would be helpful.

Good programming style

Do's

1. Use a few standards and agreed upon control constructs
2. Use GOTO in a disciplined manner
3. Use user-defined data types to model entities in the problem domain
4. Hide data structures behind access functions
5. Use appropriate variable names
6. Use indentation, parentheses, blank spaces and lines to enhance readability

Don'ts

1. Don't be too clever
2. Avoid "dangling if"
3. Avoid "dangling else"
4. Don't nest too deeply
5. Don't use the same identifier for multiple purposes
6. Examine routines that have more than five formal parameters

Dangling if, dangling else, …

Dangling “if”:

```
if (condition) { }
    if (condition 1) { }
        if (condition 2) { }
else { }
```

- In the above example, there are multiple “*ifs*” with multiple conditions and here we want to pair the outermost if with the else part.
- But the else part doesn’t get a clear view with which “*if*” condition it should pair.
- This leads to inappropriate results in programming.

Dangling if, dangling else, …

Dangling “else”:
This can lead to serious problems. It can lead to wrong interpretations by the compiler and ultimately lead to wrong results.

For example:

```
Initialize k=0 and o=0
if(ch>=3)
if(ch<=10)
    k++;
else
    o++;
```

In this case, we don’t know when the variable “*o*” will get incremented. Either the first “*if*” condition might not get satisfied or the second “*if*” condition might not get satisfied. Even the first “*if*” condition gets satisfied, the second “*if*” condition might fail which can lead to the execution of the “*else*” part. Thus it leads to wrong results.

Coding Standards

- Provide a uniform appearance to code written by different engineers
- Improve readability, maintainability and reduce complexity
- Proactively address some commonly occurring issues with code
- Help in code reuse
- Promote sound programming practices and increase efficiency of programmers

Examples:

- Rules for usage of type of data (local, global)
- Standard headers for different modules
- Naming conventions
- Error and Exception handling conventions

- Some common coding practices:
  - Testable programming
  - Secure programming
  - Defensive programming

Defensive Programming - **Murphy's Law:** If anything can go wrong, it will

```c
// Event registration function - what could go
wrong?
int register_for_event (char* username, char*
event_name, int max_capacity) {
    FILE* file = fopen("registrations.txt", "a");
    int current_count =
count_registrations (event_name);
    if (current_count < max_capacity) {
        fprintf(file, "%s,%s\n", username,
event_name);
        fclose(file);
        return 1; // Success
    }
    fclose(file);
    return 0; // Event full
}
```

```c
// Helper function
int count_registrations (char* event_name) {
    FILE* file = fopen("registrations.txt",
"r");
    char line[100];
    int count = 0;
    while (fgets(line, sizeof(line), file)) {
        if (strstr(line, event_name)) {
            count++;
        }
    }
    fclose(file);
    return count;
}
```

- ***What if scenarios*** - list five ways this could fail
- ***Missing checks*** - what validations are absent?
- ***Resource management*** - any leaks or unsafe operations?
- ***Data integrity*** - could the data get corrupted?

Defensive Programming - **Murphy's Law:** If anything can go wrong, it will

- **What if scenarios - Five ways this could fail**
  - ***Race Condition / TOCTOU (Time-of-Check to Time-of-Use)***
    - Scenario: Two users register at the same time when 1 seat remains (max_capacity - current_count == 1).
  - ***First-Time Execution (registrations.txt does not exist)***
    - Scenario: The program runs on a fresh machine where registrations.txt hasn't been created yet.
  - ***Partial Substring Collision in strstr***
    - Scenario: An event named "Tech" is being checked, but the file already contains registrations for "TechFest" or "Biotech". Or, a user is named "AliceTech".
  - ***Buffer Truncation on Long Lines***
    - Scenario: A username + event name line exceeds 99 characters.
  - ***Disk Full or Write Permission Denied***
    - Scenario: The filesystem runs out of space, quotas are exceeded, or write permissions are restricted.

Defensive Programming - **Murphy's Law:** If anything can go wrong, it will

- **Missing checks - What validations are absent?**
  - ***No NULL pointer checks on files:***
    - Neither function checks if (file == NULL) after fopen.
  - ***No NULL or validity checks on string arguments:***
    - Neither username nor event_name is validated against NULL, empty strings (""), or maximum acceptable length.
  - ***No delimiter injection sanitization:***
    - If username or event_name contains , or \n, it alters the CSV structure, inserting phantom columns or rows.
  - ***No duplicate registration check:***
    - The same user can register for the same event multiple times until the capacity is reached.
  - ***Ignored fprintf / fclose return values:***
    - The code assumes disk writes and buffer flushes always succeed.

Defensive Programming - **Murphy's Law:** If anything can go wrong, it will

- **Resource management - Any leaks or unsafe operations?**
  - ***Crash-induced resource leaks:***
    - Because fopen can fail, passing NULL to fclose(file) triggers undefined behavior (and crashes on many platforms).
  - ***Unnecessary early file open:***
    - register_for_event() opens registrations.txt in "a" mode before knowing whether the event is full. If the event is full, that file handle was opened, held, and closed for no reason.
  - ***No file locking:***
    - There is no mechanism (like flock() or fcntl()) to ensure mutual exclusion across concurrent processes or threads.
  - ***Inefficient disk thrashing:***
    - For every registration attempt, the entire file is scanned from start to finish via count_registrations, causing $O(N)$ disk I/O per write operation.

Defensive Programming - **Murphy's Law:** If anything can go wrong, it will

- **Data integrity - could the data get corrupted? YES!!**
  - ***Interleaved writes:***
    - Under concurrent processes writing simultaneously without file locks, chunks written by fprintf can interleave at the operating system or buffer layer, resulting in mangled CSV rows (e.g., user1,eveuser2,event2nt1).
  - ***Format corruption via user input:***
    - An attacker entering a username like "eve,HackCon\nadmin" will inject entire new records into the file.
  - ***Delimiter collisions:***
    - Commas inside a name (e.g., "Smith, John") shift columns and corrupt downstream parsing.

Defensive Programming

- "Defensive programming is like insurance - it costs a little upfront but saves massive headaches later.
- The question isn't 'What could go wrong?' - it's 'What will go wrong when Murphy's Law strikes?'"
- **Key mindset:** "Code like the user is actively trying to break your system, because eventually, someone will!"

Coding Standard Practices: Secure Programming

- **Secure Coding**: Developing computer software to guard against accidental introduction of security vulnerabilities (Defects, Bugs, Logic Flaws)

Some practices are:

- **Validate Input:** Validation of input from all untrusted data sources
- **Heed Compiler Warnings:** Compile using highest warning level and eliminate warnings via code modifications
- **Use static and dynamic analysis** tools to detect additional flaws
- **Default Deny:** access decisions are based on permissions
- **Adhere to Principle of Least Privilege:** process should execute with least set of privileges
- **Elevated permissions** for least amount of time
- **Sanitize Data sent to other systems:** Sanitize all data sent to complex subsystems



Software Engineering

Thank You

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Attribution and Licensing

- Parts of this slideset include content adapted from the CMU 17-313: Foundations of Software Engineering course offered by Carnegie Mellon University.
- This material is used under the terms of the MIT License.
  © 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.

- Parts of the slideset include content derived from various Internet reports and articles.

- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Unit-3 Lecture – 4

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Course Coordinator and Anchor (RR Campus): Prof Vadiraja Acharya
Course Anchor (EC Campus) : Prof M Sheela Devi

Department of Computer Science and Engineering

Implementation

Static Analysis

Note:

- Static analysis is a method of debugging that is done by automatically examining the source code without having to execute the program.

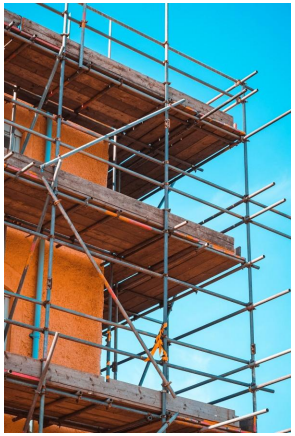
- Static analysis is generally good at finding coding issues such as:
  - Programming errors
  - Coding standard violations
  - Undefined values
  - Syntax violations
  - Security vulnerabilities

- Static code analysis is a form of white-box testing that can help identify security issues in source code. On the other hand, dynamic code analysis is a form of black-box vulnerability scanning that allows software teams to scan running applications and identify vulnerabilities.

Coding Standard Practices: Testable Programming

- **Testable Coding**: Developing computer software that is easy to test, find and fix bugs

- Some practices are:
  - **Assertions**: To identify out-of-range values wherever possible
  - **Test Points**: Methods to set/retrieve current module status and variable contents
  - **Scaffolding**: Code to emulate a feature of the system that the object would call.
  - **Test Harness**: Code written to drive objects/modules/components as a part of the complete system
  - **Test Stubs**: Code to return known, fixed values to emulate functions that are not being tested
  - **Instrumenting**: Execution logging and messages to visualize execution.
  - **Building test data sets**: For both valid and invalid data



Scaffolding is **any code that we write, not as part of the application, but simply to support the** process of Unit and Integration testing

Coding Guidelines

- Provide a uniform appearance to code written by different engineers
- Generic suggestions that improve understanding and readability of code
- Reduce cost incurred by early detection of errors
- Reduce complexity and ease maintenance over lifetime of the product

Examples

- Code should be well Documented
- Indent code with spaces and lines
- Minimize the length of functions
- Reduce the usage of GOTO statements
- Avoid using same identifiers for multiple purposes

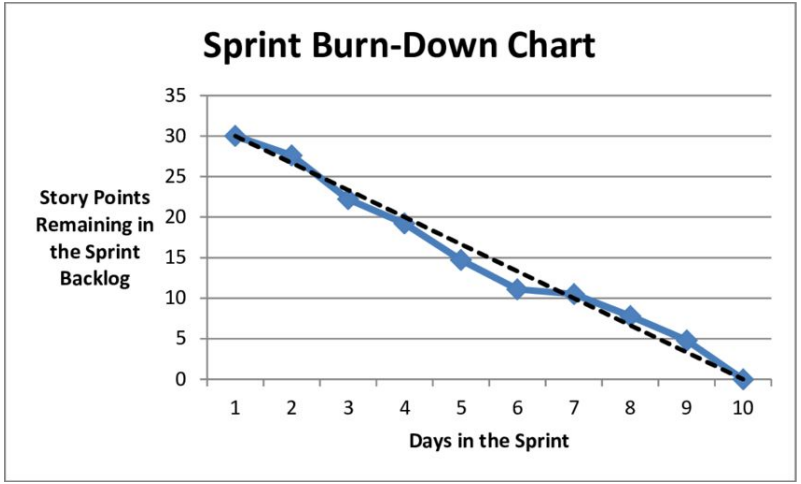
One of the more commonly used “C coding” guidelines” document is here.

Managing construction – Quality for Agile projects

- **Sprint Burndown**
  - Graphically communicated key metric representing completion of work throughout sprint based on story points
  - Goal of the team is to consistently deliver all work
- **Team Velocity Metric**
  - “Amount” of software or stories completed during a sprint
  - Can be measured in story points or hours
  - Used for estimation and planning
- **Throughput**
  - Indicates total value-added work output by the team
  - Represented by units of work completed in period of time
- **Cycle Time**
  - Total time that elapses from the moment work is started on an item till it’s completion

Sprint Burndown chart

- The Sprint Burndown Chart makes the work of the Team visible.
- It is a graphic representation that shows the rate at which work is completed and how much work remains to be done.
- The chart slopes downward over Sprint duration and across Story Points completed.
- What makes the chart an effective reporting tool is that it shows Team progress towards the Sprint Goal, not in terms of time spent but in terms of how much work remains.



Managing construction – Construction Technical Quality

- Activities for ensuring construction quality:
  - **Peer Review**: Common and Informal process of developer seeking advice or comments from other developers
  - **Unit Testing**: Writing code that calls individual modules/functions and tests their working
  - **Test-first**: Designing and building test harness before code. (Potential reason for doing so?)
  - **Code Stepping**: Code is executed one statement at a time and the values of variables and flow of control is analyzed
  - **Pair Programming**: Two developers work on same code with one focusing on logic and other on syntax
  - **Debugging**: Analyzing code to locate a known defect
  - **Code Inspections**: formal process of peer review
  - **Static Analysis**: examination of code without execution

Code Reviews

Reviews (generally)

- **Review**: a constructive review of a fellow team-member's artifact (design specification, code), providing suggested improvements

  - Documentation
  - defects in program logic, efficiency
  - program structure
  - coding standards & uniformity with codebase
  - everything is fair game

- Feedback → revision / refactoring → (loop) → approval

Code reviews

- **Code review:** a constructive review of a fellow developer's code

- A required sign-off from another team member before a developer is permitted to check in changes or new code

Why code review?
Didn't we already test?

Code reviews are valuable

- Catch many bugs and design flaws early
- Promote teamwork and build trust
- > 1 person has seen every piece of code
  - Insurance against author's disappearance ("winning lottery")
  - Accountability (both author and reviewers are accountable)
- Forcing function for documentation and code improvements
  - Authors must articulate their decisions
  - Authors participate in the discovery of flaws
  - Prospect of a review raises your quality threshold
- Less experienced devs get experience (without hurting code quality)
  - Pair them up with experienced developers
  - Learn by example by being a reviewer as well as a reviewee

Let's look at the data

- Average defect detection rates
  - Unit testing: 25%
  - Integration testing: 45%
  - **Design and code inspections: 55% and 60%** **<<<<<<<<!!**

- 11 programs developed by the same group of people
  - No reviews: average 4.5 errors per 100 LOC
  - **With reviews: average 0.82 errors per 100 LOC** **<<<<<<<<!!**

- After AT&T introduced reviews
  - **14% increase in productivity and a 90% decrease in defects** **<<<<<!!**

Code Reviews at Google

"All code that gets submitted needs to be reviewed by at least one other person, and either the code writer or the reviewer needs to have readability in that language. Most people use Mondrian [now Critique] to do code reviews, and obviously, we spend **a good chunk of our time** reviewing code."

-- Amanda Camp, Software Engineer, Google



See: https://google.github.io/eng-practices/review/

Code Reviews at Yelp

"At Yelp we use review-board. An engineer works on a branch and commits the code to their own branch. The reviewer then goes through the diff, adds inline comments on review board and sends them back. The **reviews are meant to be a dialogue**, so typically comment threads result from the feedback. Once the reviewer's questions and concerns are all addressed, they'll click "Ship It!" and the author will merge it with the main branch for deployment the same day."

-- Alan Fineberg, Software Engineer, Yelp



Code reviews at Facebook

"Once an engineer has prepared a change, she submits it to this [code review] tool, which will notify the person or people she has asked to review the change, along with others that may be interested in the change – such as people who have worked on a function that got changed.



At this point, the reviewers can make comments, ask questions, request changes, or accept the changes. If changes are requested, the submitter must submit a new version of the change to be reviewed. All versions submitted are retained, so reviewers can compare the change to the original, or just changes from the last version they reviewed. Once a change has been submitted, the engineer can merge her change into the main source tree for deployment to the site during the next weekly push, or earlier if the change warrants quicker release."

- Ryan McElroy, Software Engineer, Facebook

Common for big picture reviews: Holistic

- Review the **goal**, the **architecture**, the **design**, or an **entire component**

- Each reviewer focuses where their area of expertise
  - Mark up with comments
  - Identify most important issues

- At meeting, may go around the table raising one issue
  - Discuss the reasons for the current approach and possible improvements

- Author addresses all issues in comments
  - Not just those raised in the meeting

Common for code reviews: Incremental

- Each change is reviewed *before* it is committed

- No change is accepted without signoff by a “committer”
  - Part of the “plan” is, who can do the actual commit
  - Committer is assumed to know the whole codebase well; most often is the developer of the change.

- Code review can (d)evolve into a design discussion

Leverage review checklists

- **Design:** Is the code well-designed and appropriate for the system?
- **Functionality:** Does the code behave as the author likely intended?
- **Complexity:** Could the code be made simpler?
- **Tests:** Does the code have correct and well-designed automated tests?
- **Naming:** Did the developer choose clear names for variables, classes, methods?
- **Comments:** Are the comments clear and useful?
- **Style:** Does the code follow style guidelines?
- **Documentation:** Did the developer also update relevant documentation?

See: https://google.github.io/eng-practices/review/reviewer/looking-for.html

Code Example

```
def login_user(username, password):
    # Check admin first
    if username == "admin" and password == "123":
        return True
    # Get users from database
    users = get_users()  # Assume this works
    # Check each user
    for user in users:
        if user['name'] == username and user['pass'] == password:
            return True
    return False
```

```
# Usage example
if login_user("john", "mypass"):
    print("Welcome!")
else:
    print("Login failed")
```

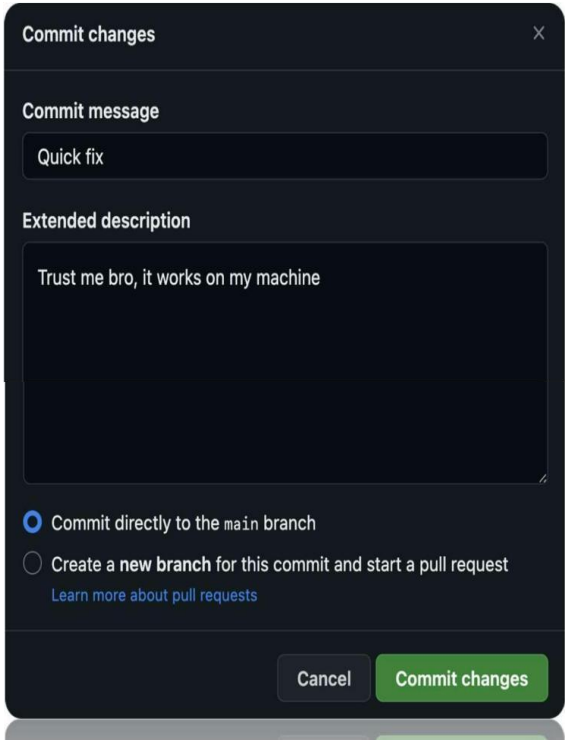
- Initial Analysis per checklist
- Security Issues
- Functionality Problems
- Maintainability concerns

Constructive Code Reviews

- **Key principles to establish:**
  - Criticize code, not coder - "This function is complex" not "You wrote complex code"
  - Explain the 'why' - Don't just say what's wrong, explain the impact
  - Suggest improvements - Be part of the solution
  - Pick your battles - Focus on high-impact issues first
  - Ask questions - "Could we simplify this?" vs "This is too complex"
- **Industry Reality Check**
  - In professional development, this review would trigger a 'Do Not Ship' flag. Security vulnerabilities are career-limiting moves.
- Code reviews aren't about perfection on first try - they're about collective quality improvement

Writing a good Pull Request is also important

"What could go wrong?" Famous last words



Trust me bro, it works on my machine

Writing a good pull request - Think like a reviewer

- Use descriptive but concise title and summary
- Describe context, rationale, and alternatives considered
- Link to relevant resources (specs, issues/bug tracker, previous PR)
- Provide screenshots/recordings for UI changes

How to write the perfect pull request

https://github.blog/2015-01-21-how-to-write-the-perfect-pull-request/

Writing good CL descriptions

https://google.github.io/eng-practices/review/developer/cl-descriptions.html

Archaeology

Learning Goals

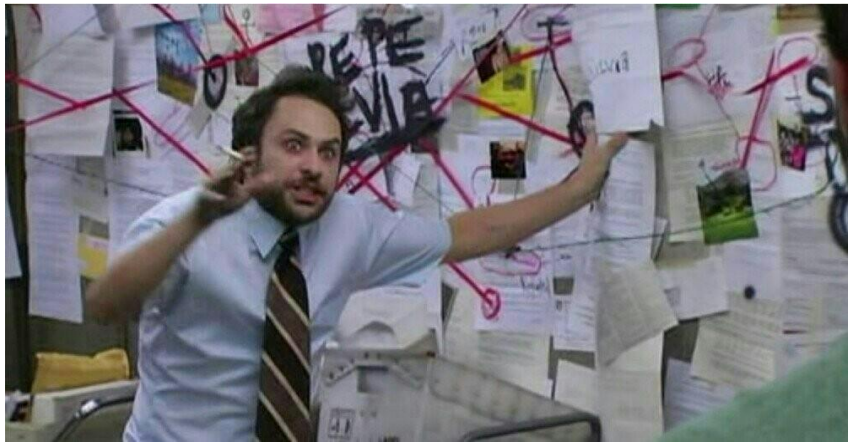
- Understand and scope the task of taking on and understanding a new and complex piece of existing software
- Appreciate the importance of configuring an effective IDE
- Contrast different types of code execution environments including local, remote, application, and libraries
- Enumerate both static and dynamic strategies for understanding and modifying a new codebase

Learning Goals

YOU WILL NEVER UNDERSTAND THE ENTIRE SYSTEM!

Challenge: How do I tackle this codebase?

- Leverage your previous experiences (languages, technologies, patterns)
- Consult documentation, whitepapers
- Talk to experts, code owners
- Follow best practices to build a working model of the system

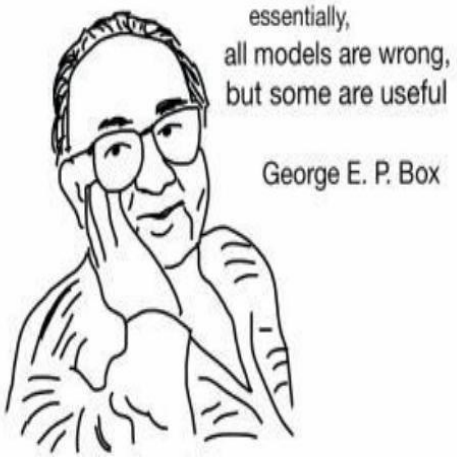


How to tackle codebases

- **Goal:** develop and test a working model or set of working hypotheses about how (some part of) a system works

- **Working model:** an understanding of the pieces of the system (components), and the way they interact (connections)

- **Observation, probes, and hypothesis testing**
  - Helpful tools and techniques!



Steps to Understand a New Codebase

- Clone the repo.
- Look at README.md
- Understand folder and file structure
- Build the codebase.
- Figure out how to make it run.
- What do you want to mess with?
- Traceability - Attach a debugger
  - View Source
  - Find the logs.
- Search for constants (strings, colors, weird integers (#DEADBEEF))

Program comprehension strategies

Novice

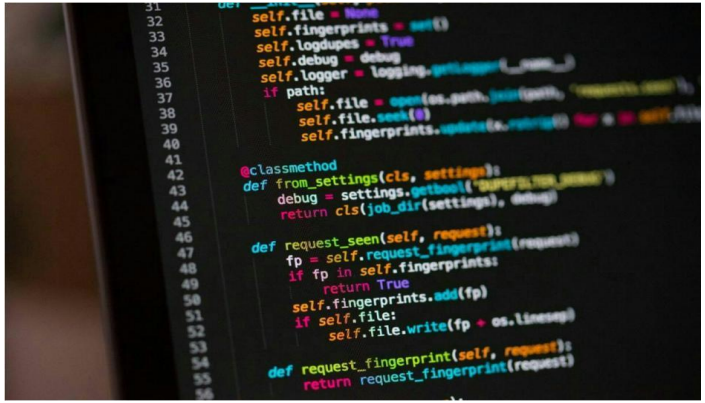
- Reads code line by line
- Revisits same code repeatedly
- Trial and error
- Only tests “happy path”

Expert

- “ Top down”
- Recognizes patterns
- Forms hypotheses
- Checks up/downstream consequences

Observation: Software is full of patterns

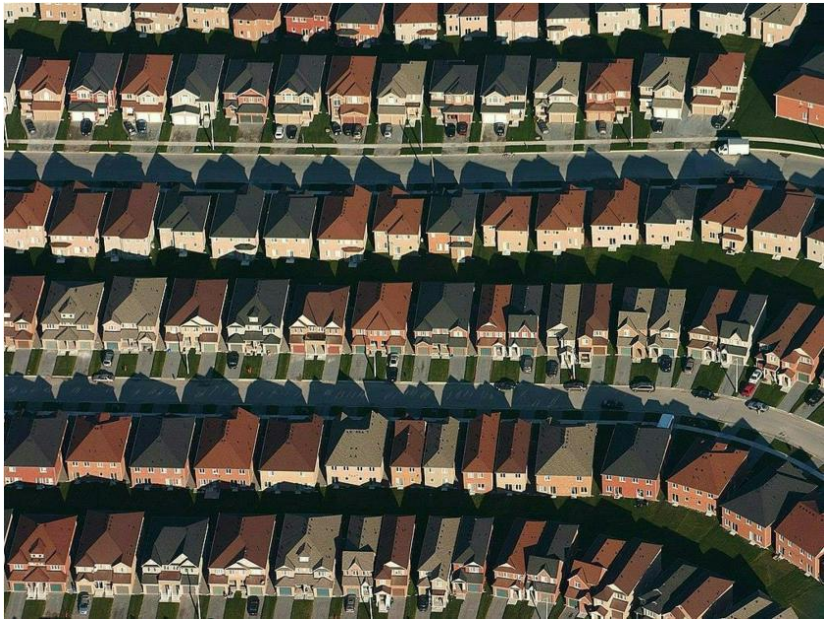
- File structure
- System architecture
- Code structure
- Names
- …



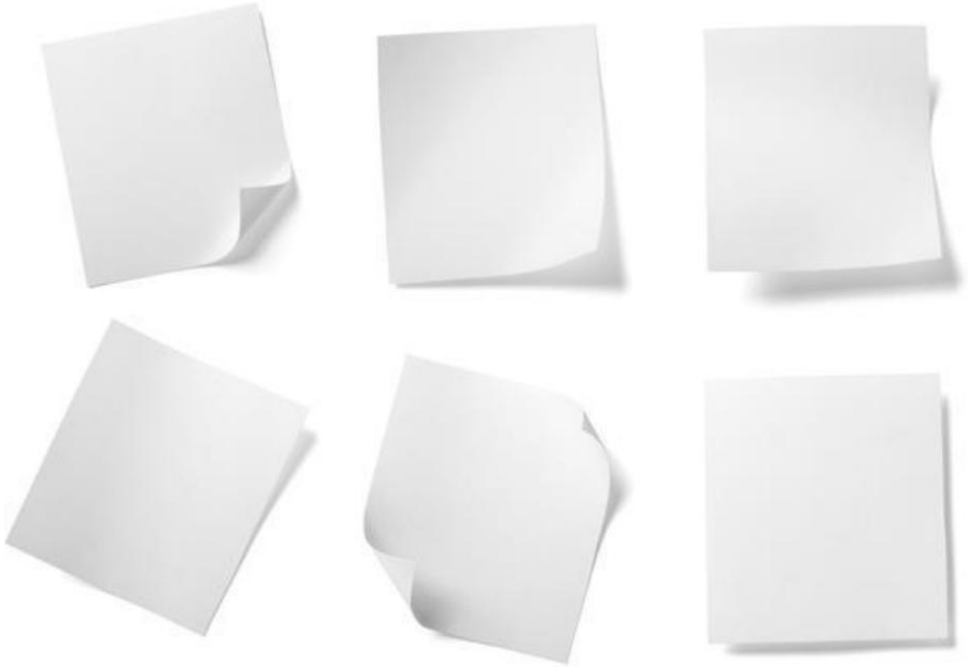


Observation: Software is massively redundant

There’s always something to copy/use as a starting point!



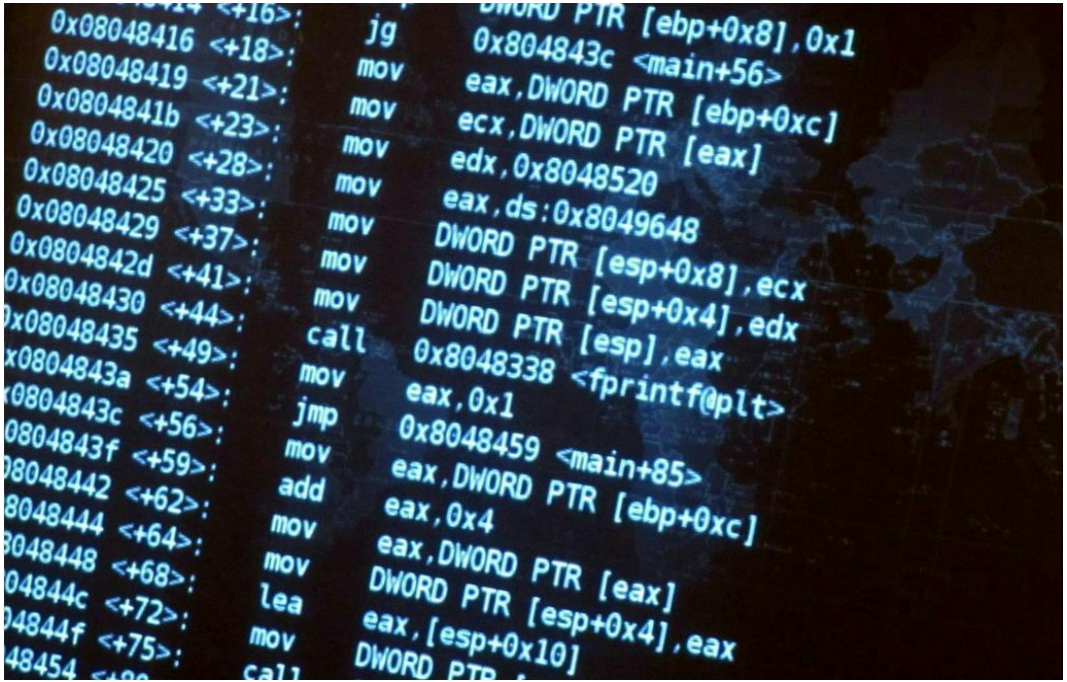
Observation: Code must run to do stuff!



Observation: If code runs, it must have a beginning…



Observation: If code runs, it must exist…



Code must exist. But where?

- **Locally installed programs:** run cmd, OS launch, I/O events, etc.
  - Binaries (machine code) on your computer

- **Local applications in dev:** build + run, test, deploy (e.g., docker)
  - Source code in repository (+ dependencies)

- **Web apps server-side:** Browser sends HTTP request (e.g., GET, POST)
  - Code runs remotely (you can only observe outputs)

- **Web apps client-side:** Browser runs JavaScript, event handlers
  - Source code is downloaded and run locally (see: browser dev tools!)

Can running code be Probed/Understood/Edited?

White-box	Grey-box		Black-box	
Source code built locally	Binaries running locally		Server-side apps running remotely	
(P+U+E)	Open source (P+U)	Closed source (P)	Open source (U)	Closed source **(Talk to NSA)**

Creating a model of unfamiliar code: Source code built locally

Information Gathering

Basic needs:

- Code/file search and navigation
- Code editing (probes)
- Execution of code, tests
- Observation of output (observation)

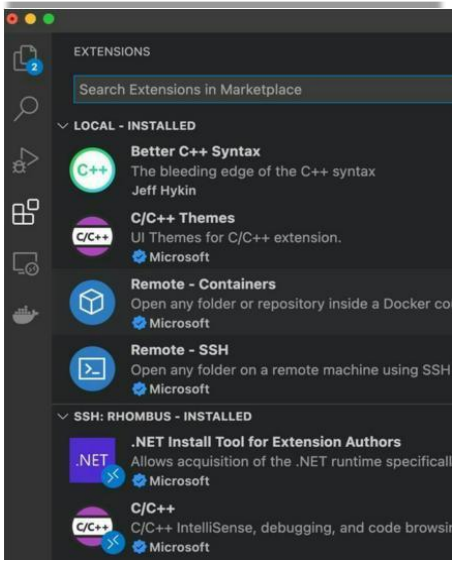
Many choices here on tools! Depends on circumstance.

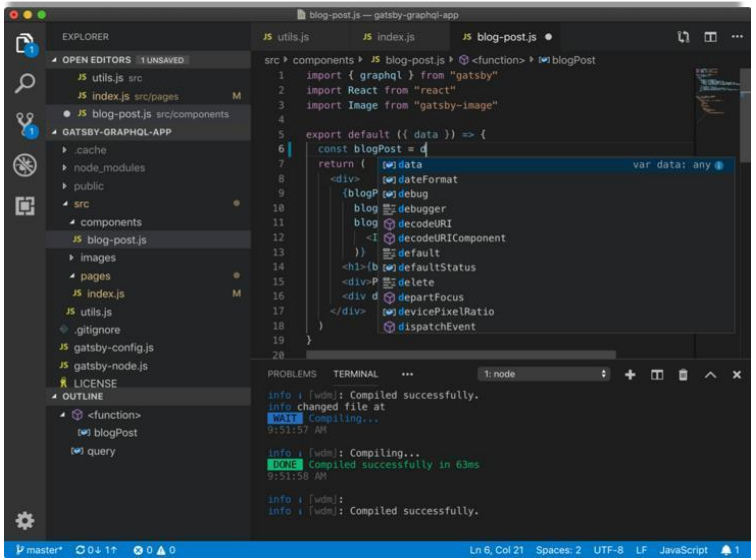
- grep/find/etc. Knowing Unix tools is invaluable
- A decent IDE
- Debugger
- Test frameworks + coverage reports
- Google (or your favorite web search engine)
- ChatGPT

Static Information Gathering: Use an IDE!

Real software is too complex to keep in your head



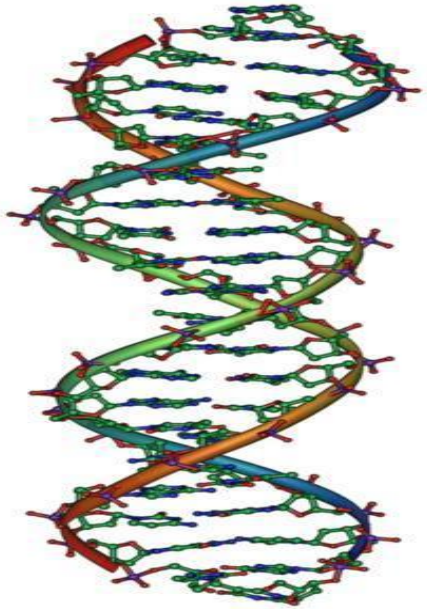




Dynamic Information Gathering

Change helps to inform and refine mental models

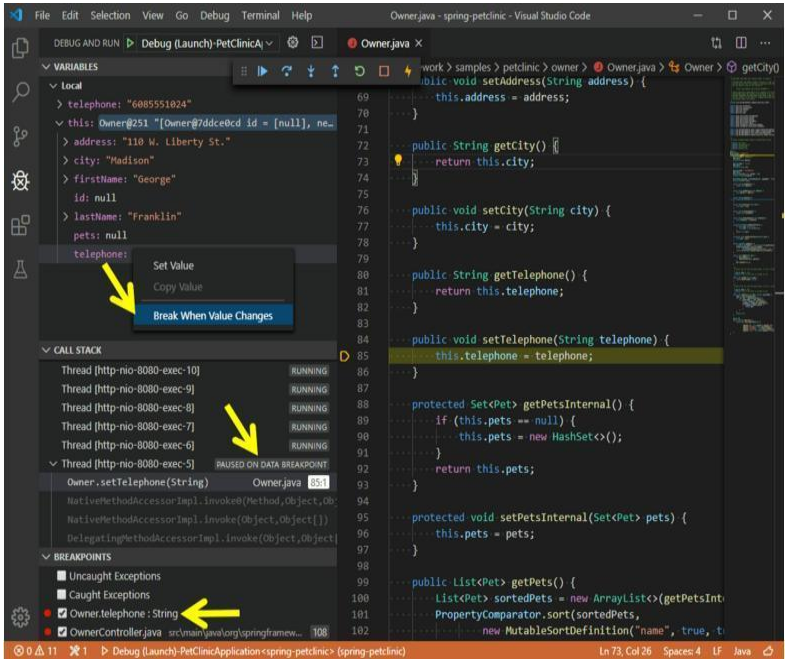
- Build it.
- Run it.
- Change it.
- Run it again.
- How did the behavior change?



Probes: Observe, control or “lightly ” manipulate execution

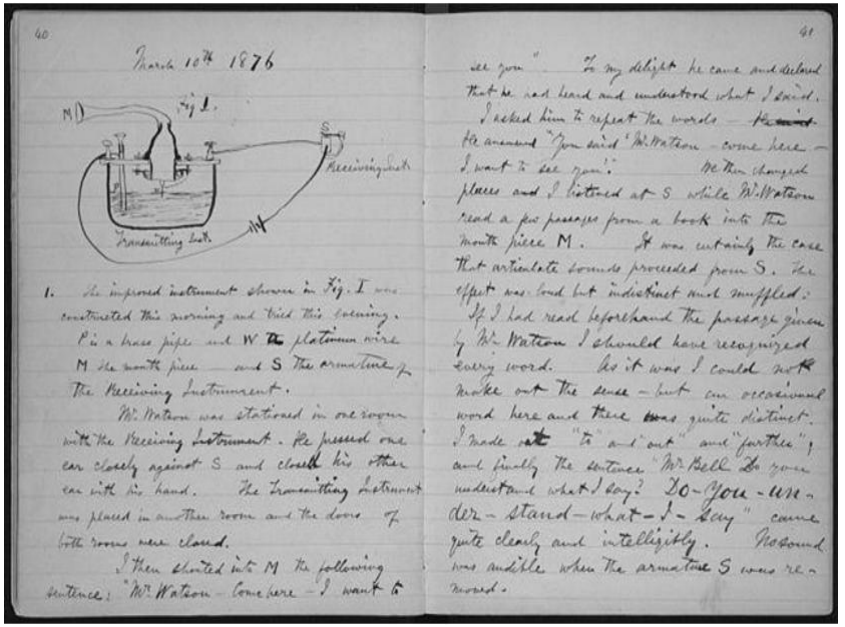
- print(“this code is running!”)
- Structured logging
- Debuggers
- Breakpoint, eval, step through / step over

  (Some tools even support remote debugging)
- Delete debugging
- Chrome Developer Tools



Document and share your findings!

- Update README and docs
  - Or better: use a Developer Wiki
  - Use Mermaid for diagrams
- Screencast on Twitch
- Collaborate with others
- Include negative results, too!



Real Example – let's do it together

- **Redis C Implementation** - a production database system used by millions of applications worldwide
  - 60,000+ stars on GitHub
  - 500,000+ lines of C code
  - 20+ years of development history
  - Used by Netflix, GitHub, Twitter
- **Scenario:** "A production Redis instance is crashing when handling large sorted sets. Your task: understand the sorted set implementation well enough to identify potential memory management issues."

Real Example – let’s do it together

- **Step 1: Clone the Latest Release**
  - git clone https://github.com/redis/redis.git
  - cd redis
- **Step 2: Understanding the File Structure and Readme**
  - Go through README.md
  - server.c — Main file for the server
  - adlist.c, ae.c — list/data/event implementations
  - networking.c, db.c, t_zset.c — networking, db, and sorted set code
  - config.c, rdb.c, etc.
  - Various t_* files for different data types

Real Example – let's do it together

- **Step 3: Finding the Entry Point**
  - Locate int main(int argc, char **argv)
  - Initializes server, parses config, starts event loop
- **Step 4: Commands and Data Types**
  - Data type implementations
    - Sorted Sets: In src/t_zset.c (not in data_structures/)
    - Lists, Hashes: In t_list.c, t_hash.c etc.
  - Command registration table is located in src/server.c
  - Sorted set command handler (e.g., zaddCommand) located in t_zset.c
  - Data structure backing for sorted sets is implemented here (skiplist and dict).

Real Example – let’s do it together

- **Step 5: Typical Bug Tracing Scenario**
  - Scenario: "Redis crashes or leaks memory when adding large sorted sets via ZADD.”
  - **Steps:**
    - Start at server.c, trace how commands are mapped.
    - Follow to t_zset.c, inspect zaddCommand.
    - Dive deeper into memory allocation and management inside sorted set insert/modify functions.
  - **Look for:**
    - Argument parsing and error handling vulnerabilities.
    - Defensive programming (e.g., check malloc/memory allocation, handle NULLs).
    - Cleanups in error conditions.
    - How skiplist nodes and dict entries are managed.

Real Example – let's do it together

- **Step 6: Testing and Validation**
  - Explore the tests/ directory for test scripts and regression tests.
  - Use make test to verify correctness or reproduce bugs.
  - For advanced debugging:
    - Build Redis with debug flags and use valgrind for memory checking

- For bugs, always start with the failing command and trace through its handler, supporting type code, and error management.



Software Engineering

Thank You

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Attribution and Licensing

- Parts of this slideset include content adapted from the CMU 17-313: Foundations of Software Engineering course offered by Carnegie Mellon University.
- This material is used under the terms of the MIT License.
  © 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.

- Parts of the slideset include content derived from various Internet reports and articles.

- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Unit-3 Lecture – 5

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Course Coordinator and Anchor (RR Campus): Prof Vadiraja Acharya
Course Anchor (EC Campus) : Prof M Sheela Devi

Department of Computer Science and Engineering

Software Analysis Tools - Static Analysis

What static analysis can and cannot do

- Type-checking is well established
- Set of data types taken by variables at any point can be used to prevent type errors (e.g. Java) or warn about potential type errors (e.g. Python)
- Checking for problematic patterns in syntax is easy and fast
- Is there a comparison of two Java strings using `==`?
- Is there an array access `a[i]` without an enclosing bounds check for `i`?
- Reasoning about termination is impossible in general (Halting problem) – not possible to identify infinite loops most of the time.

What static analysis can and cannot do

- Reasoning about exact values is hard, but conservative analysis via abstraction is possible
- Is the bounds check before `a[i]` guaranteeing that `i` is within bounds?
- Can the divisor ever take on a zero value?
- Could the result of a function call be `42`?
- Will this multi-threaded program give me a deterministic result?
- Be prepared for “MAYBE”
- Verifying some advanced properties is possible but expensive
- CI-based static analysis usually over-approximates conservatively

Static Analysis is well suited to detecting certain defects

- Security: Buffer overruns, improperly validated input…
- Memory safety: Null dereference, uninitialized data…
- Resource leaks: Memory, OS resources…

Static analysis can be applied to all attributes

- Find bugs
- Refactor code
- Keep your code stylish!
- Identify code smells
- Measure quality
- Find usability and accessibility issues
- Identify bottlenecks and improve performance

Activity: Analyze the Python program statically

```
def n2s(n: int, b: int):
 if n <= 0:
 return '0'
 r = '' "
  while n > 0:
    u = n % b
    if u >= 10:
        u = chr(ord('A') + u-10)
    n = n // b
    r = str(u) + r
    return r
```

- What is the type of variable `u`?
- Will the variable `u` be a negative number?
- Will this function always return a value?
- Will the program divide by zero?
- Will the returned value ever contain a minus sign '-'?

Tools for Static Analysis

























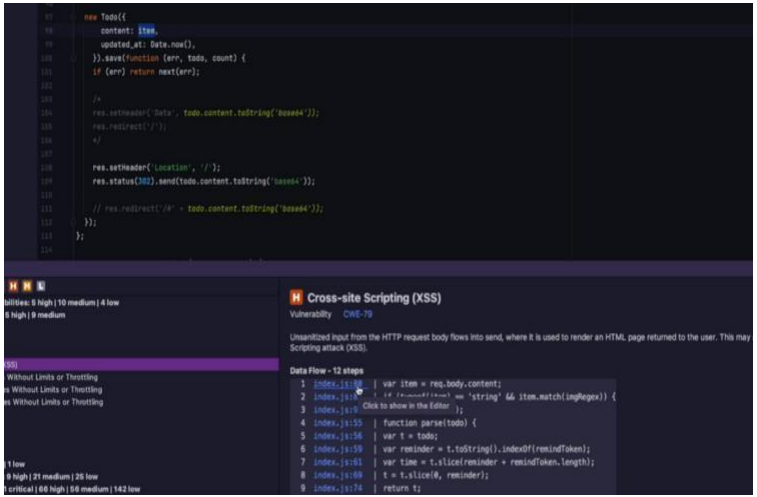


Static analysis is also integrated into IDEs

cppcoreguidelines.cpp

```
1  // To enable only C++ Core Guidelines checks
2  // go to Settings/Preferences | Editor | Inspections | C/C++ | Clang-Tidy
3  // and provide: -*,cppcoreguidelines-* in options
4
5  void fill_pointer(int* arr, const int num) {
6      for(int i = 0; i < num; ++i) {
7          arr[i] = 0;
8      }
9  }
10
11 void fill_array(int ind) {
12     int arr[3] = {1,2,3};
13     arr[ind] = 0;
14 }
15
16 void cast_away_const(const int& magic_num)
17 {
18     const_cast<int&>(magic_num) = 42;
19 }
20
```

Do not use pointer arithmetic



What makes a good static analysis tool?

- Static analysis should be **fast**
  - Don’t hold up development velocity
  - This becomes more important as code scales
- Static analysis should report **few false positives**
  - Otherwise developers will start to ignore warnings and alerts, and quality will decline
- Static analysis should be **continuous**
  - Should be part of your continuous integration pipeline
  - Diff-based analysis is even better -- don’t analyze the entire codebase; just the changes
- Static analysis should be **informative**
  - Messages that help the developer to quickly locate and address the issue
  - Ideally, it should suggest or automatically apply fixes

Static Analysis: Broad classification

- **Linters:**
  - Shallow syntax analysis for enforcing code styles and formatting

- **Pattern-based bug detectors:**
  - Simple syntax or API-based rules for identifying common programming mistakes

- **Type-annotation validators:**
  - Check conformance to user-defined types
  - Types can be complex (e.g., “Nullable”)

- **Data-flow analysis / Abstract interpretation:**
  - Deep program analysis to find complex error conditions (e.g., ”can array index be out of bounds?”)

Linters

- Cheap, fast, and lightweight static source analysis
- Use linters to enforce style guidelines, Don't rely on manual inspection during code review!
- Linters use very "shallow " static analysis to enforce formatting rules
  - Ensure proper indentation
  - Naming convention
  - Line sizes
  - Class nesting
  - Documenting public functions
  - Parenthesis around expressions
  - What else?
- Use linters to improve maintainability
  - Why ? We spend more time reading code than writing it.
  - Various estimates of the exact %, some as high as 80%
  - Code is ownership is usually shared
  - The original owner of some code may move on quickly understand your code

Patten-based Static Analysis Tools

- Bad Practice
- Correctness
- Performance
- Internationalization
- Malicious Code
- Multithreaded Correctness
- Security
- Dodgy Code

SpotBugs

FindBugs
because it's easy

Docs and Info
FindBugs 2.0
Demo and data
Users and supporters
FindBugs blog
Fact sheet
Manual
Manual(ja/日本語)
FAQ
Bug descriptions
Bug descriptions(ja/日本語)
Bug descriptions(fr)
Mailing lists
Documents and Publications
Links

Downloads

FindBugs Swag

Development
Open bugs
Reporting bugs
Contributing
Dev team
API [no frames]
Change log
SF project page
Browse source
Latest code changes

FindBugs Bug Descriptions

This document lists the standard bug patterns reported by FindBugs version 3.0.1.

Summary

Description	Category
BC: Equals method should not assume anything about the type of its argument	Bad practice
BIT: Check for sign of bitwise operation	Bad practice
CN: Class implements Cloneable but does not define or use clone method	Bad practice
CN: clone method does not call super.clone()	Bad practice
CN: Class defines clone() but doesn't implement Cloneable	Bad practice
CNT: Rough value of known constant found	Bad practice
Co: Abstract class defines covariant compareTo() method	Bad practice
Co: compareTo()/compare() incorrectly handles float or double value	Bad practice
Co: compareTo()/compare() returns Integer.MIN_VALUE	Bad practice
Co: Covariant compareTo() method defined	Bad practice
DE: Method might drop exception	Bad practice
DE: Method might ignore exception	Bad practice
DMI: Adding elements of an entry set may fail due to reuse of Entry objects	Bad practice
DMI: Random object created and used only once	Bad practice
DMI: Don't use removeAll to clear a collection	Bad practice
Dm: Method invokes System.exit(...)	Bad practice
Dm: Method invokes dangerous method runFinalizersOnExit	Bad practice
ES: Comparison of String parameter using == or !=	Bad practice
ES: Comparison of String objects using == or !=	Bad practice
Eq: Abstract class defines covariant equals() method	Bad practice
Eq: Equals checks for incompatible operand	Bad practice
Eq: Class defines compareTo(...) and uses Object.equals()	Bad practice
Eq: equals method fails for subtypes	Bad practice
Eq: Covariant equals() method defined	Bad practice
FI: Empty finalizer should be deleted	Bad practice
FI: Explicit invocation of finalizer	Bad practice
FI: Finalizer nulls fields	Bad practice
FI: Finalizer only nulls fields	Bad practice
FI: Finalizer does not call superclass finalizer	Bad practice
FI: Finalizer nullifies superclass finalizer	Bad practice
FI: Finalizer does nothing but call superclass finalizer	Bad practice
FS: Format string should use %n rather than \n	Bad practice
GC: Unchecked type in generic call	Bad practice
HE: Class defines equals() but not hashCode()	Bad practice
HE: Class defines equals() and uses Object.hashCode()	Bad practice
HE: Class defines hashCode() but not equals()	Bad practice
HE: Class defines hashCode() and uses Object.equals()	Bad practice
HE: Class inherits equals() and uses Object.hashCode()	Bad practice
IC: Superclass uses subclass during initialization	Bad practice
IMSE: Dubious catching of IllegalMonitorStateException	Bad practice
ISC: Needless instantiation of class that only supplies static methods	Bad practice
It: Iterator next() method can't throw NoSuchElementException	Bad practice
J2EE: Store of non serializable object into HttpSession	Bad practice
JCIP: Fields of immutable classes should be final	Bad practice
ME: Public enum method unconditionally sets its field	Bad practice

Use type annotations to detect common errors

- Uses static types to prevent meaningless operations from executing in the first place (instead of dealing with bad results later)

- Annotations can enhance type system already in the language

    - Examples: Java Checker Framework, MyPy, TypeScript







Example: Detecting null pointer exceptions

```
// return value
@NonNull String toString() { ... }
// parameter
int compareTo(@NonNull String other)
{ ... }
```

- **@Nullable** indicates that an expression may be null
- **@NonNull** indicates that an expression must never be null
- Guarantees that expressions annotated with **@NonNull** will never evaluate to null, forbids other expressions from being dereferenced

https://checkerframework.org/manual/#nullness-annotations

NASA's Mars Climate Orbiter



Remember the Mars Climate Orbiter incident from 1999?



NASA's Mars Climate Orbiter (cost of $327 million) was lost because of a discrepancy between use of metric unit Newtons and imperial measure Pound -force.

https://www.simscale.com/blog/2017/12/nasa-mars-climate-orbiter-metric/

Limitations of Type-based Static Analysis

- Only considers the signature and annotations of methods
  - Doesn't look at the implementation of methods that are being called
- Can't handle dynamically generated code well
  - Examples: Spring Framework, Templates
- Can produce false positives!
  - Byproduct of necessary approximations
- Can only analyze code that is annotated
  - Requires that dependent libraries are also annotated
    - Can be tricky to retrofit annotations into existing codebases

The best QA strategies employ a combination of tools

How Many of All Bugs Do We Find? A Study of Static Bug Detectors

Andrew Habib
andrew.a.habib@gmail.com
Department of Computer Science
TU Darmstadt
Germany

Michael Pradel
michael@binaervarianz.de
Department of Computer Science
TU Darmstadt
Germany

ABSTRACT

Static bug detectors are becoming increasingly popular and are widely used by professional software developers. While most work on bug detectors focuses on whether they find bugs at all, and on how many false positives they report in addition to legitimate warnings, the inverse question is often neglected: How many of all real-world bugs do static bug detectors find? This paper addresses this question by studying the results of applying three widely used static bug detectors to an extended version of the Defects4J dataset that consists of 15 Java projects with 594 known bugs. To decide which of these bugs the tools detect, we use a novel methodology that combines an automatic analysis of warnings and bugs with a manual validation of each candidate of a detected bug. The results of the study show that: (i) static bug detectors find a non-negligible amount of all bugs, (ii) different tools are mostly complementary to each other, and (iii) current bug detectors miss the large majority of the studied bugs. A detailed analysis of bugs missed by the static detectors shows that some bugs could have been found by variants of the existing detectors, while others are domain-specific problems that do not match any existing bug pattern. These findings help potential users of such tools to assess their utility, motivate and outline directions for future work on static bug detection, and provide a basis for future comparisons of static bug detection with other bug finding techniques, such as manual and automated testing.

International Conference on Automated Software Engineering (ASE '18), September 3–7, 2018, Montpellier, France. ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3238147.3238213

1 INTRODUCTION

Finding software bugs is an important but difficult task. For average industry code, the number of bugs per 1,000 lines of code has been estimated to range between 0.5 and 25 [21]. Even after years of deployment, software still contains unnoticed bugs. For example, studies of the Linux kernel show that the average bug remains in the kernel for a surprisingly long period of 1.5 to 1.8 years [8, 24]. Unfortunately, a single bug can cause serious harm, even if it has been subsisting for a long time without doing so, as evidenced by examples of software bugs that have caused huge economic loses and even killed people [17, 28, 46].

Given the importance of finding software bugs, developers rely on several approaches to reveal programming mistakes. One approach is to identify bugs during the development process, e.g., through pair programming or code review. Another direction is testing, ranging from purely manual testing over semi-automated testing, e.g., via manually written but automatically executed unit tests, to fully automated testing, e.g., with UI-level testing tools. Once the software is deployed, runtime monitoring can reveal so far missed bugs, e.g., collect information about abnormal runtime

Tool	Bugs
Error Prone	8
Infer	5
SpotBugs	18
Total:	**31**
Total of 27 unique bugs	

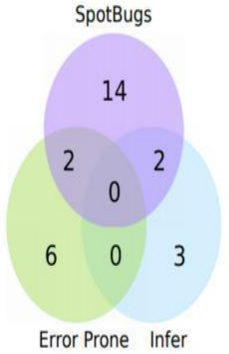


Figure 4: Total number of bugs found by all three static checkers and their overlap.

Which tool to use?

- Depends on use case, available resources
- **Linters**: Fast, cheap, easy to address issues or set ignore rules
- **Pattern-based bugs**: Intuitive, but need to deal with false positives.
- **Type-annotation-based checkers**: More manual effort required; needs overall project commitment. But good payoff once adopted.
- **Deep analysis tools**: Can find tricky issues, but can be costly. Might need some awareness of the analysis to deal with false positives.
- The best QA strategy involves multiple analysis and testing techniques!

Static Analysis vs Dynamic Analysis

Static Analysis

- Requires only source code
- Conservatively reasons about all possible inputs and program paths
- Reported warnings may contain false positives.
- Can report all warnings of a particular class of problems.
- Advanced techniques like verification can prove certain complex properties, but rarely run in CI due to cost.

Dynamic Analysis

- Requires successful build + test inputs
- Observes individual executions
- Reported problems are real, as observed by a witness input
- Can only report problems that are seen. Highly dependent on test inputs. Subject to false negatives
- Advanced techniques like symbolic execution can prove certain complex properties, but rarely run in CI due to cost.



Software Engineering

Thank You

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Attribution and Licensing

- Parts of this slideset include content adapted from the CMU 17-313: Foundations of Software Engineering course offered by Carnegie Mellon University.
- This material is used under the terms of the MIT License.
  © 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.

- Parts of the slideset include content derived from various Internet reports and articles.

- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Unit-3 Lecture – 6

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Course Coordinator and Anchor (RR Campus): Prof Vadiraja Acharya
Course Anchor (EC Campus) : Prof M Sheela Devi

Department of Computer Science and Engineering

Static Analysis - In-Class Activity

Static Analysis vs Dynamic Analysis

Static Analysis

- Requires only source code
- Conservatively reasons about all possible inputs and program paths
- Reported warnings may contain false positives.
- Can report all warnings of a particular class of problems.
- Advanced techniques like verification can prove certain complex properties, but rarely run in CI due to cost.

Dynamic Analysis

- Requires successful build + test inputs
- Observes individual executions
- Reported problems are real, as observed by a witness input
- Can only report problems that are seen. Highly dependent on test inputs. Subject to false negatives
- Advanced techniques like symbolic execution can prove certain complex properties, but rarely run in CI due to cost.

Example 1: Bad C Code

```
#include <stdio.h>
#include <string.h>

void copy_string(char *dest, const char *src, int maxlen) {
    strncpy(dest, src, maxlen);
    dest[maxlen - 1] = '\0';
    printf("Copied string: %s\n", dest);
}
```

Example 1: Issues Identified

```
#include <stdio.h>
#include <string.h>

void copy_string(char *dest, const char *src, int maxlen) {
    strncpy(dest, src, maxlen);
    dest[maxlen - 1] = '\0';
    printf("Copied string: %s\n", dest);
}
```

- No null check for `dest` or `src`
- No check for `maxlen <= 0`
- Assumes `dest` has sufficient memory allocated
- No return value or error reporting

Example 1: Improved C Code

```
void copy_string(char *dest, const char *src, int maxlen) {
    if (!dest || !src || maxlen <= 0) {
        fprintf(stderr, "Invalid input\n");
        return;
    }
    strncpy(dest, src, maxlen - 1);
    dest[maxlen - 1] = '\0';
    printf("Copied string: %s\n", dest);
}
```

Example 1: Real-World Incidents

- Heartbleed (OpenSSL): unchecked length leading to info leak
- Apple goto fail bug: poor error signaling
- Null dereference crashes in production

Example 2: Bad C Code

```
void format_name(char *first_name, char *last_name) {
    strcat(first_name, " ");
    strcat(first_name, last_name);
    for (int i = 0; last_name[i]; ++i) {
        if (last_name[i] >= 'a' && last_name[i] <= 'z') {
            last_name[i] -= 32;
        }
    }
    printf("Formatted Name: %s\n", first_name);
}
```

Example 2: Issues Identified

```
void format_name(char *first_name, char *last_name) {
    strcat(first_name, " ");
    strcat(first_name, last_name);
    for (int i = 0; last_name[i]; ++i) {
        if (last_name[i] >= 'a' && last_name[i] <= 'z') {
            last_name[i] -= 32;
        }
    }
    printf("Formatted Name: %s\n", first_name);
}
```

- Modifies input parameters
- Potential buffer overflow with strcat
- Mutates input without returning result

Example 2: Improved C Code

```
int format_name(const char *first, const char *last, char *out, int size) {
    if (!first || !last || !out || size <= 0) return -1;
    char upper_last[64];
    for (int i = 0; last[i] && i < 63; ++i)
        upper_last[i] = toupper((unsigned char)last[i]);
    upper_last[63] = '\0';
    snprintf(out, size, "%s %s", first, upper_last);
    return 0;
}
```

Example 2: Real-World Incidents

- OpenSSL API misuse with mutable objects
- Microsoft GDI buffer corruption
- Mutation causing state corruption

Example 3: Bad C Code

```
void delete_file(const char *filename) {
    char command[256];
    sprintf(command, "rm -f %s", filename);
    system(command);
    printf("Deleted file: %s\n", filename);
}
```

Example 3: Issues Identified

```
void delete_file(const char *filename) {
    char command[256];
    sprintf(command, "rm -f %s", filename);
    system(command);
    printf("Deleted file: %s\n", filename);
}
```

- Command injection via `filename`
- No length or character checks
- `sprintf` buffer overflow possible
- Uses `system()` instead of safe APIs

Example 3: Improved C Code

```
int delete_file(const char *filename) {
    if (!filename || strlen(filename) > 128) return -1;
    for (const char *p = filename; *p; ++p)
        if (strchr(";&|><`", *p)) return -2;
    return unlink(filename);
}
```

Example 3: Real-World Incidents

- Shellshock vulnerability
- CUPS daemon command injection
- Admin tools with unsafe shell construction

Example 4: Good C Code

```
int safe_concat(const char *a, const char *b, char *out, size_t out_size) {
    if (!a || !b || !out || out_size == 0) return -1;
    if (strlen(a) + strlen(b) + 1 >= out_size) return -2;
    snprintf(out, out_size, "%s%s", a, b);
    return 0;
}
```

Example 4: Why This Is Good

```
int safe_concat(const char *a, const char *b, char *out, size_t out_size) {
    if (!a || !b || !out || out_size == 0) return -1;
    if (strlen(a) + strlen(b) + 1 >= out_size) return -2;
    snprintf(out, out_size, "%s%s", a, b);
    return 0;
}
```

- Input validation and size checks
- Safe string handling (`snprintf`)
- Constant correctness and return codes
- Defensive and testable

Example 4: Best Practices

- Portable and testable
- No global state or side effects
- Clearly separates responsibility

Example 5: Good Python Code

```
def safe_divide(a: float, b: float) -> float:
    if not isinstance(a, (int, float)) or not isinstance(b, (int, float)):
        raise TypeError("Inputs must be numbers.")
    if b == 0:
        raise ValueError("Division by zero is not allowed.")
    return round(a / b, 2)
```

Example 5: Why This Is Good

- Uses type hints and validation
- Raises exceptions instead of silent failures
- Follows Pythonic practices (EAFP)
- Clean and testable

Example 5: Best Practices

- Uses idiomatic constructs
- Clean separation of concerns
- Robust exception handling

Example 6: Bad Python Code

```
def normalize_scores(scores):
    total = sum(scores)
    for i in range(len(scores)):
        scores[i] = scores[i] / total
    print("Normalized:", scores)
```

Example 6: Issues Identified

```
def normalize_scores(scores):
    total = sum(scores)
    for i in range(len(scores)):
        scores[i] = scores[i] / total
    print("Normalized:", scores)
```

- Mutates input list directly
- No check for sum == 0 (ZeroDivisionError)
- No type or structure checks
- Mixes logic with print/output

Example 6: Improved Python Code

```
def normalize_scores(scores):
    if not scores or sum(scores) == 0:
        return []
    return [s / sum(scores) for s in scores]
```

Example 6: Real-World Incidents

- Shared state mutation in ML pipelines
- Bugs from unexpected input changes
- Reduced testability and reuse



Software Engineering

Thank You

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Attribution and Licensing

- Parts of this slideset include content adapted from the CMU 17-313: Foundations of Software Engineering course offered by Carnegie Mellon University.
- This material is used under the terms of the MIT License.
  © 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.

- Parts of the slideset include content derived from various Internet reports and articles.

- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Unit 1 – Lecture 7

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Course Coordinator and Anchor (RR Campus): Prof Vadiraja Acharya
Course Anchor (EC Campus) : Prof M Sheela Devi

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Continuous Integration & Continuous Deployment

What does a developer do?

The code is written … now what?

- Get the source code
- Install dependencies
- Run static analysis
- Compile the code
- Generate documentation
- Run tests
- Create artifacts for customers
- Ship!
- Operate, monitor, repeat

Which of these tasks should be handled manually?

What does a developer do?

The code is written … now what?

- Get the source code
- Install dependencies
- Run static analysis
- Compile the code
- Generate documentation
- Run tests
- Create artifacts for customers
- Ship!
- Operate, monitor, repeat

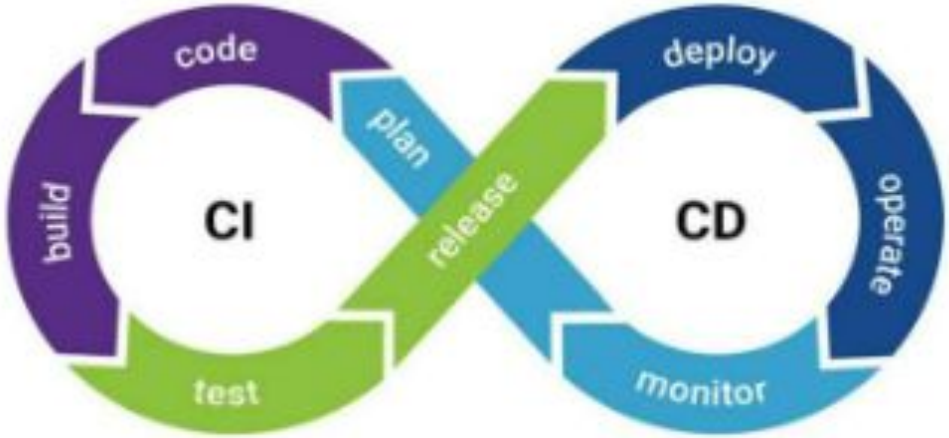


Which of these tasks should be handled manually?

NONE!

Instead, Orchestrate with Tools

- Build system – covers dependency management and actual build
- Test tools – automate testing as much as possible
- Code review, analysis tools
- Deployment of build



Instead, Orchestrate with Tools

catch
Can we ~~remove~~ human error?

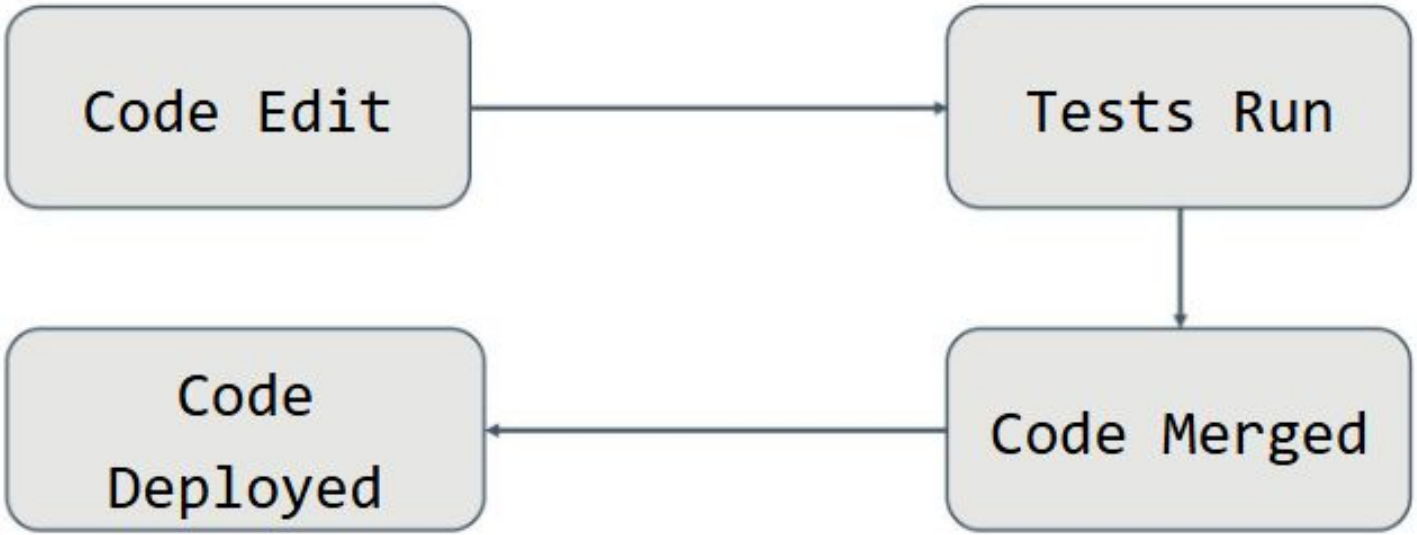
Can we catch human error before we ship our code?
Can we automate tasks to prevent problems?

Instead, Orchestrate with Tools

Approach: Automate what we can

Review what we cannot

CI/CD Pipeline overview



Continuous Integration

Catch mistakes before you push your code!

History of CI



(1999) Extreme Programming (XP) rule: “Integrate Often”



(2000) Martin Fowler posts “Continuous Integration” blog

cruisecontrol

(2001) First CI tool



(2005) Hudson/Jenkins

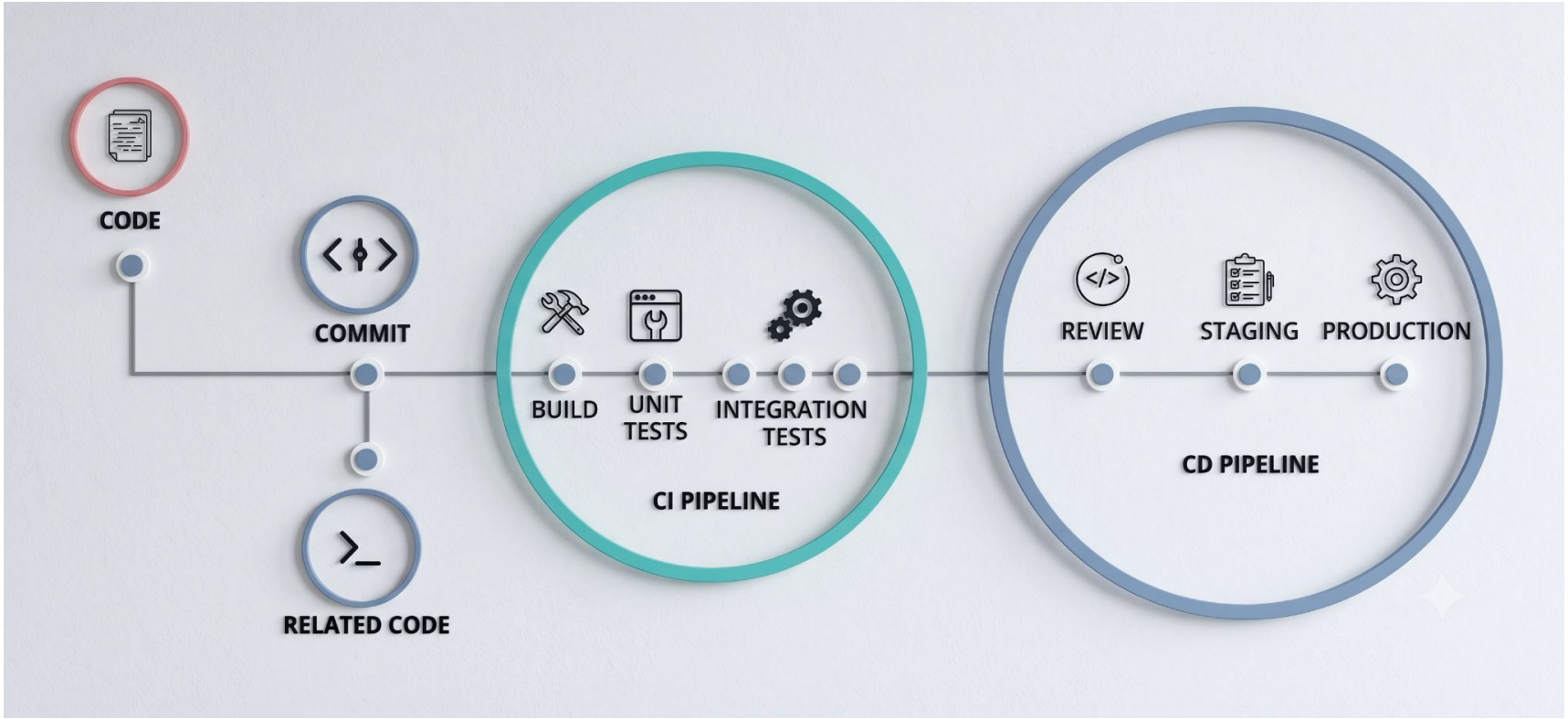


(2011) Travis CI



(2019) GitHub Actions

Example CI/CD Pipeline



Developers say:

- CI helps us catch bugs earlier
- CI makes us less worried about breaking our builds
- CI lets us spend less time debugging

- *“[CI] does have a pretty big impact on [catching bugs]. It allows us to find issues even before they get into our main repo, ... rather than letting bugs go unnoticed, for months, and letting users catch them.”*

Best Practices for Build

For problems we can’t easily automate, we can perform code review

Best Practices for Build

- Automate everything that is feasible
- One-step build
- CI tools to build and test code on every commit
- All dependencies captured in build files
- Don’t ever break the build! Use git branching to ensure main branch stays working

Good Build System

- **Dependency management**
  - Identifies dependencies between files (including externals)
  - Compilations in the right order to pick up the right dependencies
  - Compile only what is needed

- **Efficiency and reliability**
  - Automation to help all deliverables to come together and transition from development to shipping code quickly.
  - Eliminates the chance of missing steps due to tribal knowledge and/or simple errors

A build system has three main roles

1. Defines **tasks** (and external resources, such as libraries)

2. Defines **dependencies** among tasks (a graph)

3. **Executes** the tasks

Example – Event Manager App

- **Defines tasks:** "What needs to happen to get your project running?"
  - Install dependencies
  - Set up database
  - Run static analysis
  - Execute tests
- **Defines dependencies:** "What order must these happen in?"
  - Database setup before running app
  - Dependencies before tests
  - Static analysis can run in parallel
- **Executes tasks:** "How do we ensure reliability?"
  - Same environment every time
  - Clear success/failure indicators
  - Automatic retry on temporary failure

Build systems can determine task order

Large projects have thousands of tasks

- Dependencies between tasks form a directed acyclic graph
- Build tools use a topological sort to create an order to compiles
  - Order nodes such that all dependencies are satisfied
  - Implemented by computing indegree (number of incoming edges) for each node
  - No dependencies go first and open door to the others

External code (libraries) also can be complex

- Build systems can manage these dependencies as well!

Pros and cons of Continuous Integration

Pros:

- The advantage of continuous integration is that it allows problems caused by the interactions between different developers to be discovered and repaired as soon as possible.
- The most recent system in the mainline is the definitive working system.

Cons:

- If the system is very large, it may take a long time to build and test, especially if integration with other application systems is involved.
- If the development platform is different from the target platform, it may not be possible to run system tests in the developer's private workspace.

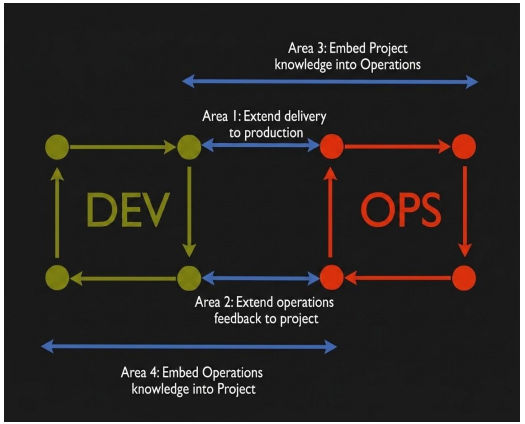
DevOps - Introduction

An introductory video:
https://www.youtube.com/watch?v=0yWAtQ6wYNM

DevOps - Introduction

- DevOps is the combination of cultural philosophies, practices, and tools that increases an organization's ability to deliver (Deploy and Support) applications and services at high velocity and become more effective
- It is the blending of the terms development and operations
- Represent a collaborative or shared approach to the tasks performed by a company's **Application Development** teams (following SDLC) and **IT operations** teams
- The term DevOps is also used as an operational philosophy that promotes better communication and knowledge between the above two Dev and Ops teams





What is DevOps?

- DevOps looks to make the enterprise processes faster, more efficient, and more reliable with the intention of increasing the **business value**
- Repetitive manual labor (error prone) is removed whenever possible
- DevOps in a sense is an extension of the agile way of doing things where you can deliver functionality incrementally and at a faster rate



Which Agile manifesto principle do you think DevOps follows?

DevOps: Concepts and terminologies

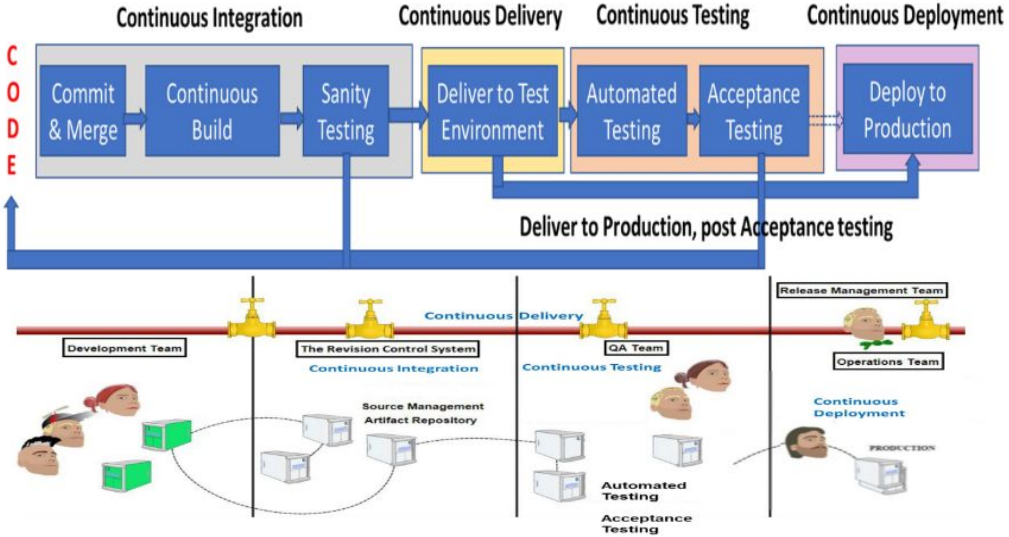
- Recap some of the terminologies you have already learnt which are also used in DevOps:
  - SDLC (Waterfall model, Agile approach)
  - Operations Methodologies (IT Service Management, Information Technology Infrastructure Library)
  - SCM(Supply Chain Management) terminologies associated with Development, Release and Deployment concepts

- **Version Control**
  - Version control systems like Git record changes to files or sets of files stored within the system
  - Developers can make changes individually or within groups and save these changes through commits or revisions within the system in one way or another
  - Ability to commit, compare, merge & restore past revisions to objects to the repository allows for richer cooperation and collaboration within and between teams and minimizes the risk

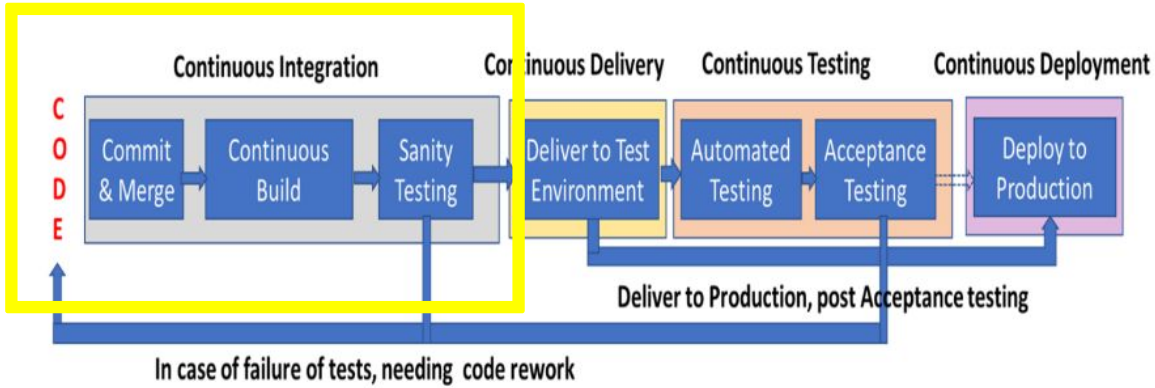
DevOps pipeline

What is the DevOps pipeline?

A DevOps pipeline is a set of automated processes and tools that allows both developers and operations professionals to work cohesively to build and deploy code to a production environment.

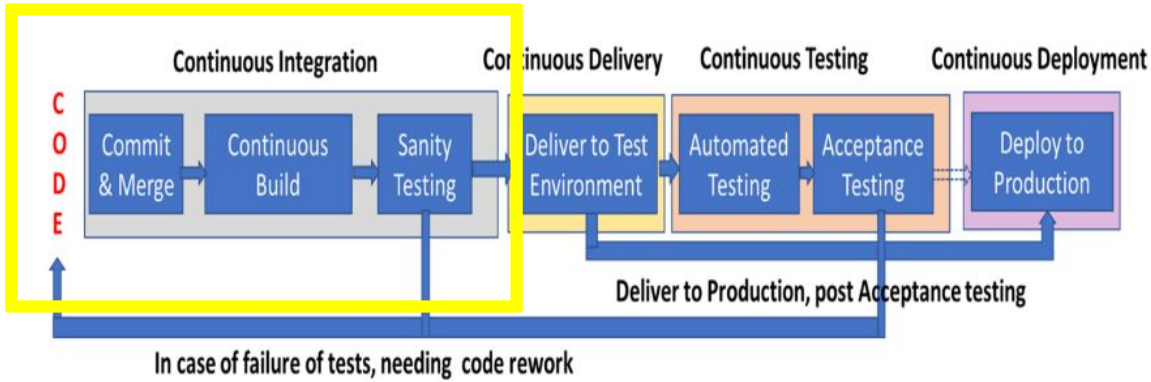


DevOps Pipeline: Continuous Integration



- **Continuous Integration (CI)** involves continuously merging the code written by developers with a mainline or "master" branch, frequently throughout the day
- Developers take a snap shot or branch code from a master branch and work on their individual components and check-in and merge the changes back into the mainline
- Merging of the code can happen multiple times through the day or in some periodicity and during merge if they find a merge conflicts, its resolved and then the code is merged

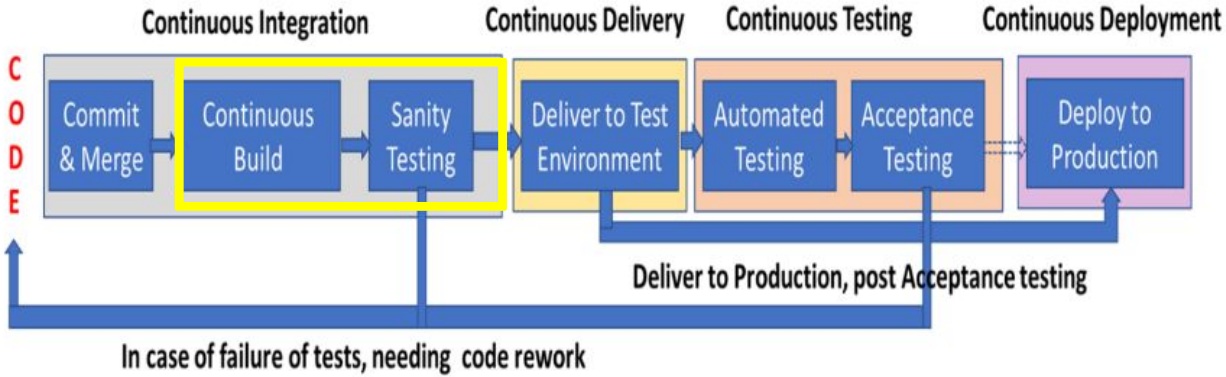
DevOps Pipeline: Continuous Integration



- **Benefits of CI:**
  - Change sets are small and hence, work for resolving conflicts would be small & incremental
  - Supports detecting of issues early
  - Avoids difficult integrations, increased visibility and communications
  - Reduced debugging time

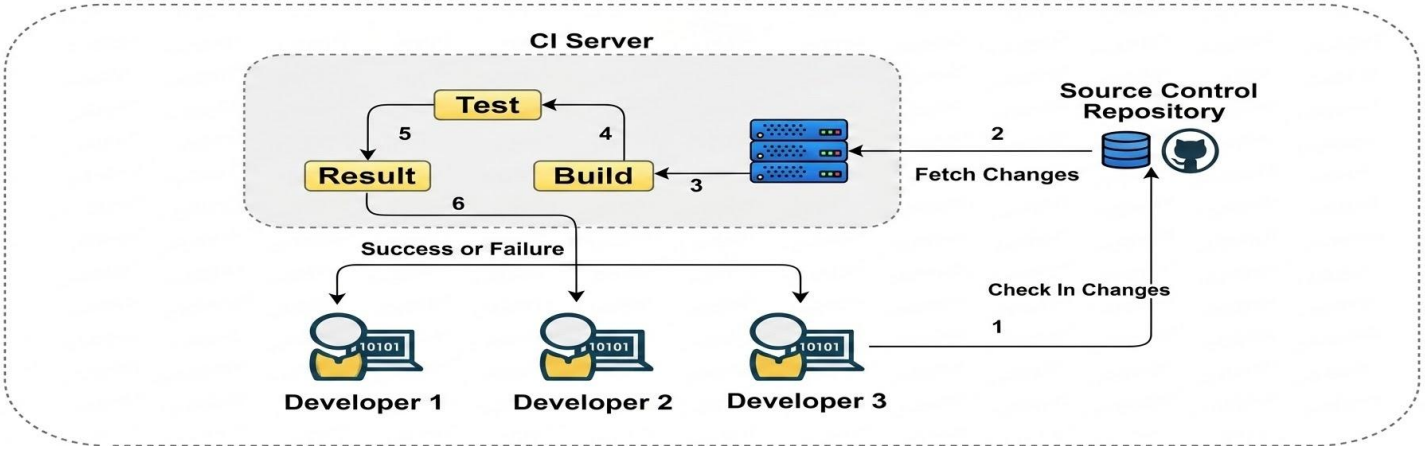
What would happen if code is not merged frequently?

DevOps Pipeline: Continuous Integration



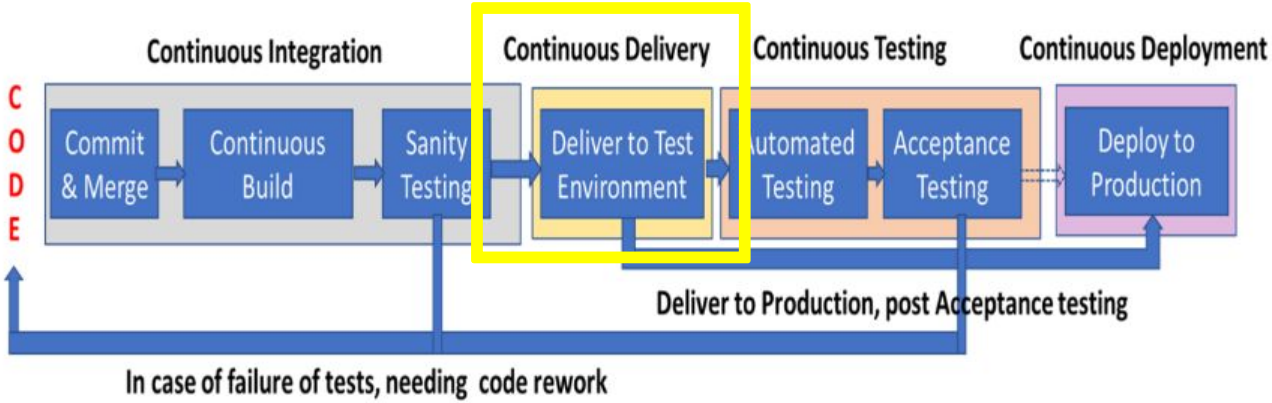
- **Continuous Build**
  - Build involves compiling, handling dependencies and involves generating an executable product
  - This may also involve running some static code analysis tools like Lint

DevOps Pipeline: Continuous Integration



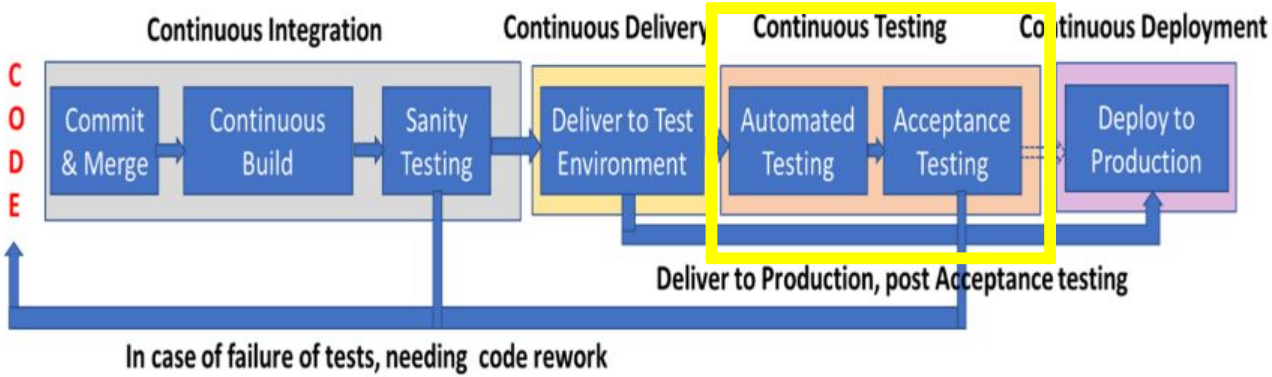
- **Sanity testing**
  - The executable built, is run with some sanity tests to ensure that the total compilation is successful, the dependencies are all resolved and the executable is in an executable state.

DevOps Pipeline: Continuous Delivery



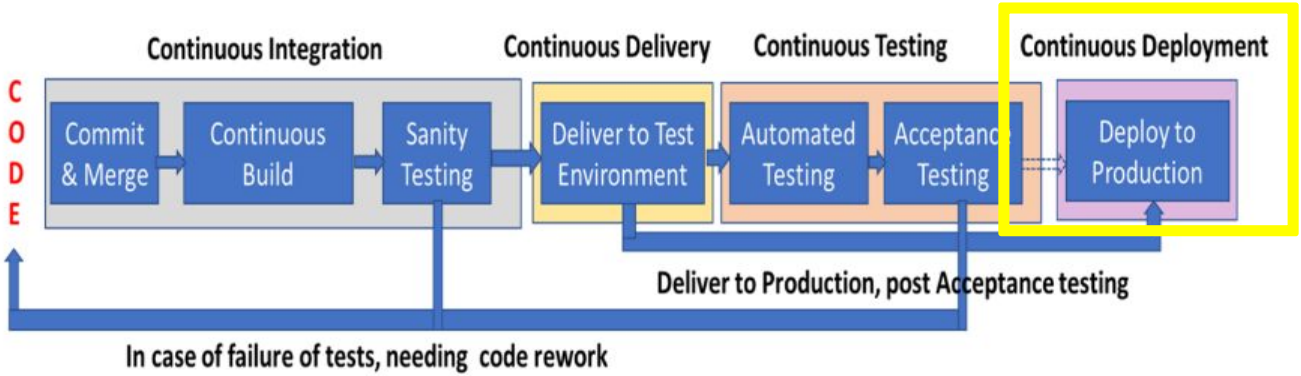
- **Continuous Delivery** is the frequent shipping of code to a given environment
  - Integrated bits are ready to be deployed & this supports release of the new changes to your test teams/customers quickly in a sustainable way
  - Process may be triggered by the last step in the continuous integration process
  - Drives the Business outcomes like deploy on demand, and supports faster time to market and of higher quality and may use tools like Jenkin Plugins

DevOps Pipeline: Continuous Testing



- **Continuous testing** is the process of executing predominantly automated tests for validating the code to ensure that the application works as envisaged and is free of bugs or defects and can be continuously deployed
  - Designed to be executed with minimum wait times and provide the earliest possible feedback & support detection (or prevention) of the risks
  - Entry criteria for testing by the QE process groups would be successful completion of the unit and integration tests
  - In the QA environment, these could include some static code analysis and validation of both functional requirements and non-functional requirements

DevOps Pipeline: Continuous Deployment



- **Continuous Deployment** (CD) is a software release process
  - Uses the delivery mechanisms for deploying the automated testing validated product, immediately and autonomously to a production environment
  - Typically used in highly mature DevOps teams
  - Significant number of times, validated integrated components are batched together and are then deployed into customer environment

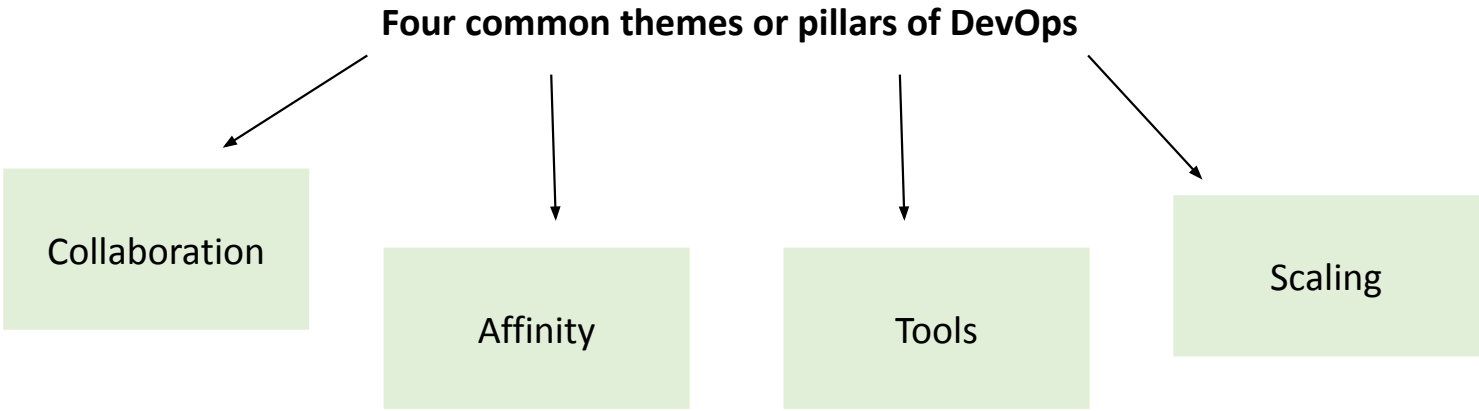
Agile and DevOps - Comparison

DevOps	Agile
A culture and approach which looks to remove the silos of Development activities of building a product and the Operations activities of deployment, support and upkeep.	A methodology or a type of iterative SDLC which as a process, supports changes, ensures more collaborations between developers and other stakeholders, reduces the planning overhead and delivers products or part of products periodically say once 2-4 weeks.
Focuses on increasing the velocity of deployment of products facilitating it by communication, collaboration, automation, usage of tools and integration among IT professionals.	Delivers products or part of products periodically say once 2-4 weeks while reducing planning overhead.
Focuses on constant testing and delivery.	Focuses on constant changes.
The target area is to give end-to-end business solutions and fast delivery.	The target area is Software development.
Focuses more on operational and business readiness.	Focuses on functional and non-function readiness.

Agile v/s DevOps

Pointers	DevOps	Agile
Definition	DevOps brings the development and operations team together.	Agile focuses on the continuous iterative approach with customer feedback, effective collaboration, small and rapid releases.
Goal	It manages the end-to-end development process.	It manages complex projects.
Task	It focuses on constant testing and delivery.	It focuses on constant changes.
Team Size	A large team size – all the stack holders.	It consists of a small team.
Team Skillset	Divides and spreads the skillset between the team of developers and operations.	Everyone has similar and equal skills.
Implementation	It offers a collaborative approach as such doesn't have any framework.	It has many frameworks – safe, scrum, Lean, and sprint.
Delivery	It provides continuous delivery daily or every few hours.	It provides incremental deployments after each sprint.
Delivery	It provides continuous delivery daily or every lew hours.	It provides incremental deployments after each sprint.
Documentation	Very light documentation.	Sufficient documentation for better team collaboration.
Focuses	Operational and business readiness.	Functional and non-functional readiness.
Importance	Development, testing, and implementation are equally important.	Developing software is inherent to Agile.
Quality and Risk	High-quality products with low risk due to automated testing and collaboration.	Product quality increases while the risk decreases after every sprint.
Popular Tools Used	TeamCity, Docker, GitLab, Puppet, Chef, AWS, Ansible.	Bugzilla, Kanboard, Active Collab, Slack, Trello, and JIRA.
Automation	The primary goal of DevOps is automation.	Agile does not emphasise on automation.
Feedback	Focuses on internal feedback to improve and enhance delivery.	Focuses on customer feedback and adjusts the product accordingly.

DevOps: Supporting pillars



The combination of these pillars enables addressing of the cultural and technical aspects of this DevOps organization

DevOps - Collaboration

- **Collaboration** is the process of working towards a specific outcome through supporting interactions and the input and support of multiple people & groups

- Effective collaboration involves
  - Theory of Mind
  - Equal participation
  - Communication

- <u>Theory of Mind</u>
  - is the ability to recognize one's perspective and understanding that others have distinct and different perspective based on their own context

- Considering different aspects of getting people to work together more effectively could be by having:
  - A shared goal
  - Empathy of a collaborative culture through effective communication
  - Teams are effective when based on relationships built on trust, empathy & reciprocity
  - Where hierarchies are less emphasized

DevOps Pillars - Affinity

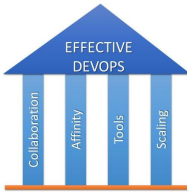
- There needs to exist a strong relationship between individuals, teams and departments along with collaborative relationships to ensure the Dev and Ops groups work together

- This is achieved by having teams with **strong affinity** between them
  - **Affinity** is the measure of the strength of the relationship between individuals, teams, business units, and even companies

  - **Building Affinity** is the process of building these inter-team relationships, navigating differing goals or metrics while keeping in mind shared organizational goals, and fostering empathy and learning between different groups of people

- Affinity is achieved by having:
  - Shared Values
  - Consistent team culture
  - Team cohesion

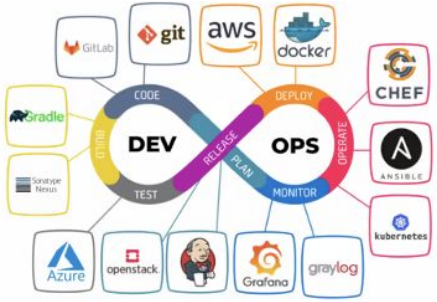
- Affinity can be measured by several factors like
  - Shared Time
  - Intensity of the relationships
  - Reciprocity of shared stories
  - Reciprocity of support





DevOps Pillars - Tools

- **Tools** are important in the DevOps environment as they act as an accelerator, driving change based on the current culture and direction
- Tools & their ease of use impacts the acceptance and proliferation of specific aspects of the culture as they become the common language between teams
- If the cost of collaboration is high, not investing in tools (or worse, investing in poor tools) raises this cost
    - **Examples of tools :** System provisioning tools, Build Tools, Automation tools, Testing tools, Monitoring tools, Log extraction tools, Deployment tools etc.





DevOps Pillars - Scaling

- **Scaling** has a focus on the processes and pivots that organizations must adopt throughout their lifecycles
- Scaling takes into account how the other pillars of effective DevOps can be applied throughout organizations as they grow, mature, and even shrink in large organizations
- Scaling could be for *organization, infrastructure, teams (hiring, retention, outsourcing), complexity, workload*





Tools used in DevOps

Infrastructure Automation	1. Cobbler 2. Foreman 3. Crowbar
Configuration Management	1. Puppet 2. SaltStack 3. Chef
Continuous Integration	1. Jenkins, Hudson 2. SVN, Git, Perforce 3. Ant, Maven
Continuous Deployment	1. Capistrano 2. Custom Tools 3. Yum, Deb, RPM
Monitoring	1. Nagios, Sensu, Zabbix 2. Custom Tools
Revision control System	1. Git 2. Cvs 3. Subversion
Software Configuration Management	1. Clearcase 2. perforce 3. Accurev
virtualization management Software	1. Vcentre 2. Hyper-V

DevSecOps

- DevSecOps, which is short for *development*, *security* and *operations*, is an application development practice that automates the integration of security and security practices at every phase of the software development lifecycle, from initial design through integration, testing, delivery and deployment.

- DevSecOps represents a natural and necessary evolution in the way development organizations approach security.

- In the past, security was 'tacked on' to software at the end of the development cycle, almost as an afterthought. A separate security team applied these security measures and then a separate quality assurance (QA) team tested these measures.

DevSecOps …

- DevSecOps integrates application and infrastructure security seamlessly into Agile and DevOps processes and tools.
- It addresses security issues as they emerge, when they're easier, faster, and less expensive to fix, and before deployment into production.
- Additionally, DevSecOps makes application and infrastructure security a shared responsibility of development, security and IT operations teams, rather than the sole responsibility of a security silo.
- It enables "**software, safer, sooner**"—the DevSecOps motto–by automating the delivery of secure software without slowing the software development cycle.

DevSecOps …

- Incorporating security continuously across the SDLC helps DevOps teams deliver secure applications with speed and quality.
- The earlier security can be included in the workflow, the sooner security weaknesses and vulnerabilities can be identified and remedied.
- This concept is sometimes called “shifting left” because it moves security testing toward developers, enabling them to fix security issues in their code as they develop, rather than waiting until the end of the cycle, when it had traditionally been done.
- By contrast, DevSecOps spans the entire SDLC, from planning and design to coding, building, testing, and release, with real-time continuous feedback loops and insights.

DevSecOps

Benefits

- **Finding issues early.** Finding issues before they're pushed further into the SDLC reduces your chances of them slipping through to production.
- **Fixing issues faster.** Automating testing and orchestrating by policy, when combined with closed feedback loops between your security and development teams, helps your teams cut though findings noise, prioritize efficiently, and speed up remediation.
- **Reducing the window of attack opportunity.** Abbreviating the time span between detection and remediation means malicious actors have a much smaller opportunity to gain access.
- **Increasing the ability to scale.** Integrating testing into your development pipeline and managing with automated policy gives you the flexibility to scale up or down without sacrificing development velocity.

DevSecOps

Challenges

- Implementing DevSecOps can pose some challenges for organizations when they are getting started. Software development involves various technologies, including frameworks, languages, and architectures that have their own unique way of operating and being developed. This can make it challenging for security teams to continuously test and monitor them at the speed required.

- Combining these development tools and techniques with improperly configured security testing mechanisms can easily cause pipelines to become brittle. Brittle pipelines can break when a part goes down or automations fail. This is an unfortunately likely outcome if security teams fail to manage all the triggered events and the policies that govern them, which can be complex and time-consuming.

DevSecOps

- Lastly, risks can be introduced anywhere along the pipeline, so it's important to implement security checks throughout the software development process to ensure that any new issues that manifest within the pipeline are detected as early as possible.

- It can, however, be difficult for teams to coordinate and manage the variety of security checks required, due to the complex conditions listed above and impediments to visibility and priority that come from distributed development and organizational nuances of DevSecOps.



Software Engineering

Thank You

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Attribution and Licensing

- Parts of this slideset include content adapted from the CMU 17-313: Foundations of Software Engineering course offered by Carnegie Mellon University.
- This material is used under the terms of the MIT License.
  © 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.

- Parts of the slideset include content derived from various Internet reports and articles.

- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Unit 3: Lecture 8

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Course Coordinator and Anchor (RR Campus): Prof Vadiraja Acharya
Course Anchor (EC Campus) : Prof M Sheela Devi

Department of Computer Science and Engineering

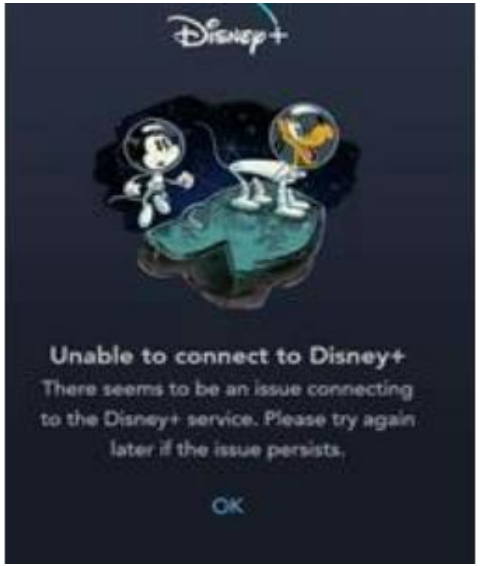
Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

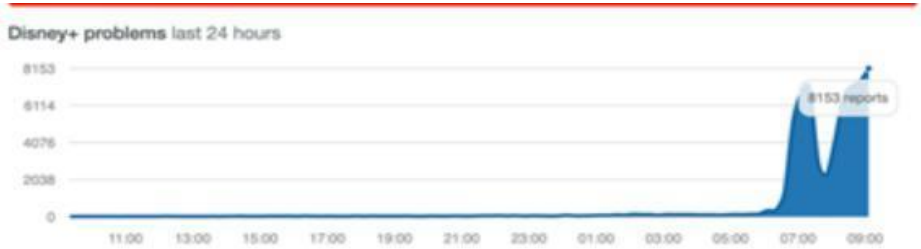
Testing

Real Issues: Disney+ Launch

- Lots of issues reported on launch day.
- Disney had planned for a spike in traffic.
- Tested massive concurrent video streaming capability.

- What do you think was the biggest problem?"
  - Not enough servers
  - Too many users
  - Tested the wrong thing
  - Bad code

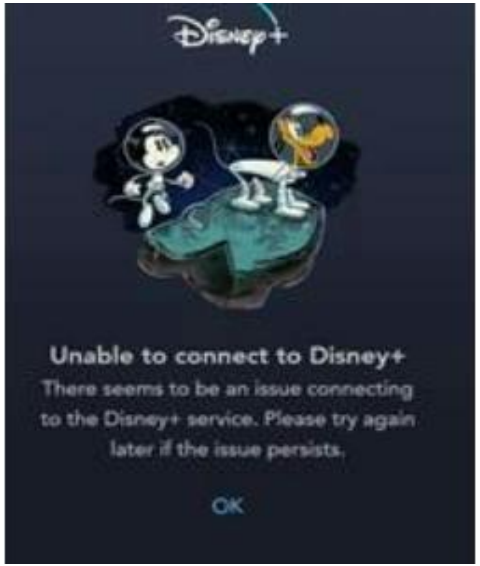


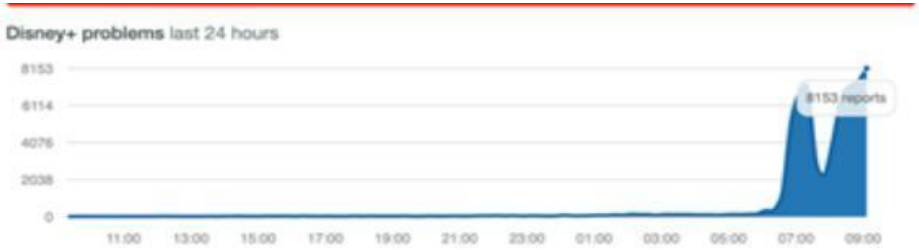


Real Issues: Disney+ Launch

- Lots of issues reported on launch day.
- Disney had planned for a spike in traffic.
- Tested massive concurrent video streaming capability.

- BUT: the stress was in paths other than streaming
  - User account creation
  - Logins and auth
  - Browsing old titles





Principles of Testing #1

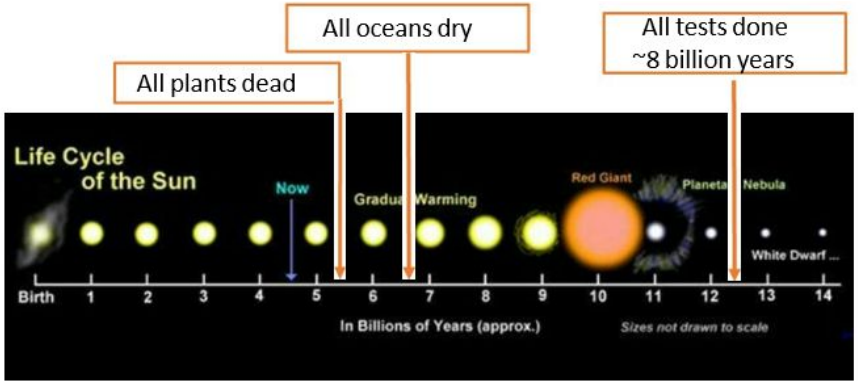
- Testing shows presence, not absence, of defects
- If you run 1000 tests and all pass, does it mean your code has no bugs?
  - No. Testing never proves absence.
- Example: Ariane 5 rocket exploded despite passing all tests (cost: $370M)
- Testing only PROVES defects exist.

Principles of Testing #2

- Exhaustive testing is impossible.
- A simple function, 1 input, string max, 26 lowercase characters + symbols (@, _, ‘)
- Assume we can use 1 zettaFLOPs: 10^{21} tests per second

```
1 def is_valid_email(email: str) -> bool:
2     ...
```

- Even for small problems, exhaustive testing is impractical. We need heuristics!



Principles of Testing #3

- Start Testing Early
  - To let tests guide design
  - To get feedback as early as possible
  - To find bugs when they are cheapest to fix
  - To find bugs when have caused least damage

Principles of Testing #4

- Defects are usually clustered
  - “Hot” components requiring frequent change, bad habits, poor developers, tricky logic, business uncertainty, innovative, size, …
  - Use as heuristic to focus test effort
  - 10 to 20% of modules have 80% of bugs



Principles of Testing #5

- The pesticide paradox

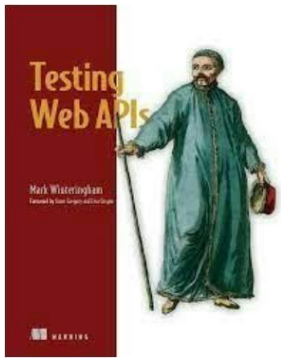
"Every method you use to prevent or find bugs leaves a residue of subtler bugs against which those methods are ineffectual."

- Re-running the same test suite again and again on a changing program gives a false sense of security

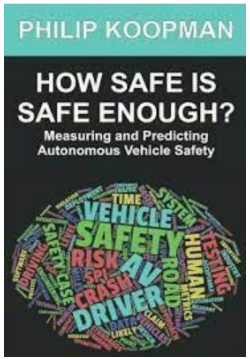
- Variation in testing

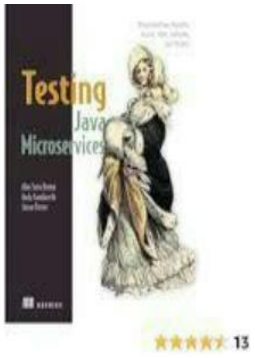
Principle of Testing #6

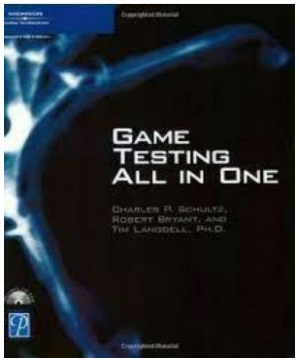
- Testing is context dependent.
- Scenario: "Testing for a pacemaker vs. a food app the same?"
  - Expected: No, risk and consequences differ











Principle of Testing #7

- Verification is not Validation.
- Verification
  - Does the software system meet the requirements specifications?
  - Are we building the software right?
- Validation
  - Does the software system meet the user's real needs?
  - Are we building the **right software**?

- Software matches spec but users hate it – what was missed?"
  - Validation (user's real needs)

Case Study - Ariane 5 Crash

- Ariane 5 self-destructed ~37–39 seconds post-launch due to an unhandled 64→16-bit numeric conversion overflow inherited from Ariane 4 assumptions; diagnostic data was then misread as valid flight data by downstream systems.

- Which principle was violated most clearly?

- What boundary/robustness tests were missing?

How to create tests?

Test Design Techniques

- Opportunistic/ exploratory testing:
  - Add some unit tests, without much planning.

- Specification-based testing (“Black Box”) :
  - Derive test cases from specifications:
    - Boundary value analysis
    - Equivalence classes
    - Combinatorial testing
    - Random testing

Specification Testing

- Texts are based on the specification.

- Advantages:
  - Avoid implementation bias.
  - Robust to changes in the implementation.
  - Tests don’t require familiarity with the code.
  - Tests can be developed before the implementation.

Example

```
"""
Compute the price of a bus ride:
  - Children under 2 ride for free.
  - Children under 18 and senior citizens over 65 pay half the fare
  - All others pay the full fare of $3.
  - On weekdays (Monday to Friday), between 7am and 9am and
    between 4pm and 6pm, a peak surcharge of $1.5 is added
    to the fare.
  - During weekends (Saturday and Sunday), there is a flat rate
    of $2 for all riders, except for children under 2.
  - Short trips under 5 minutes during off-peak times are free,
    except on weekends.
  - If the trip occurs on a public holiday, a special holiday surcharge
    of $2 is added, ignoring other surcharges and the weekend flat rate.
"""
def bus_ticket_price(age: int,
                     ride_datetime: datetime,
                     ride_duration: int,
                     is_public_holiday: bool) -> float:
    ...
```

What about Exhaustive testing ?

- **Idea: Try all values** !!!
  - **Age: int**(2-17) years
  - **datetime : Datetime** (hh:mm + M/D/Y)
  - **rideTime : int** (in minutes, 1-2 hours)
  - **is_public_holiday: bool**(2 values)

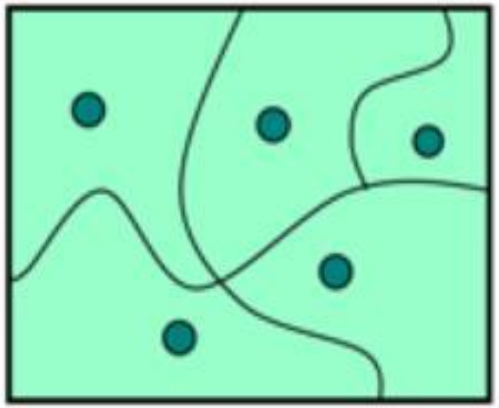
- 116 x 1440 (minutes per day) x 1826 (days in the next 5 years) x 120 (ride time) x 2 = **72 billion test cases**.

What about Exhaustive testing?

- Exhaustive testing is usually impractical – even for trivially small problem
- Key Problem: choosing test suite
  - **Small enough** to finish in a useful amount of time
  - **Large enough** to provide a useful amount of validation
- Alternative: **Heuristics**

Equivalence Partitioning

- Identify sets with same behavior (**equivalence class**)
- Try one input from each set
- Equivalence classes derived from specifications (e.g., cases, input ranges, error conditions, fault models)
- Requires domain-knowledge



Boundary Value Analysis

- **Key Insight:** Errors often occur at the boundaries of a variable value

- For each variable, select:
  - Minimum,
  - Min+1,
  - Medium,
  - Max-1,
  - Maximum,
  - Possibly also invalid values Min-1, Max+1

Boundary Value Analysis

```python
"""
 Compute the price of a bus ride:
   - Children under 2 ride for free.
   - Children under 18 and senior citizens over 65 pay half the fare
   - All others pay the full fare of $3.
   - On weekdays (Monday to Friday), between 7am and 9am and
     between 4pm and 6pm, a peak surcharge of $1.5 is added
     to the fare.
   -  During weekends (Saturday and Sunday), there is a flat rate
     of $2 for all riders, except for children under 2.
   -  Short trips under 5 minutes during off-peak times are free,
     except on weekends.
   - If the trip occurs on a public holiday, a special holiday surcharge
     of $2 is added, ignoring other surcharges and the weekend flat rate.
 """
def bus_ticket_price(age: int,
                     ride_datetime: datetime,
                     ride_duration: int,
                     is_public_holiday: bool) -> float:
    ...
```

Variable	Domains
age	<2, [2,17], [18,65], >65
ride_datetime	weekdays peak and off-peak, weekends peak and off-peak …
ride_duration	<5, >=5
is_public_holiday F, T	

Pairwise Testing

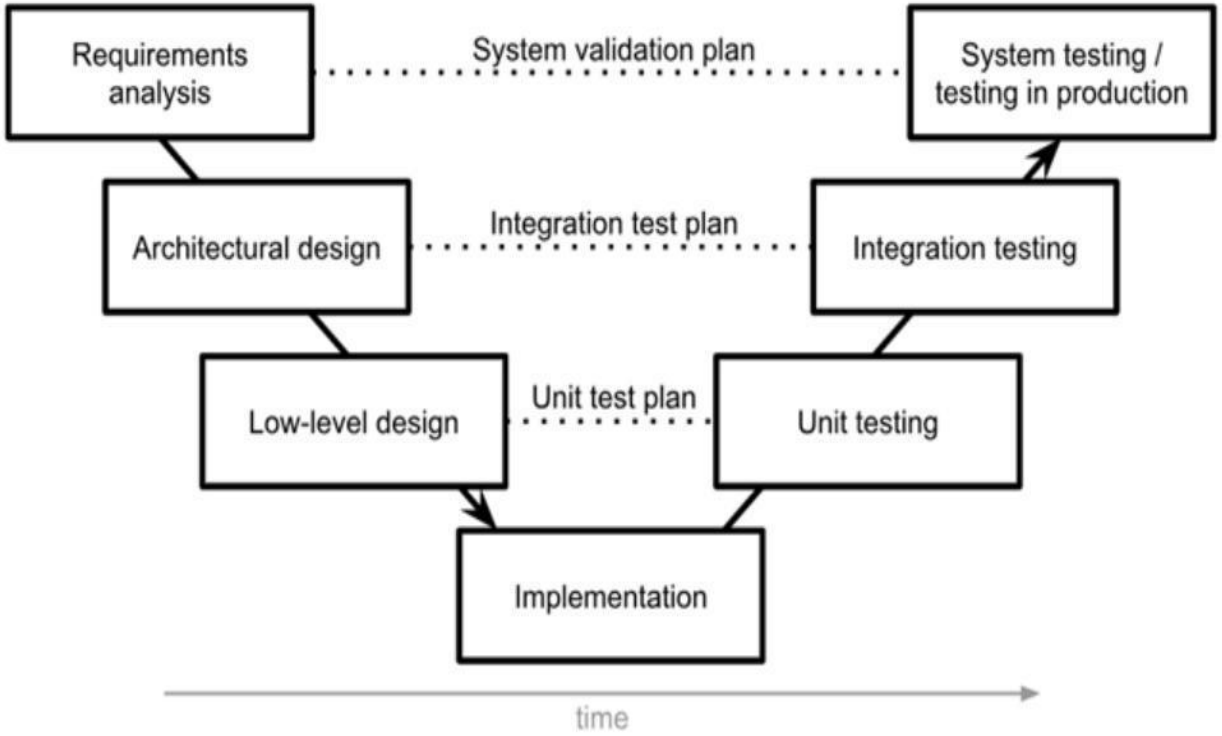
Key Insight: some problems only occur as the result of an interaction between parameters/components

- Examples of interactions:
  - The bug occurs for senior citizens traveling on weekends (pairwise interaction)
  - The bug occurs for senior citizens traveling on weekends during peak hours (3-way interaction)
  - The bug occurs for adults traveling long trips during public holidays that are weekends. (4-way interaction)

- **Claim: Considering pairwise interactions finds about 50% to 90% of defects**

When to create and run tests

The V Model


Requirements analysis
System validation plan
System testing / testing in production
Architectural design
Integration test plan
Integration testing
Low-level design
Unit test plan
Unit testing
Implementation
time

Test Cases

- Test cases are a key part of test documentation
- Tells the tester what to test, how to check it, and what the expected results are
- Once test is executed, determines if software satisfies requirement or function properly
- Act as the starting point for text execution
- Well written test cases enable anyone to step into tester role and determine functioning of software
- Ensure proper test coverage of application under test
- Writing test cases helps in thinking through the details and ensures testing is addressed from multiple angles

Test Cases

Typical Test Case Parameters are

- Test Case ID
- Test Scenario
- Test Case Description
- Test Setup
- Test Steps
- Prerequisite if any
- Test Parameters
- Expected Result
- Actual Result
- Comments

Sample
Test Case Id: TC1
Title: Login Page – Authenticate Successfully on gmail.com
Description: A registered user should be able to successfully login at gmail.com.
Precondition: the user must already be registered with an email address and password.
Assumption: a supported browser is being used.
Test Steps:
1.Navigate to gmail.com
2.In the 'email' field, enter the email address of the registered user.
3.Click the 'Next' button.
4.Enter the password of the registered user
5.Click 'Sign In'
Expected Result: A page displaying the gmail user's inbox should load, showing any new message at the top of the page.

Value proposition of writing test cases

- Test Scenarios identify and isolate areas to be tested
  - Broken down into detailed subsection and creates a framework for comprehensive and successful testing
- Beneficial to future teammates
- Good source of truth on how system and particular feature works
- Provide following value
  - Ensures good test coverage
  - Allows tester to find different ways of validating feature
  - Negative test cases are also documented
  - Reusable for the future

Types of Test Cases

Different types of test cases ensure assurance of quality
Work with a subset of tests that achieve the highest quality products

Types

- **Positive**
  - Verify the software is doing what it's supposed to do .
- **Negative**
  - Verify the software is not doing what it is not supposed to do.
- **Destructive**
  - Scenarios the software can handle before it breaks.

Example

- Allow login when valid credentials are supplied
- Don't allow login when invalid credentials are supplied

Writing test cases

- Keep the title short
- Include a strong description
- Be clear and concise
- Include the Expected result
- Make test case reusable

Test suites

- Test Suite is a container that has a set of tests which helps testers in executing and reporting the test execution status

- Has three states
  - Active
  - In-Progress
  - Completed

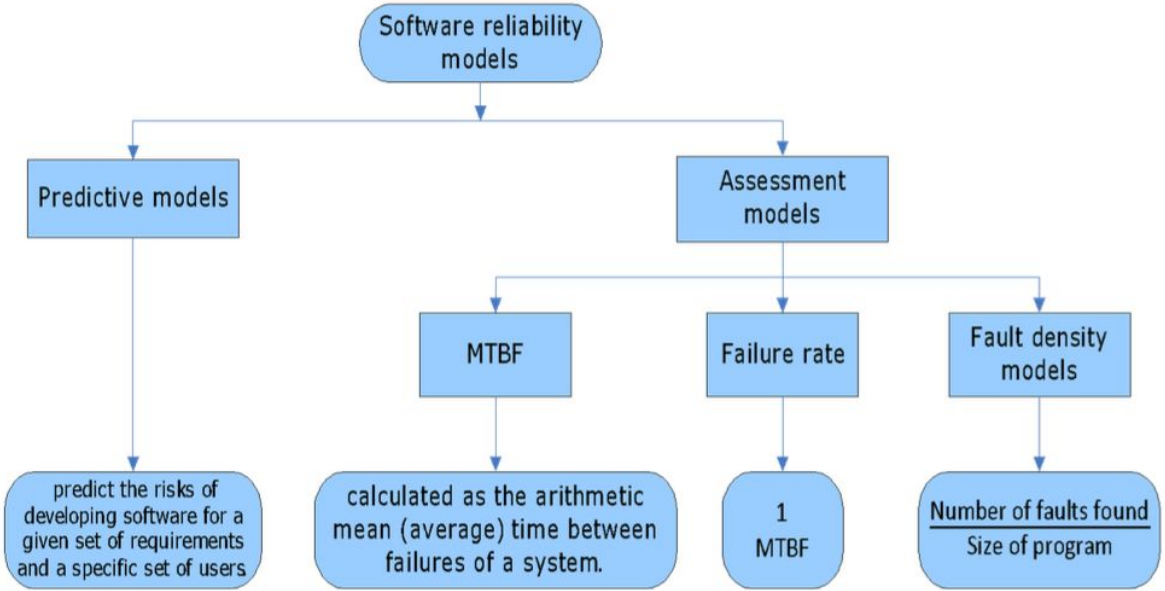
- Test case can be added to multiple test suites and test plans

- Test suites are created based on the cycle or scope

- Can contain any type of tests
  - Functional
  - Non-Functional

Software Reliability

Software Reliability is the probability of failure-free software operation for a specified period of time in a specified environment





Software Engineering

Thank You

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Attribution and Licensing

- Parts of this slideset include content adapted from the CMU 17-313: Foundations of Software Engineering course offered by Carnegie Mellon University.
- This material is used under the terms of the MIT License.
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Unit 3: Lecture 9

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Course Coordinator and Anchor (RR Campus): Prof Vadiraja Acharya
Course Anchor (EC Campus) : Prof M Sheela Devi

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

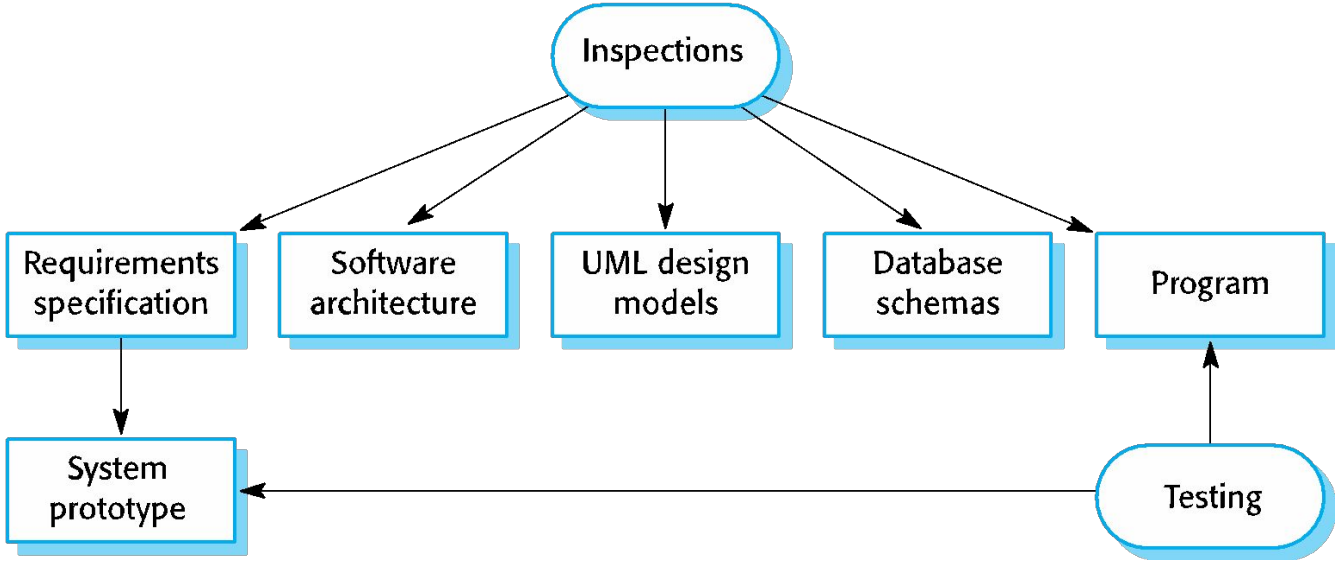
Inspection and Testing

Inspections and Testing

- Software inspections Concerned with analysis of the static system representation to discover problems *(*static verification)
  - May be supplement by tool-based document and code analysis.

- Software testing Concerned with exercising and observing product behaviour (dynamic verification)
  - The system is executed with test data and its operational behaviour is observed.

Inspections and Testing



Software Inspections

- These involve people examining the source representation with the aim of discovering anomalies and defects.
- Inspections not require execution of a system so may be used before implementation.
- They may be applied to any representation of the system (requirements, design, configuration data, test data, etc.).
- They have been shown to be an effective technique for discovering program errors.

Advantages of inspections

- During testing, errors can mask (hide) other errors. Because inspection is a static process, you don't have to be concerned with interactions between errors.

- Incomplete versions of a system can be inspected without additional costs. If a program is incomplete, then you need to develop specialized test harnesses to test the parts that are available.

- As well as searching for program defects, an inspection can also consider broader quality attributes of a program, such as compliance with standards, portability and maintainability.

Stages of testing

- Development testing, where the system is tested during development to discover bugs and defects.
- Release testing, where a separate testing team test a complete version of the system before it is released to users.
- User testing, where users or potential users of a system test the system in their own environment.

Development Testing

Development Testing

Development testing includes all testing activities that are carried out by the team developing the system.

- Unit testing, where individual program units or object classes are tested. Unit testing should focus on testing the functionality functions.
- Component testing, where several individual units are integrated to create composite components. Component testing should focus on testing component interfaces.
- System testing, where some or all of the components in a system are integrated and the system is tested as a whole. System testing should focus on testing component interactions.

Unit Testing

- Unit testing is the process of testing individual components in isolation.
- It is a defect testing process.
- Units may be:
  - Individual functions or methods within an object
  - Object classes with several attributes and methods
  - Composite components with defined interfaces used to access their functionality.

Object Class Testing

- Complete test coverage of a class involves
  - Testing all operations associated with an object
  - Setting and interrogating all object attributes
  - Exercising the object in all possible states.

- Inheritance makes it more difficult to design object class tests as the information to be tested is not localised.

Automated Testing

- Whenever possible, unit testing should be automated so that tests are run and checked without manual intervention.

- In automated unit testing, you make use of a test automation framework (such as JUnit) to write and run your program tests.

- Unit testing frameworks provide generic test classes that you extend to create specific test cases. They can then run all of the tests that you have implemented and report, often through some GUI, on the success of otherwise of the tests.

Component testing

- Software components are often composite components that are made up of several interacting elements.
  - Example: In the weather station system, the reconfiguration component includes elements that deal with each aspect of the reconfiguration.
- You access the functionality of these elements through the defined component interface.
- Testing composite components should therefore focus on showing that the component interface behaves according to its specification.
  - You can assume that unit tests on the individual objects within the component have been completed.

Interface testing

- Objectives are to detect faults due to interface errors or invalid assumptions about interfaces.

- **Interface types**
  - Parameter interfaces Data passed from one method or procedure to another.
  - Shared memory interfaces Block of memory is shared between procedures or functions.
  - Procedural interfaces Sub-system encapsulates a set of procedures to be called by other sub-systems.
  - Message passing interfaces Sub-systems request services from other sub-systems

Interface errors

- **Interface misuse**
  - A calling component calls another component and makes an error in its use of its interface e.g. parameters in the wrong order.
- **Interface misunderstanding**
  - A calling component embeds assumptions about the behaviour of the called component which are incorrect.
- **Timing errors**
  - The called and the calling component operate at different speeds and out-of-date information is accessed.

Interface testing guidelines

- Design tests so that parameters to a called procedure are at the extreme ends of their ranges.
- Always test pointer parameters with null pointers.
- Design tests which cause the component to fail.
- Use stress testing in message passing systems.
- In shared memory systems, vary the order in which components are activated.

System testing

- System testing during development involves integrating components to create a version of the system and then testing the integrated system.
- The focus in system testing is testing the interactions between components.
- System testing checks that components are compatible, interact correctly and transfer the right data at the right time across their interfaces.
- System testing tests the emergent behavior of a system.

System and component testing

- During system testing, reusable components that have been separately developed and off-the-shelf systems may be integrated with newly developed components. The complete system is then tested.

- Components developed by different team members or sub-teams may be integrated at this stage. System testing is a collective rather than an individual process.
  - In some companies, system testing may involve a separate testing team with no involvement from designers and programmers.

Regression testing

- Regression testing is testing the system to check that changes have not ‘broken’ previously working code.
- In a manual testing process, regression testing is expensive but, with automated testing, it is simple and straightforward.
- All tests are rerun every time a change is made to the program.
- Tests must run ‘successfully’ before the change is committed.



Software Engineering

Thank You

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Attribution and Licensing

- Parts of this slideset include content adapted from the CMU 17-313: Foundations of Software Engineering course offered by Carnegie Mellon University.
- This material is used under the terms of the MIT License.
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Unit 3: Lecture 10

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Course Coordinator and Anchor (RR Campus): Prof Vadiraja Acharya
Course Anchor (EC Campus) : Prof M Sheela Devi

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Release Testing

Release Testing

- Release testing is the process of testing a particular release of a system that is intended for use outside of the development team.

- The primary goal of the release testing process is to convince the supplier of the system that it is good enough for use.
  - Release testing, therefore, has to show that the system delivers its specified functionality, performance and dependability, and that it does not fail during normal use.

- Release testing is usually a black-box testing process where tests are only derived from the system specification.

Release and System Testing

- Release testing is a form of system testing.

- Important differences:
  - A separate team that has not been involved in the system development, should be responsible for release testing.
  - System testing by the development team should focus on discovering bugs in the system (defect testing).
  - The objective of release testing is to check that the system meets its requirements and is good enough for external use (validation testing).

User Testing

User testing

- User or customer testing is a stage in the testing process in which users or customers provide input and advice on system testing.

- User testing is essential, even when comprehensive system and release testing have been carried out.
  - The reason for this is that influences from the user's working environment have a major effect on the reliability, performance, usability and robustness of a system.
  - These cannot be replicated in a testing environment.

Types of user testing

- **Alpha testing**
  - Users of the software work with the development team to test the software at the developer's site.
- **Beta testing**
  - A release of the software is made available to users to allow them to experiment and to raise problems that they discover with the system developers.
- **Acceptance testing**
  - Customers test a system to decide whether or not it is ready to be accepted from the system developers and deployed in the customer environment. Primarily for custom systems.

Agile methods and acceptance testing

- In agile methods, the user/customer is part of the development team and is responsible for making decisions on the acceptability of the system.
- Tests are defined by the user/customer and are integrated with other tests in that they are run automatically when changes are made.
- There is no separate acceptance testing process.
- Main problem here is whether or not the embedded user is ‘typical’ and can represent the interests of all system stakeholders.

Performance Testing and Debugging

Performance Testing

- Goal: Identify *performance bugs*. What are these?
  - Unexpected bad performance on some subset of inputs
  - Performance degradation over time
  - Difference in performance across versions or platforms

- Not as easy as functional testing. What's the baseline?
  - Fast = good, slow = bad // but what's the threshold?
  - How to get reliable measurements?
  - How to debug where the issue lies?

Performance regression testing helps identify trends

- Measure execution time of critical components
- Log execution times and compare over time



Performance bugs are “bad” bugs

Discovering, Reporting, and Fixing Performance Bugs

Adrian Nistor[1], Tian Jiang[2], and Lin Tan[2]
[1]University of Illinois at Urbana-Champaign, [2]University of Waterloo
nistor1@illinois.edu, {t2jiang, lintan}@uwaterloo.ca

- Fixing performance bugs is usually more difficult than fixing non-performance bugs
- Performance bugs usually don’t generate incorrect results or Crashes
- Difficult to diagnose:
  - system load, hardware configuration, network conditions, user-specific workflows, interactions with other systems
- Big impact on user experience

Profiling and tracing

- Profiling is a process to analyze and measure the performance of a program or specific parts of its code (e.g., functions).
- Tracing is about understanding the flow of execution and the behavior of a program.
- Record sequential events (function calls) that occur during the execution of a program
- Both can be used to identify bottlenecks in execution time and memory

Soak testing

- A system may behave exactly as expected under artificially limited execution conditions, but fail in production after extended use.
  - E.g., Memory leaks may take longer to lead to failure

- Soak testing a system involves applying a significant load over a significant period of time and observing system resilience.

- Time-consuming to run but useful to apply at big release milestones or when making infrastructure changes.

Chaos Engineering

Reliability testing

- What happens when some components of a large complex system fail? Can the system recover and keep working?
- How can you test the reliability of something as complex as Netflix or Google maps or Instagram?
- One idea: simulate a large-scale deployment and induce random failures in various components

  Test in Production with Chaos Engineering

What is chaos engineering?

"Chaos Engineering is the discipline of experimenting on a system in order to build confidence in the system's capability to withstand turbulent conditions in production."

Chaos Engineering: Testing in Production

- Purposefully take down components in a live deployment.
- Observe system response. Do failovers work correctly?
- Tests the failure-handling and fallback capabilities of large systems.
- Useful in preparing for natural disasters or cyberattacks.

Example: Google

- Terminate network in Sao Paulo for testing:
  - Hidden dependency takes down links in Mexico which would have remained undiscovered without testing

- Turn off data center to find that machines won't come back:
  - Ran out of DHCP leases (for IP address allocation) when a large number of machines come back online unexpectedly.

Why would you break things on purpose?





Failures in Microservice Architectures

- Network may be partitioned
- Server instance may be down
- Communication between services may be delayed
- Server could be overloaded and responses delayed
- Server could run out of memory or CPU

Example: Netflix

- Significant deployment on AWS cloud. Hundreds of updates to microservices and infrastructure through the day.
- Chaos Monkey randomly takes down AWS instances or network connections or randomly changes config files.
- How to tell ”are we still good?”
- Key metric: Stream Starts per Second (SPS)
- Measures availability



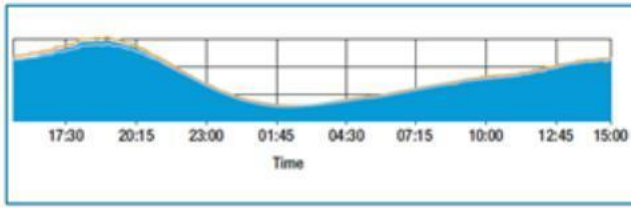


FIGURE 2. A graph of SPS ([stream] starts per second) over a 24-hour period. This metric varies slowly and predictably throughout a day. The orange line shows the trend for the prior week. The y-axis isn't labeled because the data is proprietary.



Software Engineering

Thank You

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Attribution and Licensing

- Parts of this slideset include content adapted from the CMU 17-313: Foundations of Software Engineering course offered by Carnegie Mellon University.
- This material is used under the terms of the MIT License.
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

Unit 3: Lecture 11

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Course Coordinator and Anchor (RR Campus): Prof Vadiraja Acharya
Course Anchor (EC Campus) : Prof M Sheela Devi

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Static and Dynamic Testing

Static and Dynamic Testing

- **Static** testing is **“Verification”.**
- **Dynamic** testing is **“Validation”**.

Static testing

- Checks for defects in the software without executing the code.
- Performed in early stages of development to avoid errors
- Easier to find the source and solution when discovered earlier

- Types of Static testing
  - Document reviews
  - Walkthroughs
  - Inspection
  - Feasibility analysis or any other form of analysis to determine if the software is what it should be or not
  - Code review

Static and Dynamic Testing

Static analysis

- Evaluation of code quality
- Data flow
- Control flow
- Cyclomatic complexity

- Note: Cyclomatic complexity of a code section is the quantitative measure of the number of linearly independent paths in it. It is a software metric used to indicate the complexity of a program. It is computed using the Control Flow Graph of the program.

- ***Cyclomatic complexity, M = E – N + 2P,*** *where,*
  E = the number of edges in the control flow graph
  N = the number of nodes in the control flow graph
  P = the number of connected components

Cyclomatic Complexity

Control Flow Graph

- A **Control Flow Graph (CFG)** is the graphical representation of control flow or computation during the execution of programs or applications.

Characteristics of Control Flow Graph:

- Control flow graph is process oriented.
- Control flow graph shows all the paths that can be traversed during a program execution.
- Control flow graph is a directed graph.
- Edges in CFG portray control flow paths and the nodes in CFG portray basic blocks.

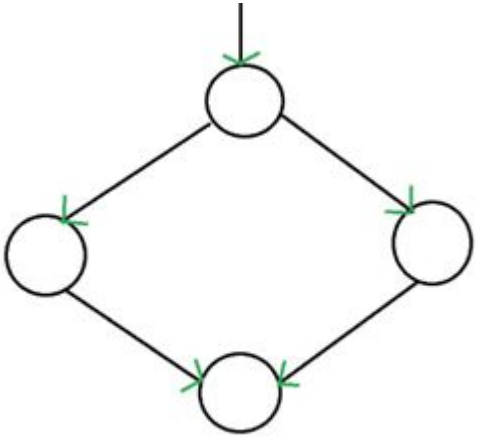
There exist 2 designated blocks in Control Flow Graph:

- **Entry Block:**
  - Entry block allows the control to enter into the control flow graph.
- **Exit Block:**
  - Control flow leaves through the exit block.

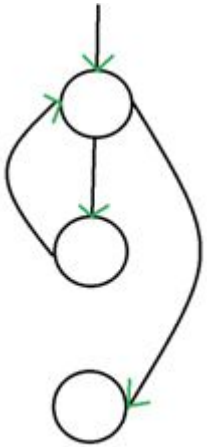
Hence, the control flow graph is comprised of all the building blocks involved in a flow diagram such as the start node, end node and flows between the nodes.

Cyclomatic Complexity

- **General Control Flow Graphs:**
  - Control Flow Graph is represented differently for all statements and loops. Following images describe it:

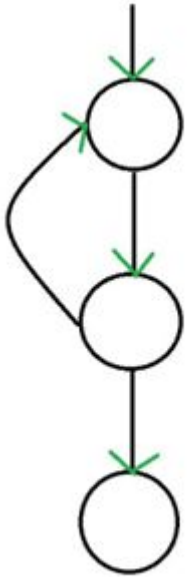


If-then-else

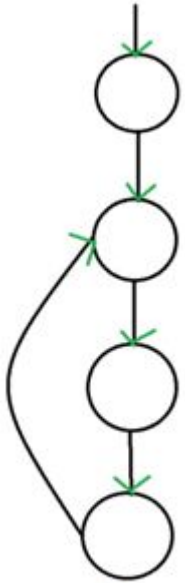


while

Cyclomatic Complexity



do-while

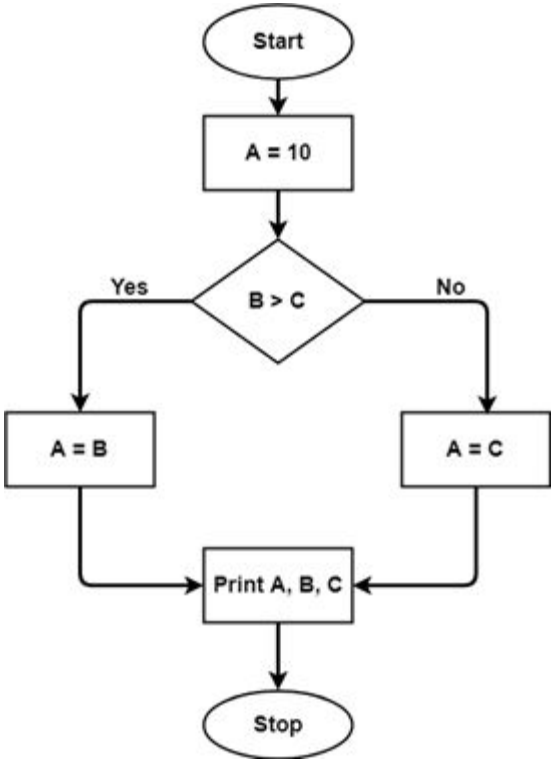


for

Cyclomatic Complexity

Cyclomatic Complexity	Meaning
1 – 10	• Structured and Well Written Code • High Testability • Less Cost and Effort
10 – 20	• Complex Code • Medium Testability • Medium Cost and Effort
20 – 40	• Very Complex Code • Low Testability • High Cost and Effort
> 40	• Highly Complex Code • Not at all Testable • Very High Cost and Effort

Cyclomatic Complexity



For the control graph, shown in the figure:

The graph shows seven shapes(nodes), seven lines(edges), hence cyclomatic complexity is

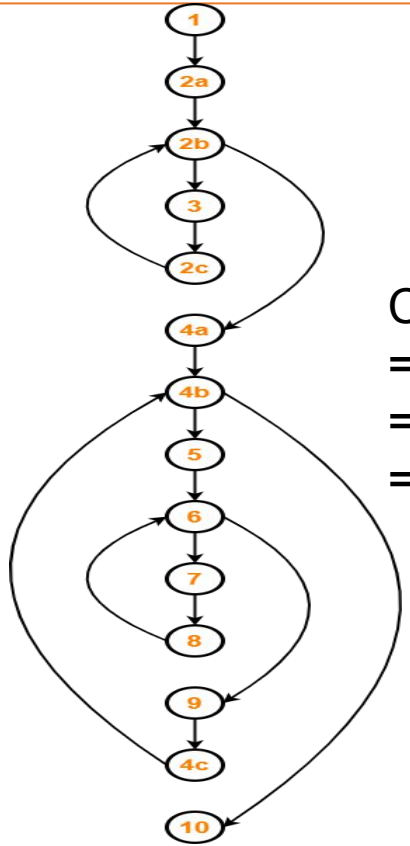
7-7+2 = 2.

Cyclomatic Complexity

```
{
     int i, j, k;
     for (i=0 ; i<=N ; i++)
    p[i] = 1;
    for (i=2 ; i<=N ; i++)
    {
    k = p[i]; j=1;
    while (a[p[j-1]] > a[k] {
        p[j] = p[j-1];
        j--;
    }
    p[j]=k;
 }
```

Cyclomatic Complexity

```
{
    int i, j, k;
    for (i=0 ; i<=N ; i++)
    p[i] = 1;
    for (i=2 ; i<=N ; i++)
    {
    k = p[i]; j=1;
    while (a[p[j-1]] > a[k] {
        p[j] = p[j-1];
        j--;
    }
    p[j]=k;
    }
}
```



Control Flow Graph

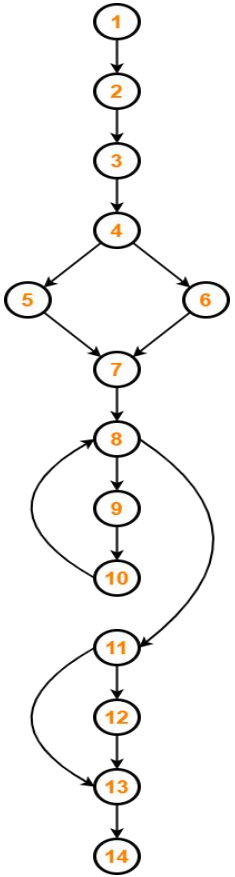
Cyclomatic Complexity
= E – N + 2
= 16 – 14 + 2
= 4

Cyclomatic Complexity - Calculate

```
begin int x, y, power;
float z;
input(x, y);
if(y<0)
     power = -y;
else
     power = y;
z=1;
while(power!=0)
{    z=z*x;
     power=power-1;
}
if(y<0)
    z=1/z;
output(z);
end
```

Cyclomatic Complexity - Calculate

```
begin int x, y, power;
float z;
input(x, y);
if(y<0)
     power = -y;
else
     power = y;
z=1;
while(power!=0)
{    z=z*x;
     power=power-1;
}
if(y<0)
    z=1/z;
output(z);
end
```



Cyclomatic Complexity
= E – N + 2
= 16 – 14 + 2
= 4

Cyclomatic Complexity

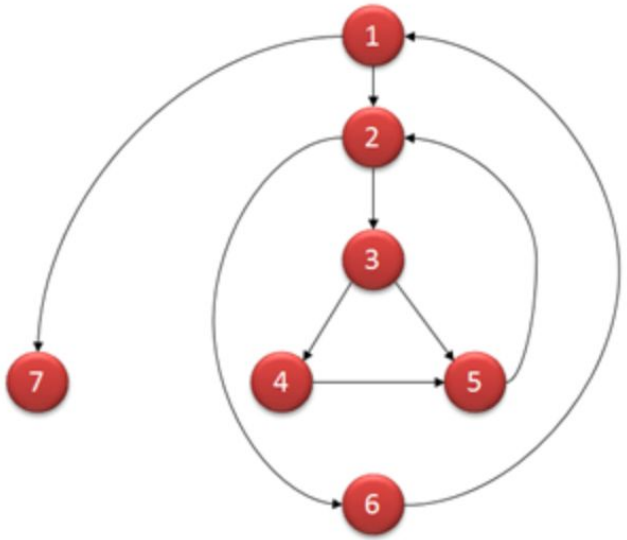
For the given code, draw the control flow graph and compute the cyclomatic complexity of the code.

```
i = 0, n=4;
while (i < n-1) do
      j = i+1
     while (j < n) do
          if A[i] < A[j] then
               Swap (A[i],A[j])
     end do
     i = i+1
end do
```

Cyclomatic Complexity

For the given code, draw the control flow graph and compute the cyclomatic complexity of the code.

```
i = 0, n=4;
while (i < n-1) do
     j = i+1
    while (j < n) do
        if A[i] < A[j] then
            Swap (A[i],A[j])
    end do
    i = i+1
end do
```



Cyclomatic Complexity = E– N + 2

Here,

E = 9

N = 7

Cyclomatic Complexity = 9 - 7 + 2

= 4

Cyclomatic Complexity - Disadvantages

- It is the measure of the program's control complexity and not the data complexity.
- In this, nested conditional structures are harder to understand than non-nested structures.
- In the case of simple comparisons and decision structures, it may give a misleading figure.

Dynamic Testing

- Dynamic testing is when you are working with the actual system (not some artifact or model that represents the system), providing an input, receiving output, and comparing the output to the expected behavior.

- It is hands-on working with the system with the intent of finding errors.

- Advantage
  - Find difficult and complex defects
- Disadvantages
  - Time and Budget

Dynamic Testing – Code based approaches

Control Flow Based Criteria

- Control flow path: graphical representation of all the paths that might be traversed during execution
- Path testing: Statement, Branch and Condition testing
- Branch Coverage
  - Executes every path at least once
  - No branch leads to abnormal behavior

Data Flow Based Criteria

- Selecting paths through program's control flow in order to explore sequences of events related to status of variables or data objects
- Statement Coverage
- Executes all statements at least once
- No side effects

Dynamic Testing – Fault based approaches

Error Guessing

- Error guessing is **a type of testing method in which prior experience in testing is used to uncover the defects in software**.
- It is an experience based test technique in which the tester uses his/her past experience or intuition to gauge the problematic areas of a software application.
  - Most plausible faults
  - Historical information and experience

Dynamic Testing – Fault based approaches

Fault Seeding

- Fault seeding is **a technique for evaluating the effectiveness of a testing process**.
- One or more faults are deliberately introduced into a code base, without informing the testers.
- The discovery of seeded faults during testing can be used to calibrate the effectiveness of the test process and testers
  - Fault injected into a copy of the code
  - Injected and other faults detected are analyzed

Dynamic Testing – Fault based approaches

Mutation Testing

- It is a type of software testing in which certain statements of the source code are changed/mutated to check if the test cases are able to find errors in source code.
- The goal of Mutation Testing is ensuring the quality of test cases in terms of robustness that it should fail the mutated source code.

  - Tests seldom-executed code with well-defined mutation operators
  - Mutants: single statement is changed in several copies
  - Test between original and mutants
  - All mutants should fail
  - Reveal more complex and real faults

Dynamic Testing – based on approach used

- **Equivalence Partitioning and Boundary Value analysis**
  - Uncover errors and reduce test cases
  - Divide input data into partitions from which test cases are derived per partition
  - Boundary value analysis handles edge case.

- **Specification Based**
  - ***Specification Based Test Design Technique*** uses the specification of the program as the point of reference for test data selection and adequacy.
  - A specification can be anything like a written document, collection of use cases, a set of models or a prototype.
    - Test functionality according to stated requirements
    - Test the behavior of the system based on specific input.

Dynamic Testing – based on approach used

- **Intuition Based**
  - Rely on the testers experience to design test cases for the product.
- **Usage Based**
  - Finding defects that are revealed by user as they interact with the product.
- **Application Domain**
  - Use specific knowledge of product domain to develop test cases.



Software Engineering

Thank You

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Attribution and Licensing

- Parts of this slideset include content adapted from the CMU 17-313: Foundations of Software Engineering course offered by Carnegie Mellon University.
- This material is used under the terms of the MIT License.
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.



Software Engineering

UNIT 3- Lecture 12-14

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Course Coordinator and Anchor (RR Campus): Prof Vadiraja Acharya
Course Anchor (EC Campus) : Prof M Sheela Devi

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Software Configuration Management

Scenario

- It's 2 AM. Your team is deploying the College Event Manager app for the university's biggest event tomorrow. Three developers have been working on different features:
  - Alice fixed the registration bug
  - Bob added payment integration
  - Charlie improved the UI
- You merge all changes... and the entire system crashes. Registration is broken. Payments fail. The UI is jumbled. Nobody knows whose code caused what problem. Nobody can go back to the 'working' version because... which version was working?

Imagine this happening before the Project Demo!!

Scenario

- "Who has lost code because of accidental overwrites?"
- "Who has had conflicts when merging team member code?"
- "Who has had 'it worked on my machine' problems?"
- "Who has multiple files named final_final_v3_REAL.py?"

Software Configuration Management

- Software Configuration Management (SCM) is a software engineering discipline consisting of standard processes and techniques often used by organizations to manage the changes introduced into its software products.
- SCM helps in identifying individual elements and configurations, tracking changes, and version selection, control, and baselining.
- SCM is a Process to systematically organize, manage and control changes in documents, code and other entities that constitute a software product.
- SCM provides a mechanism for managing all changes in an efficient, cost-effective and timely manner.
- SCM also puts into place a framework for maintenance and support of a product.

Goals of SCM

- Increase productivity by increased and planned coordination among the programmers and eliminate confusion and mistakes

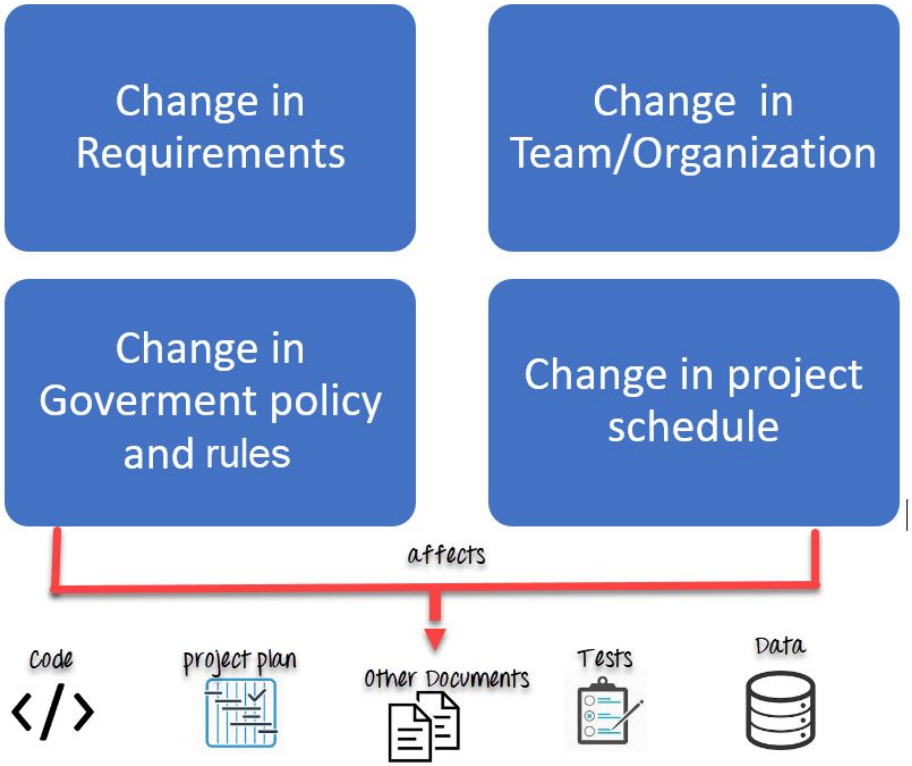
  - Identifying elements and configurations, tracking changes, version selection, control and baselining
  - Avoid configuration related problems
  - Effective management of simultaneous updating of source files
  - Build management
  - Defect tracking

Why do we need SCM ?

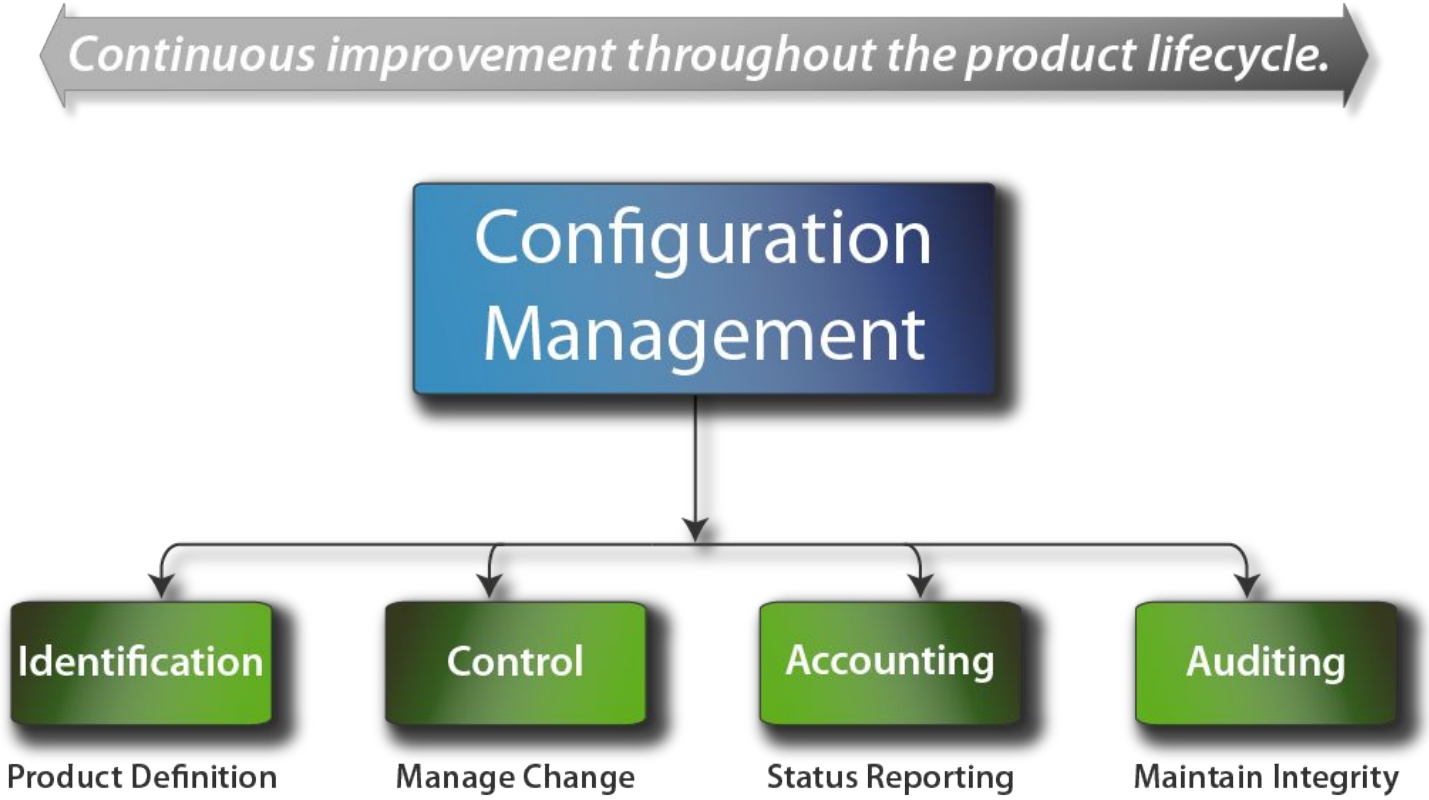
- Software engineering in large products/projects with multiple people across locations need to be reliably managed for

  - Multiple people working concurrently on the same piece of software
  - Working on more than one version
  - Working on released systems
  - Changes in configuration due to changes in user requirements, budget, etc
  - Custom configured systems
  - Software must run on different machines
  - Coordination among stakeholders
  - Controlling costs in making changes

SCM


Change in Requirements
Change in Team/Organization
Change in Goverment policy and rules
Change in project schedule
affects
Code
project plan
Other Documents
Tests
Data

Basic elements of SCM



Basic elements of SCM

SCM Element	Method
Software Configuration Item identification	Define baseline components
Software Configuration Control	Mechanism for initiating, preparing, evaluating, approving or disapproving all change proposals.
Software Configuration Auditing	Mechanism for determining the degree that the current state of a software system reflects its baselines (requirements and planning documents)
Software Configuration status accounting	Mechanism for maintaining a record of how a system has evolved, and the state of a system relative to published documents and written agreements.

Benefits of SCM

- Permits orderly development of software configuration items
- Ensures the orderly release and implementation of new or revised software products
- Ensures only approved changes to both new and existing software products are implemented and deployed
- Ensures that software changes are implemented in accordance with approved specifications
- Ensure that the documentation accurately reflects updates
- Evaluates and communicates the impact of changes
- Prevents unauthorized changes from being made

Case Study: Google

- Google manages a monorepo with 2 billion lines of code.
  - How? SCM at scale
  - 40,000+ commits per day
  - Automated testing on every commit
  - Instant rollback capability
  - Complete audit trail

Configuration Management Roles

- **Configuration Manager**
  - Identify configuration items.
  - Define procedures for creating and promotions and releases.
- **Developer**
  - Create versions triggered by change requests or normal developmental activities.
  - Checks in changes and resolves conflicts.
- **Auditor**
  - Validate processes for selection and evaluation of promotions for release.
  - Ensures consistency and completeness.
- **Change Control Board Member**
  - Responsible for approving or rejecting change requests

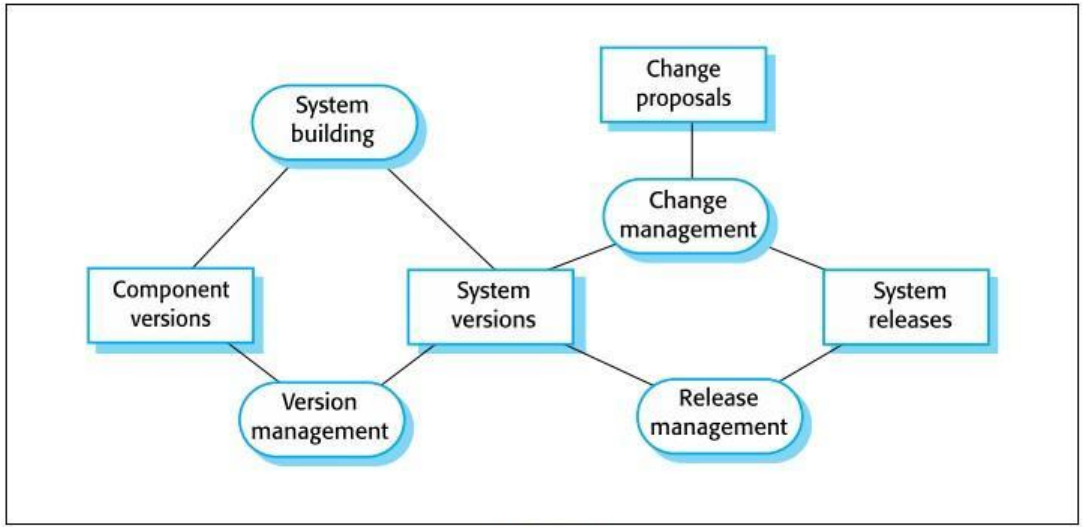
Software configuration Management Planning

- Planning is done by a Configuration manager and starts during the early phases of the project

- Results in a Software Configuration Management Plan
  - May follow a public or internal standard
  - Defines the Configuration Items and a naming scheme
  - Define who takes responsibility for Configuration Management procedures and creation of baselines
  - Defines policies for change control and version management
  - Describes tools to assist in Configuration Management and any limitations
  - Defines the configuration management database

Software Configuration Management - Activities

- Configuration item Identification
- Configuration Management Directories
- Baselining
- Branch Management
- Version Management
- Build Management
- Install
- Promotion Management
- Change Management
- Release Management
- Defect Management



Configuration Items

- **Configuration item**: Independent or aggregation of hardware, software or both, that is designated for configuration management and treated as a single entity

- **List everything in your project that should be under version control.**

- Software configuration items could be
  - All types of code files and Drivers for tests
  - Requirement, analysis, design, test and other non temporary documents
  - User or developer manuals
  - System configurations

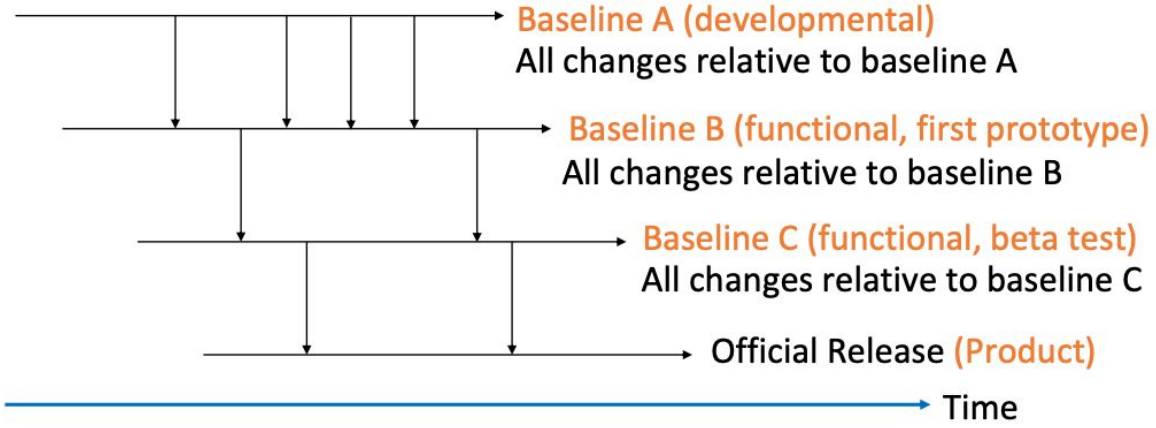
Challenges associated with configuration items

- What items need Configuration control?
  - Some items must be maintained for the lifetime of the software. Similar to object modelling

- When to place entities under configuration control
  - Start too early, too much bureaucracy
  - Start too late, introduces chaos

Baselines

- **Baseline:** Specification or product that has been formally reviewed and agreed to by responsible management and serves as a basis and can only be changed on formal review
- As systems get developed, baselines get developed after a review
  - **Examples:**
    - **Baseline A:** APIs have been defined; Bodies of methods are empty
    - **Baseline B:** All data access methods are implemented and tested



SCM Activities

- **Codeline**: This term commonly describes **a group of files that evolve together over the life of the software application or application component**.
- A codeline might include source code, build scripts, images, compiled code, and other files that are required to create the application.

- **Branch**: copy or clone of all or a portion of the source code

- Reasons for Branching
  - Supporting concurrent development
  - Capturing of solution configurations
  - Support multiple versions of a solution
  - Enable experimentation in isolation without impacting the stable version
  - Branching ensures that overall product is stable
  - Merging is bringing back and integrating changes over multiple branches
  - Frequent merging helps decrease the likelihood and complexity of a merge conflict.

SCM Activities – Branch Management

- Branching in version control and software configuration management, is the duplication of an object under version control (such as a source code file or a directory tree).
- Each object can thereafter be modified separately and in parallel so that the objects become different. In this context the objects are called **branches**.
- The users of the version control system can branch any branch.

- Many branching strategies which may be applied in combination
  - Single branch
  - Branch by customer or organization
  - Branch by developer or workspace
  - Branch by module or component
  - Branch Management entails having a well defined branching policy, owner for the code lines and usage of branches for release

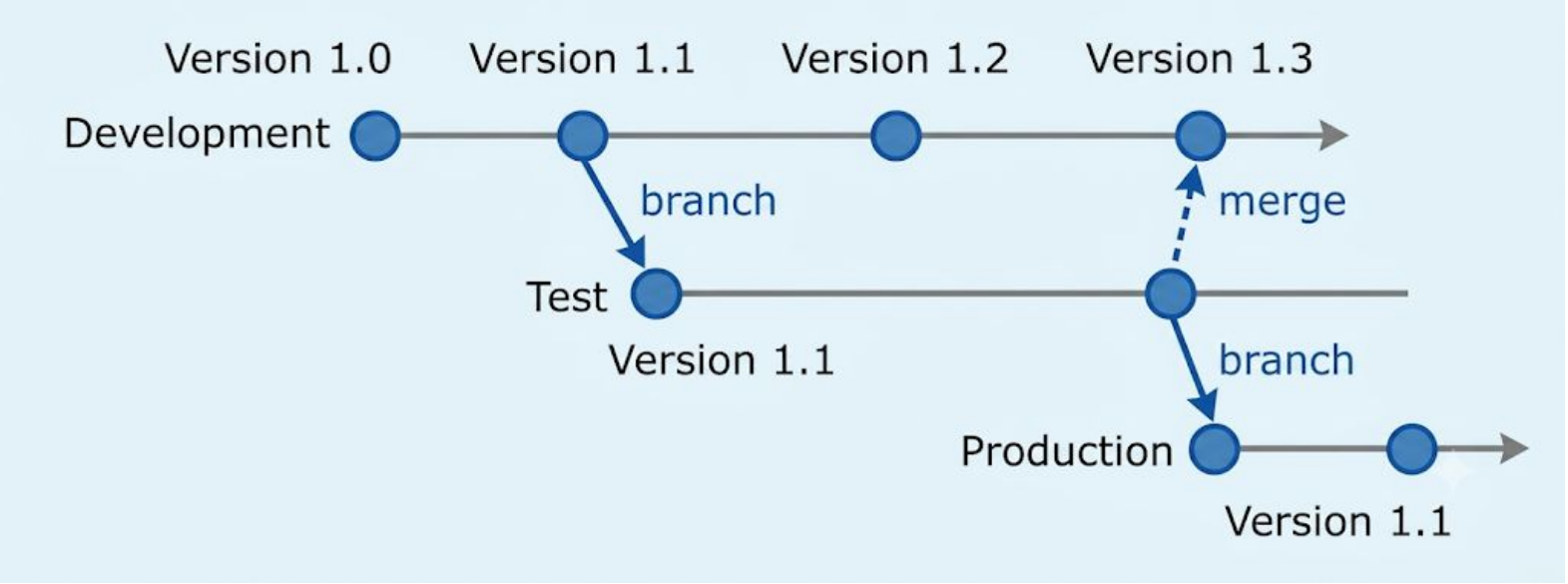
Branching Strategies

- GitFlow: Large product (50 developers), strict release cycles, multiple versions in production
  - main branch: Production-ready code
  - develop branch: Integration branch
  - feature/ branches: New features
  - hotfix/ branches: Production fixes
  - release/ branches: Release preparation

Branching Strategies

- Trunk-Based Development: Small team (~4 people), rapid iteration, weekly releases
  - Single main branch
  - Short-lived feature branches
  - Frequent integration
  - Feature flags for incomplete work

GitFlow


Version 1.0
Version 1.1
Version 1.2
Version 1.3
Development
branch
merge
Test
Version 1.1
branch
Production
Version 1.1

SCM Activities – Version Management

- **Version Management**: keeping track of different versions of software components and systems
  - Usage of tools like git for version management
  - Changes to a version are identified by a number (7.5.4)
    - 7 – Major number
    - 5 – Minor number
    - 4 – Patch number
      - Example:
        - v1.0.0: Initial release
        - v1.0.1: Bug fix for registration
        - v1.1.0: Add payment feature
        - v2.0.0: Complete UI redesign

SCM Activities – Version Management

- Key Features of Version Control System
  - Changes are attributable or traceable
  - Change history is recorded and can be reverted
  - Better conflict resolution
  - Easier code maintenance and quality monitoring
  - Less software regression
  - Better organization and communication

SCM Activities - Build Management

- **Build Management**: creating the application program for a software release by compiling and linking source code and libraries to build artifacts such as binaries or executables
- Done using tools like Make, Apache Ant, Maven, etc
- Compilation and linking of files in the correct order
- No need to recompile if no change in source code results in shorter build time

SCM Activities - Build Process

- Build Process
  - Fetching code from the source control repository
  - Compiling the code and checking the dependencies
  - Linking the libraries, code, etc
  - Running tests and building artifacts.
  - Archive logs and send notification emails
  - May result in version number change.

SCM Activities – Install Management

- Software installation is the first interaction with the customer .
- If install fails, may result in negative perception

- **Software installation**: placing multiple files containing executable code, downloading or copying from a repository, images, libraries, configuration files from the internet

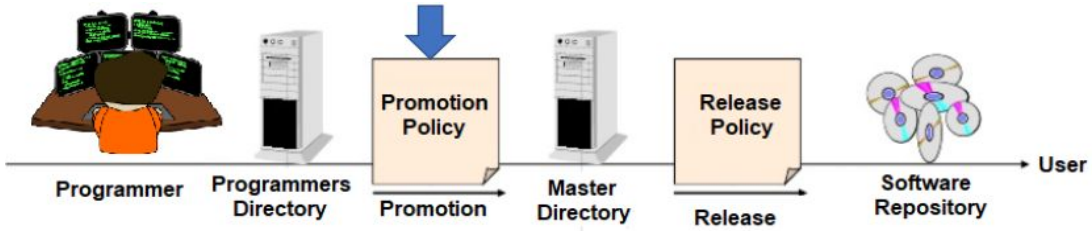
  - Interaction with OS functions for validating the resources needed, permissions, versions, identifiers to ensure enforcement of licenses
  - May involve customizations for localizations
  - Sometimes may be automated using zip, shell scripts, InstallAware and Jenkins

SCM Activities- Promotion Management

- Changes made by a programmer are only available in their environment and need to be promoted to a central master directory .
- Promotion is done based on certain promotion policies .

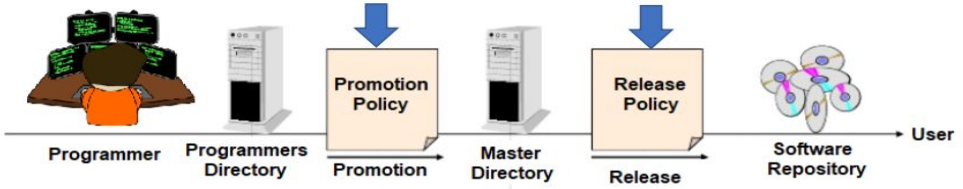
- Promotion could be based on baselining criteria which was planned for and would involve some amount of verification
- It would further be authorized and moved to the master directory

- Food for thought: Could compiled and executable code with a few warnings (not errors) be promoted to the master directory?



SCM Activities – Controlling changes

- Changes are controlled at two points
  - Promotion Policy
  - Release Policy



- Change Policies: the promotion and release policy is dictated via environment
  - **Informal**: good for research type environments and promotions
  - **Formal**: good for externally developed configuration item and for releases

- Change policies guarantee that each version, revision or release conforms to commonly accepted criteria and a consistent process is applied to how things are done
  - Enforced through engineering processes and tools
  - Audited to ensure conformation

SCM Activities – Release Management

- Movement of code to the customer and the software repository is done via release policies
- **Release Policy**: Gating quality criteria that is planned for and includes verification with metrics
- On meeting metrics, it would be authorized for a release
- **Release Management**: managing, planning, scheduling and controlling a software build through different stages and environments, testing and deploying software releases

- Version vs Release vs Revision
  - **Release**: formal distribution of an approved version
  - **Version**: Initial **release** or **re-release** of a configuration item associated with a complete compilation or recompilation of the item
  - Different versions have different functionality
  - **Revision**: change to a version that corrects only errors in the design/code, but does not affect the documented functionality.

SCM Activities – Bug or defect management

- **Bug**: Consequence of a coding fault
- **Defect**: variation or deviation from expected business requirements
  - Driven through defect management tools like Bugzilla

- Defect Management Process
  - Discover: Bug found in testing or production
  - Report: Log in tracking system (Bugzilla, Jira)
  - Validate: Reproduce and confirm
  - Analyze: Categorize severity (Critical, High, Medium, Low)
  - Assign: Developer takes ownership
  - Fix: Code changes made
  - Verify: Tester confirms fix
  - Close: Update version, release notes
  - Release: Deploy fix to customers

SCM Tools – Types of S/W Config tools

- Source Code Administration
  - Used for source code control.
- Software Build
  - Used for building source code.
- Software Installation
  - Used for packaging and installing the products.
- Software Bug Tracking
  - Used for tracking bugs and changes.

SCM Tools – Source Code Administration Tools

- **RCS** (Revision Control System)
  - Very old but in use.
  - Used for version control system.
- **CVS** (Concurrent Version system)
  - Based on RCS, allows concurrent working without locking.
  - Has a web frontend.
- **ClearCase**
  - Multiple servers, process modelling , policy check mechanism.
- **GitHub**
  - Development platform for version control and project management.

Software Build

- **Software build** :
  - Process of converting source code into standalone software artifact(s).
  - Has a compilation process where build is converted to executable.
    - **Make**
      - Automatically builds executable programs using Makefiles.
      - Makefiles derive target program.
    - **Cruise Control**
      - Open source tool for continuous software builds.
    - **FinalBuilder**
      - Automate build release management tool.
      - Easily define and maintain reliable build process.
    - **Maven**
      - Software Project Management and comprehension tool.
      - Based on project object model.

Software installation tools

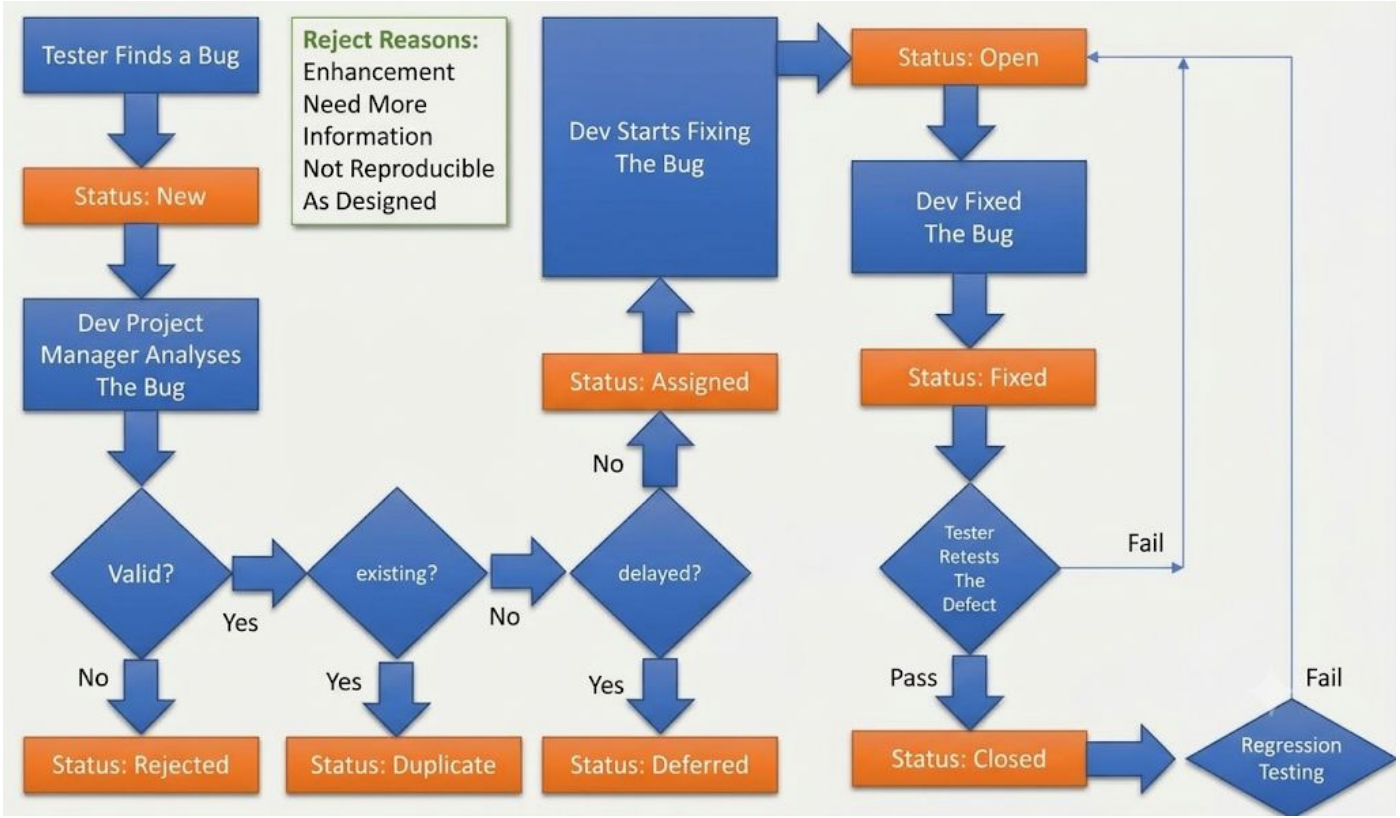
- Cross platform tools that produce installers for multiple Operating systems
  - Installer: installs all the necessary files on the system
  - Could be customized to meet specific needs
  - **Bootstrapper**: small installer that does the pre-requisites and updates the big bundle
  - **DeployMaster** (Windows)
    - Distribute windows software or files via internet/CD/DVD
    - Works with different versions of windows.'
  - **InstallShield**
    - Simplifies creation of windows installers, MSI packages and InstallScript installers for Windows.
    - De- facto for MSI Installation.
  - **InstallAware** (Windows)
    - Windows installer platform for MS windows OS.
    - Supports Internet deployment.
  - **WiseInstaller**
    - Configure and install Microsoft windows applications.

Software Bug or Defect Tracking Tools

- Software application that keeps track of reported software bugs in a software project
- Clear and centralized overview of development requests (bugs and improvements)
- Bug tracking systems support the lifecycle of a bug from logging to resolution

  - Examples: Bugzilla, FogBugz, Trac Edgewall, Backlog, etc

Software Bug or Defect Lifecycle



SCM in Scrum-Agile Approach

- SCM is the responsibility of the “whole team” and is automated as much as possible.
- The definitive versions of components are held in a shared project repository.
- Developers copy versions from the repository into their workspace, make changes to the code and use system-building tools to create a new system on their computer.
- Once the changes are tested, the modified components are pushed to the project repository.
- Versions of the modified code are available to all team members.

Typical Agile SCM Flow

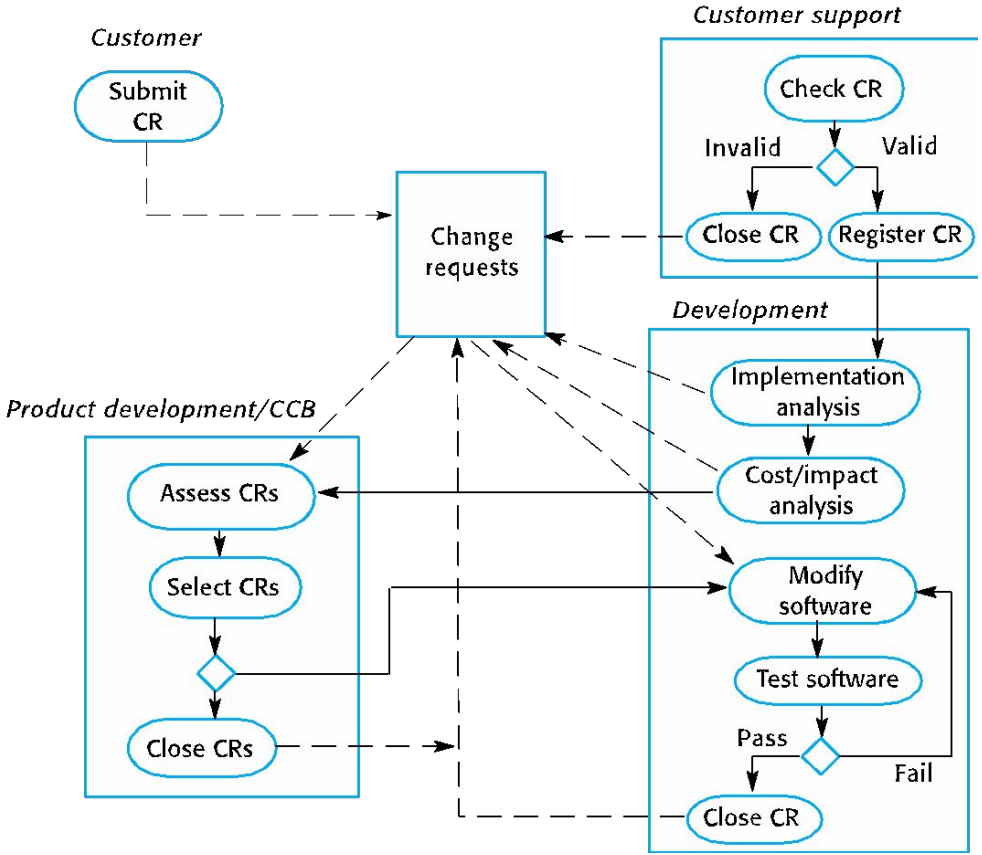
- Developer pulls latest code
- Creates feature branch
- Makes changes locally
- Runs tests locally
- Commits and pushes
- CI pipeline runs automatically
- Code review via pull request
- Merge to main after approval
- Automatic deployment to staging

Change management

Change management

- Organizational needs and requirements change during the lifetime of a system, bugs have to be repaired and systems have to adapt to changes in their environment.
- Change management is intended to ensure that system evolution is a managed process and that priority is given to the most urgent and cost-effective changes.
- The change management process is concerned with analyzing the costs and benefits of proposed changes, approving those changes that are worthwhile and tracking which components in the system have been changed.

The change management process



A partially completed change request form (a)

Change Request Form

Project: SICSA/AppProcessing **Number:** 23/02
Change requester: I. Sommerville **Date:** 20/07/12
Requested change: The status of applicants (rejected, accepted, etc.) should be shown visually in the displayed list of applicants.

Change analyzer: R. Looek **Analysis date:** 25/07/12
Components affected: ApplicantListDisplay, StatusUpdater

Associated components: StudentDatabase

A partially completed change request form (b)

Change Request Form

Change assessment: Relatively simple to implement by changing the display color according to status. A table must be added to relate status to colors. No changes to associated components are required.

Change priority: Medium
Change implementation:
Estimated effort: 2 hours
Date to SGA app. team: 28/07/12 **CCB decision date:** 30/07/12
Decision: Accept change. Change to be implemented in Release 1.2
Change implementor: **Date of change:**
Date submitted to QA: **QA decision:**
Date submitted to CM:
Comments:

Factors in change analysis

- The consequences of not making the change
- The benefits of the change
- The number of users affected by the change
- The costs of making the change
- The product release cycle

Change management and agile methods

- In some agile methods, customers are directly involved in change management.
- The propose a change to the requirements and work with the team to assess its impact and decide whether the change should take priority over the features planned for the next increment of the system.
- Changes to improve the software improvement are decided by the programmers working on the system.
- Refactoring, where the software is continually improved, is not seen as an overhead but as a necessary part of the development process.

Software Build of Materials (SBOM)

What is a Software Bill of Materials (SBOM)?

- A **Software Bill of Materials** (SBOM) is a detailed inventory of all software components, libraries, and modules—both open-source and proprietary—that are used in a software product.
  - It’s like a "parts list" in manufacturing but for software.

- **Purpose:** The SBOM provides transparency about what's inside a software package. It allows development teams, security analysts, and legal teams to identify dependencies, potential vulnerabilities, and licensing constraints.

  - **Example Analogy:** In the same way a car manufacturer tracks every part that goes into a vehicle, an SBOM helps software manufacturers track every code dependency.

- **Typical Contents:** Package name, version, origin, checksum, license, relationship to other components.

Why SBOM Matters – Security, Compliance & Traceability

- **Security Benefits:**
  - When a vulnerability (e.g., Log4Shell) is discovered, SBOMs allow organizations to quickly identify if their software includes the vulnerable component.
  - Enables proactive patch management and risk mitigation by identifying transitive dependencies.
- **Compliance & Licensing:**
  - Helps track open-source components and their licenses (e.g., GPL, MIT, Apache) to ensure legal compliance.
  - Critical for avoiding legal risks from unintentional use of incompatible licenses.
- **Traceability & Audit Readiness:**
  - SBOMs make it easier to answer questions like: “Which version of libxyz are we using?” or “When did this component get introduced?”
  - Regulatory bodies (e.g., in the US) are starting to mandate SBOMs for government and critical infrastructure software.

SBOM: Standards

- **Standards:**
  - **SPDX (Software Package Data Exchange):** An open standard by the Linux Foundation. Widely adopted and supported by many tools.
  - **CycloneDX:** A lightweight SBOM format by the OWASP foundation, suitable for security-focused workflows.
  - **SWID (Software Identification Tags):** ISO standard mainly used in enterprise asset management.

SBOM: Tools

- **Tooling Support:**
  - **Syft:** CLI tool to generate SBOMs in multiple formats.
    - Example: syft myapp:latest -o cyclonedx-json
  - **Trivy:** Can scan containers for vulnerabilities and also output SBOMs.
  - **OWASP Dependency-Check:** Performs vulnerability scanning and generates reports.
  - **GitHub's Native SBOM Integration:** GitHub Actions can now auto-generate SBOMs during CI builds.

Example: SBOM Entry (CycloneDX)

```
{
  "bomFormat": "CycloneDX",
  "components": [
    {
      "name": "log4j",
      "version": "2.14.1",
      "purl": "pkg:maven/org.apache.logging.log4j/log4j-core@2.14.1",
      "licenses": [ {"license": {"id": "Apache-2.0"}} ]
    }
  ]
}
```



Software Engineering

Thank You

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.

Attribution and Licensing

- Parts of this slideset include content adapted from the CMU 17-313: Foundations of Software Engineering course offered by Carnegie Mellon University.
- This material is used under the terms of the MIT License.
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.

- Parts of the slideset include content derived from various Internet reports and articles.

- Parts of the slideset include content derived from other university syllabus and class material.